

RWLang webalkalmazás-fejlesztőknek

A modellközpontú szemlélettől az első alkalmazáson át productionig

RWLang projekt

V1 – 2026. szeptember

Előszó

Ez a könyv webalkalmazás-fejlesztőknek szól, akik most találkoznak először az RWLang-gal, és akár a modellközpontú alkalmazástervezéssel is. Nem feltételezi, hogy az olvasó ORM-ek, domain modellek vagy capability-alapú rendszerek fogalmaiban gondolkodik. Az első fejezet ezért nem szintaxissal kezd, hanem azzal, hogy *miből áll egy alkalmazás*: milyen üzleti adatokat kezel, milyen műveletek végezhetőek rajtuk, ki végezheti ezeket a műveleteket, és hol lép be mindez a HTTP világába.

A könyv ajánlott olvasási sorrendje tudatosan fokozatos. Először felépítjük a model- és capability-szemléletet, majd lefordítjuk és telepítjük az RWLang programokat. Ezután egy szándékosan kicsi, A–Z alkalmazáson végigmegyünk a modelltől a formon és adatbázison át a HTTP válaszig. Csak ezután bontjuk szét részletesen a nyelvi, adatbázis-, auth-, fájl- és böngészős témákat.

Amikor az olvasó már normál alkalmazást tud írni, következik a nagyobb projektek szervezése, majd egy wiki/CMS példa. A külső integráció és a tesztelés ezután jön, mert ezekhez már érdemes egy teljes alkalmazás fejével gondolkodni. A production részben kerül elő a megosztott Redis state és cache, a security hardening, az observability, a deployment és a disaster recovery. A könyv végén a CLI és a teljes `server.toml` referencia már nem tananyagként, hanem visszakereshető operátori segédletként szolgál.

Hogyan olvasd a könyvet?

Első olvasáskor ne próbáld megjegyezni az összes konfigurációs kulcsot vagy production limitet. Az első cél az, hogy három kérdésre mindig tudj válaszolni:

1. *Milyen typed adatot kezel ez a feature?*
2. *Milyen művelet olvassa vagy módosítja ezt az adatot?*
3. *Milyen authority vagy capability engedi ezt a műveletet?*

Ha ez a három kérdés tiszta, a route, a form, az SQL, az auth és a konfiguráció már ugyanannak a modellnek a különböző határai lesznek, nem egymástól független trükkök.

Öt visszatérő szó

A könyv néhány angol eredetű technikai szót következetesen használ. Már az elején érdemes ugyanazt érteni alattuk:

model fordító által ismert, névvel ellátott typed adatalak; nem ORM-osztály és nem automatikus adatbázisséma;

typed az adat alakja és típusa a fordító számára ismert, nem csak futás közben derül ki;

capability explicit hozzáférési lehetőség egy erőforráshoz vagy érzékeny művelethez, például Db, Transaction vagy AppFs;

policy a külső határon érvényes szabály, például auth, rate limit, CORS vagy trusted proxy;
runtime a lefordított RWLang programot végrehajtó szerveroldali környezet, amely a limiteket és capability-határokat is kikényszeríti.

A későbbi fejezetek ezeket pontosítják, de a fenti jelentés első olvasásra elég.

A könyv konvenciói

- A `main.rw` az alkalmazás belépési pontja.
- A production konfiguráció elsődleges felülete a trusted TOML fájl.
- Secret érték nem kerül parancssorba vagy plaintext TOML mezőbe; a konfiguráció `*_file` hivatkozásokat használ.
- SQL szöveg compiler-owned, felhasználói érték csak typed bind paraméterként kerülhet lekérdezésbe.
- HTML-ben a `{ ... }` interpoláció escape-elt; raw HTML escape hatch nincs.
- Üzleti módosítás transactionben történik.

A könyv a V1 surface-t célozza. Ha egy capability nincs benne ebben a surface-ben, azt nem kerülőúttal kell megvalósítani, hanem külön nyelvi/runtime feature-ként.

Tartalomjegyzék

Előszó	ii
I. Első lépések: gondolkodásmód és működő alkalmazás	1
1. Hogyan gondolkodjunk egy RWLang alkalmazásról?	2
1.1. Először a fogalmak, utána az oldalak	2
1.2. Mit jelent a <code>model</code> ?	3
1.3. A <code>model</code> a rétegek közti szerződés	4
1.4. Az üzleti fogalomhoz műveletek tartoznak	4
1.5. <code>Capability</code> : a hozzáférés is része a modellnek	5
1.6. A <code>route</code> nem az üzleti modell	5
1.7. Egy egyszerű tervezési módszer	6
1.8. Mit vigyél tovább a következő fejezetekbe?	6
2. Telepítés, binárisok és futtatási topológiák	7
2.1. Build a forrásból	7
2.2. A binárisok telepítési helye	8
2.3. Első alkalmazás	9
2.4. A konfigurációból elsőre csak a minimum kell	9
2.5. Production process és reverse proxy	10
2.6. Futtatási topológiák részletesen	10
2.7. Közvetlen, egydomaines HTTPS	10
2.8. Apache reverse proxy mögött	11
2.9. Nginx reverse proxy mögött	12
2.10. Ajánlott production könyvtárstruktúra	12
2.11. Első ellenőrzés	13
3. A-tól Z-ig: egy minimális webalkalmazás	14
3.1. A könyvtár létrehozása	14
3.2. Az adatbázis-séma	14
3.3. A <code>model</code> és a lekérdezések	15
3.4. A GET oldal	15
3.5. A POST action	16
3.6. A <code>route</code> -ok és a validáció	16
3.7. A teljes alkalmazás egy helyen	17
3.8. SQLite kapcsolat developmenthez	17
3.9. Forrásellenőrzés és migration	17
3.10. Minimális development konfiguráció	18
3.11. Az első HTTP kérés	18

3.12.	Mit történik hibás inputnál?	18
3.13.	Developmentből production felé	19
3.14.	Mit tanultunk ebből az egy példából?	19
II.	Alap webalkalmazás-fejlesztés	20
4.	Nyelvi alapok és HTTP-határok	21
4.1.	Forrásstruktúra és statement-határok	21
4.2.	Typed számítás webalkalmazás-kódban	23
4.3.	Domainértékek és handlerkód	25
4.4.	HTTP- és prezentációs határ	26
5.	Adatbevitel: typed SQL, CRUD, formok és PRG	29
5.1.	Typed SQL és CRUD	29
5.2.	Formok, validáció és PRG	30
6.	Adatbázis mélyebben: tranzakció, migráció és konkurencia	32
6.1.	Három külön felelősség	32
6.2.	A query visszatérési típusa szerződés	33
6.3.	Bindolj, ne SQL-t építs	34
6.4.	Olvasás és mutáció authority-határa	34
6.5.	Optimistic locking: ne írd felül más munkáját	35
6.6.	Indexet a hozzáférési mintához tervezz	36
6.7.	Migration: a séma a release része	36
6.8.	Expand/contract: tartsd meg a rollback lehetőségét	37
6.9.	Adatbázis-hiba kezelése	38
6.10.	SQLite, PostgreSQL vagy MariaDB?	38
6.11.	Backup szempont adatbázis-fejlesztőként	38
6.12.	Egy CMS adatbázis-rétegének mintája	39
6.13.	Fejezet végi ellenőrzőlista	39
7.	Autentikáció, session és jogosultságkezelés	40
7.1.	A teljes login útvonal	40
7.2.	Local auth: külön identity adatbázis	41
7.3.	Account lifecycle operátori CLI-val	41
7.4.	Session invalidation: auth_generation	42
7.5.	Session store: memory vagy Redis	42
7.6.	Cookie policy	42
7.7.	Logout	42
7.8.	LDAP / LDAPS vállalati identityhez	43
7.9.	TOTP és recovery code	43
7.10.	Route authorization	44
7.11.	A principal használata az oldalon	44
7.12.	Route role nem mindig elég	44
7.13.	Login rate limiting	45
7.14.	CSRF, Origin és Fetch Metadata	45
7.15.	Auth audit	45
7.16.	Local auth vagy LDAP?	46
7.17.	Rich text és autentikált CMS	46
7.18.	Production ellenőrzőlista	46

8.	Fájlok, feltöltés és médiatár	47
8.1.	A mentális modell: három külön dolog	47
8.2.	Általános fájlfeltöltés: Upload	47
8.3.	AppFs: az alkalmazás írható fájltere	48
8.4.	Mi történik egy feltöltés során?	48
8.5.	Képfeltöltés: preferáld az Image típust	49
8.6.	Byte-limit és pixel-limit nem ugyanaz	49
8.7.	Kép referencia az adatbázisban	49
8.8.	Biztonságos megjelenítés HTML-ben	50
8.9.	A beépített media endpoint	50
8.10.	CMS-példa: hero kép feltöltése	50
8.11.	Adatbázis és AppFs együtt alkothat üzleti állapotot	51
8.12.	Statikus asset és feltöltött fájl nem ugyanaz	51
8.13.	Mit ne csinálj?	51
8.14.	A jelenlegi media surface határai	51
8.15.	Fejlesztői ellenőrzőlista feltöltéshez	51
9.	JSON API, CORS és böngészős kliensintegráció	52
9.1.	Az első typed JSON endpoint	52
9.2.	Adatbázisból JSON lista	53
9.3.	Typed JSON POST body	54
9.4.	Content negotiation és a 406 válasz	54
9.5.	API hibák	55
9.6.	Same-origin: az alapértelmezett és legegyszerűbb modell	55
9.7.	CORS: exact origin allowlist	56
9.8.	Preflight	56
9.9.	Credentialed CORS session-cookie-val	56
9.10.	CSRF JSON API-n	57
9.11.	Teljes böngészős POST folyamat	57
9.12.	Reverse proxy és origin	58
9.13.	Mit ne tegyél?	58
9.14.	Melyik architektúrát válaszd?	58
9.15.	Kipróbálható példa	58
10.	Statikus assetek és frontend-integráció	59
10.1.	Három külön fájlélekciklus	59
10.2.	A static root konfigurálása	60
10.3.	Fingerprint: a hosszú cache kulcsa	60
10.4.	Egy minimális asset build pipeline	61
10.5.	ETag és conditional GET	61
10.6.	Precompressed Brotli és gzip	61
10.7.	MIME, nosniff és path confinement	62
10.8.	CSS, kép és font az RWLang HTML-ben	62
10.9.	Fontos V1-határ: JavaScript kiszolgálható, de RWLang HTML-ben nem fut	62
10.10.	RWLang szolgálja ki vagy Apache/Nginx?	63
10.11.	Nginx minta közvetlen static kiszolgálásra	63
10.12.	Apache minta közvetlen static kiszolgálásra	64
10.13.	Deploy: asset és alkalmazáskód ugyanahhoz a release-hez tartozzon	64
10.14.	Ellenőrzés curl-lel	65
10.15.	Gyakorlati döntési szabály	65

11. Nagyobb alkalmazások szervezése	66
11.1. Három külön kérdés	66
11.2. A <code>mod</code> nem szöveges <code>include</code>	66
11.3. Mikor válts feature-alapú szervezetre?	67
11.4. Domain object: üzleti névtér, nem OOP-osztály	67
11.5. A dependency továbbra is explicit	69
11.6. Miért maradjon a route top-level?	70
11.7. Ajánlott névkonvenció	70
11.8. Egy CMS lehetséges szervezete	70
11.9. Mit ne csinálj?	71
11.10. Refaktorálási út: egy fájlból nagyobb alkalmazás	71
 III. Gyakorlati alkalmazások	 72
12. Gyakorlati alkalmazások: wiki és CMS	73
12.1. Egyszerű wiki	73
12.2. CMS és híroldal	74
 IV. Haladó integráció és minőség	 77
13. Külső szolgáltatások és egress policy	78
13.1. Először a V1-határ	78
13.2. Miért veszélyes a tetszőleges URL?	78
13.3. Named target: a hálózati capability konfigurációja	79
13.4. A policy több rétegen ellenőriz	79
13.5. IPv4 és IPv6 ugyanahhoz a policyhoz tartozik	80
13.6. Méret- és időkorlátok	80
13.7. Secret nem konfigurációs string	80
13.8. GET és JSON POST a trusted adapterben	81
13.9. Hibamodell	81
13.10. Retry és idempotencia	81
13.11. Szinkron kérés vagy háttér munka?	82
13.12. Mit jelent ez az RWLang alkalmazásfejlesztőnek?	82
13.13. Tesztelési minimum	82
13.14. A fejezet lényege	82
 14. Tesztelés, hibamodellek és fejlesztői diagnosztika	 83
14.1. Az első kapu: <code>rwlang-cli check</code>	83
14.2. A negatív teszt ugyanolyan értékes, mint a pozitív	84
14.3. A hibák négy külön világa	84
14.4. Konfigurációt külön tesztelj	84
14.5. Migráció: status, verify, apply, verify	85
14.6. Üzleti teszt: állapotot bizonyíts, ne implementációt	85
14.7. Adatbázis teszt: SQLite nem bizonyít mindent	86
14.8. Auth és authorization tesztmátrix	86
14.9. Fájl, média, API és cache tesztek	86
14.10. HTTP hibakód nem egyenlő programhibával	87
14.11. Request ID-val hibakeresni	88
14.12. Deployment smoke teszt	88

14.13.	Mit jelent a repository <code>verify.sh</code> ?	89
14.14.	Ajánlott fejlesztői ciklus	89
14.15.	Hibakeresési döntési sorrend	89
14.16.	Minimum release checklist alkalmazáshoz	90
V. Production és üzemeltetés		91
15. Cache, Redis, rate limiting és többpéldányos működés		92
15.1.	Miért nem általános Redis kliens?	92
15.2.	Production Redis konfiguráció	93
15.3.	Shared session több szerverpéldánnyal	93
15.4.	Public page cache	94
15.5.	Mit szabad public cache-be tenni?	94
15.6.	Operator által szabott cache-határok	94
15.7.	Cache invalidation: generation, nem kulskeresés	95
15.8.	Stampede védelem és annak határa	95
15.9.	Redis hiba cache-elt request közben	95
15.10.	Route rate limiting	95
15.11.	Rate-limit scope-ok	96
15.12.	Fixed-window modell több instance-nál	96
15.13.	Memory backend csak fejlesztési kivétel	97
15.14.	Egy tipikus többpéldányos topológia	97
15.15.	Rolling restart és cache generation	98
15.16.	Mit monitorozz?	98
15.17.	Redis recovery szerepe	98
15.18.	Gyakorlati példa: cache-elt híroldal publikálással	98
15.19.	Ellenőrzőlista	99
16. Security hardening: a trust boundary összekapcsolása		100
16.1.	Az öt legfontosabb trust boundary	100
16.2.	HTTP edge: Host, proxy és parser	101
16.3.	TLS és secret kezelés	101
16.4.	Identity: authentication nem authorization	102
16.5.	Session, CSRF és CORS együtt	102
16.6.	SQL és üzleti mutáció	103
16.7.	Upload, média és AppFs	103
16.8.	Redis és shared runtime state	103
16.9.	Outbound egress és SSRF	103
16.10.	Resource limitek is security kontrollók	104
16.11.	Log hygiene és audit integrity	104
16.12.	Process és filesystem hardening	104
16.13.	Dependency és build supply-chain	105
16.14.	Production security gate	105
16.15.	Gondolkodási minta code reviewhoz	105
17. Naplózás, audit, metrics és hibakeresés		107
17.1.	A három naplóosztály szerepe	107
17.2.	Request ID: egy kérés végigkövetése	108
17.3.	Mit szabad és mit tilos logolni?	108
17.4.	Security audit és HTTP elutasítások	109

17.5.	Business audit: tartós üzleti változástörténet	109
17.6.	Prometheus metrics külön management socketen	110
17.7.	Apache vagy Nginx mellett ki scrape-elje a metricset?	111
17.8.	Health: liveness és readiness nem ugyanaz	111
17.9.	Logrotate és SIGHUP	112
17.10.	Mi történik, ha a logfájl nem írható?	112
17.11.	Gyakorlati production hibakeresési folyamat	113
17.12.	Minimum monitoring baseline	113
17.13.	Retention, SIEM és auditvédelem	113
17.14.	Ellenőrzőlista	114
18.	Production deployment és service lifecycle	115
18.1.	Release útvonal	115
18.2.	systemd service	115
18.3.	Controlled restart és readiness	116
18.4.	Migration és rollback nem ugyanaz	116
18.5.	Recovery gate a deployment előtt	116
18.6.	Deploy utáni minimum ellenőrzés	116
19.	Backup, restore és disaster recovery	118
19.1.	Először készíts state inventoryt	118
19.2.	Mi nem üzleti source of truth?	118
19.3.	RPO és RTO: a technikai mentés előtt üzleti döntés	119
19.4.	A V1 biztonságos baseline: kontrollált maintenance backup	119
19.5.	Application DB: a backend saját backup eszközét használd	119
19.6.	A local-auth DB külön security backup	120
19.7.	AppFs: a teljes data root tartós állapot lehet	120
19.8.	Minden recovery ponthoz tartozzon manifest	121
19.9.	Restore drill: a backup bizonyítása	121
19.10.	Upgrade előtt legyen recovery döntés	121
19.11.	Expand/contract tartja nyitva a gyors rollback ablakot	122
19.12.	Rollback és restore két külön művelet	122
19.13.	Tiltott rövidítések	123
19.14.	Production recovery checklist	123
VI.	Referencia és operátori gyorssegédlet	124
20.	Az RWLang CLI és a napi fejlesztői/üzemeltetői workflow	125
20.1.	Melyik feladathoz melyik program?	125
20.2.	A CLI nem a szerver alternatívája	126
20.3.	Az első parancs: <code>rwlang-cli check</code>	126
20.4.	Forrásfájl futtatás és telepített CLI	126
20.5.	Migration workflow	127
20.6.	Local-auth adminisztráció	127
20.7.	TOTP adminisztráció különösen érzékeny művelet	128
20.8.	A secret fájl is input contract	129
20.9.	Server preflight: a CLI után a szerver saját ellenőrzése jön	129
20.10.	Napi fejlesztői ciklus	129
20.11.	Application release workflow	130
20.12.	Az RWLang platform release workflowja más	130

20.13.	Automatizálás: read-only és mutáló lépések legyenek külön	130
20.14.	Gyakori hibák	131
20.15.	Egy oldalnyi operátori runbook	131
21.	A server.toml konfigurációs referencia	133
21.1.	Betöltés, precedence és fail-closed szabályok	133
21.2.	Hogyan olvasd a táblákat?	134
21.3.	[server] és [tls]	134
21.4.	[database], [redis] és [auth]	135
21.5.	[web]	135
21.6.	[storage] és [static_assets]	136
21.7.	[lifecycle], [observability] és [logging]	136
21.8.	[rate_limit] és [cache]	137
21.9.	[limits]	137
21.10.	[cgroup]	138
21.11.	Production baseline	138
21.12.	Automatikus source reload	139
21.13.	Változtatási workflow	139
22.	Fejlesztői referencia és továbblépés	140
22.1.	Melyik témát hol keresd?	140
22.2.	Napi fejlesztési ellenőrzőlista	141
A.	Függelékek és production checklist	142
A.1.	Ajánlott production könyvtárstruktúra	142
A.2.	Apache reverse proxy teljes minimálminta	143
A.3.	Nginx reverse proxy teljes minimálminta	143
A.4.	systemd service minta	144
A.5.	logrotate minta	145
A.6.	Release előtti production gate	146
A.7.	Gyors hibadiagnosztikai térkép	146
A.8.	Mit vigyél magaddal productionbe?	147

I. rész

Első lépések: gondolkodásmód és működő alkalmazás

1. fejezet

Hogyan gondolkodjunk egy RWLang alkalmazásról?

Mielőtt telepítünk, route-ot írunk vagy adatbázist kötünk az alkalmazáshoz, érdemes tisztázni egy szemléleti kérdést. Sok webes technológiában természetes kiindulópont az oldal, az endpoint vagy a controller: „*kell egy /products URL, írjunk hozzá handlert*”. RWLangban hasznosabb előbb azt megkérdezni: „*milyen üzleti adat létezik, milyen alakban, milyen műveletek végezhetők rajta, és milyen határok között?*”

Ez nem akadémiai különbség. A nyelv több biztonsági és helyességi garanciája abból származik, hogy az alkalmazás fontos adatainak, bemenetei, lekérdezései és erőforrás-hozzáférései explicit szerződések.

1.1. Először a fogalmak, utána az oldalak

Vegyünk egy egyszerű terméklistát. A feladatot lehetne így kezdeni:

Kell egy GET oldal, egy POST endpoint és egy SQL INSERT.

RWLangban célszerűbb így gondolkodni:

1. Van egy üzleti fogalom: *Product*.
2. A programban ennek ismert adatainak van: `id`, `name`, `price`.
3. Vannak olvasó és módosító műveletek: listázás és létrehozás.
4. A módosítás adatbázis-tranzakciót igényel.
5. A HTTP réteg typed inputot ad a műveletnek, és policy-t rendel hozzá.
6. A megjelenítés már az így kapott typed adatot rendereli HTML-be vagy JSON-ba.

A gondolkodás iránya tehát nagyjából:

```
üzleti fogalom
  -> typed adatainak
    -> muveletek
      -> capability-k es policy-k
        -> HTTP route
          -> HTML / JSON
```

Nem minden alkalmazásnál kell ezt papíron végigírni. A sorrend mégis segít elkerülni, hogy az üzleti szabályok véletlenül route-okba, template-ekbe vagy kézzel összefűzött SQL-be szóródjanak szét.

1.2. Mit jelent a model?

A könyvben két közeli fogalommal találkozunk:

- **adatmodell vagy domain modell:** annak emberi leírása, milyen üzleti fogalmak és kapcsolatok léteznek az alkalmazásban;
- **model deklaráció:** az RWLang konkrét, fordító által ismert adatalakja.

Például:

```
model Product {  
  id: Int  
  name: String  
  price: Int  
}
```

Ez azt mondja a fordítónak, hogy egy `Product` rekordnak pontosan ilyen mezői és típusai vannak. A typed SQL query eredményalakja ehhez a szerződéshez igazodik, és ugyanez az érték később HTML- vagy JSON-kimenetben is használható.

A model nem automatikus adatbázisséma

A fenti deklaráció önmagában nem hoz létre `products` táblát. Az adatbázissémát migration hozza létre, például:

```
CREATE TABLE products (  
  id INTEGER PRIMARY KEY,  
  name TEXT NOT NULL,  
  price INTEGER NOT NULL  
);
```

A két réteg kapcsolatban áll, de nem ugyanaz. Ez tudatos szétválasztás:

- a migration az adatbázis tartós sémájának életciklusa;
- a `model` az alkalmazáskód typed adatszerződése;
- a query mondja meg explicit módon, hogyan lesz adatbázissorokból model rekord.

Ezért RWLangban nincs szükség arra a feltételezésre, hogy minden model automatikusan egy ORM-entitás vagy egy azonos nevű tábla.

A model nem mutable objektum

A `Product` nem klasszikus objektumorientált osztály. Nincs mögötte rejtett `save()` metódus, implicit adatbázis-kapcsolat vagy mutable objektuméletciklus. Ha adatot akarunk módosítani, explicit műveletet és explicit erőforrást használunk.

Például az olvasás:

```
query fn listProducts(db: Db) -> Result<List<Product>, DbError> sql {  
  SELECT id, name, price  
  FROM products  
  ORDER BY id DESC  
  LIMIT 50  
}
```

A módosítás pedig külön, tranzakcióhoz kötve jelenik meg:

```

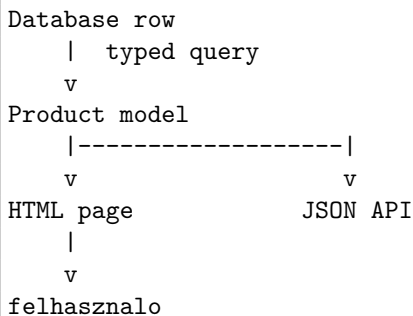
query fn createProduct(
  tx: Transaction,
  name: String,
  price: Int
) -> Result<Void, DbError> sql {
  INSERT INTO products(name, price)
  VALUES (:name, :price)
}

```

A függvény aláírásából látszik, hogy melyik művelet csak olvas, és melyik kap módosításra alkalmas tranzakciót.

1.3. A model a rétegek közti szerződés

Érdemes a `model`-re nem pusztán „adatbázisos struktúraként”, hanem a rétegek között mozgó typed szerződésként gondolni.



A fordító emiatt tud olyan hibákat korán megfogni, amelyeket dinamikus rendszerekben csak futás közben vennénk észre. Ha egy query `Product`-ot ígér, a kiválasztott/returned mezőknek a model shape-jéhez kell igazodniuk. Nem az a cél, hogy a program „majd valahogy” értelmezze az eredményt.

Ez a szemlélet később az object authorizationnál és a JSON API-nál is visszatér: a rendszer ismert, typed rekordokkal dolgozik ahelyett, hogy tetszőleges, futás közben változó objektumokat adna tovább.

1.4. Az üzleti fogalomhoz műveletek tartoznak

Egy model önmagában ritkán érdekes. A valódi alkalmazásban műveletek tartoznak hozzá:

- `Product.list`;
- `Product.byId`;
- `Product.create`;
- egy CMS-ben például `Article.publish`;
- egy wikiben `WikiPage.update`.

Kis programban ezek lehetnek top-level függvények. Nagyobb programban a később bemutatott `object domain namespace` fogja őket egy üzleti fogalom köré rendezni.

Fontos különbség:

Elem	Kérdés, amelyre válaszol
<code>model</code>	Milyen alakú adat a <code>Product</code> ?
<code>query fn</code>	Hogyan olvassuk vagy módosítjuk a tartós adatot?
<code>object</code>	Mely üzleti fogalomhoz tartoznak ezek a műveletek?
<code>page/action</code>	Hogyan kapcsolódik az üzleti művelet a webes handlerhez?
<code>route</code>	Milyen HTTP útvonalon, inputtal és policy-val érhető el?

1.5. Capability: a hozzáférés is része a modellnek

RWLangban nem csak az adat típusa explicit, hanem az is, milyen erőforráshoz fér hozzá egy művelet. Egy függvény nem „valahonnan” kap adatbázist, fájlrendszert vagy current usert.

```
page fn home(ctx: PageContext, db: Db)
  -> Result<Html, PageError> {
    let products = listProducts(db)?;
    return Ok(html {
      <ul>
        @for product in products {
          <li>{{ product.name }}</li>
        }
      </ul>
    });
  }
```

A `db: Db` az aláírásban látható. Fájlkezelésnél `AppFs` capability, módosító adatbázis-műveletnél `Transaction`, autentikált műveletnél pedig az `auth` policy és a hozzá tartozó typed kontextus válik relevánssá.

Ebből egy hasznos tervezési szabály következik:

Ha egy műveletnek érzékeny erőforrás kell, annak a kódból és a policy-ból látszania kell.

1.6. A route nem az üzleti modell

A route fontos, de a rendszer külső határa. Például:

```
route create POST "/products"
  form name<String> price<Int>
  validate name length 1 100 price range 0 100000000
  => create;
```

Ez azt mondja meg, hogyan léphet be egy HTTP kérés a rendszerbe: melyik metóduson és pathon, milyen typed inputtal, milyen validációval és később milyen `auth/cache/rate-limit` policy-val.

Az üzleti fogalom azonban nem a `/products` URL. Ugyanaz a `Product` később HTML oldalon, JSON API-ban, admin felületen vagy más route mögött is használható.

Ezért nagyobb alkalmazásban különösen hasznos a következő határ:

```
domain/model      : mi az adat es milyen muvelet ertelmes rajta?
HTTP route        : ki, hogyan es milyen policy-val hivhatja?
runtime config    : milyen infrastruktura-es eroforras-hatarok kozott futhat?
```

1.7. Egy egyszerű tervezési módszer

Amikor új feature-t kezdesz, a következő öt kérdés jó kiindulópont:

1. **Mi az üzleti főnév?** – Product, Article, Invoice, WikiPage.
2. **Milyen typed adatra van ténylegesen szükség?** – ne automatikusan a teljes adatbázissort másold át.
3. **Mely műveletek olvasnak, és melyek módosítanak?**
4. **Milyen capability kell hozzájuk?** – Db, Transaction, AppFs, auth, később hálózati egress.
5. **Milyen HTTP/policy felületen tesszük elérhetővé?** – route, input, validáció, auth, cache, rate limit.

Egy egyszerű CRUD esetén ez pár perc gondolkodás. Egy CMS-nél vagy ügyviteli rendszernél viszont ez akadályozza meg, hogy az alkalmazás néhány hónap után route-ok, SQL-ek és implicit jogosultsági feltételek nehezen követhető halmazává váljon.

1.8. Mit vigyél tovább a következő fejezetekbe?

Nem kell minden alkalmazást „domain-driven design” terminológiával megtervezni, és nem kell minden hárommezős rekordból külön object hierarchy-t építeni. A könyvben használt modell-szemlélet ennél egyszerűbb:

- nevezd meg a fontos üzleti adatot;
- add meg a typed alakját;
- az adatmozgást explicit query-kkel írd le;
- a módosítást explicit tranzakcióhoz kösd;
- az erőforrás-hozzáférést capability-ként tedd láthatóvá;
- a HTTP-t és a security policy-t külső határként kezeld.

A következő fejezetekben először lefordítjuk, telepítjük és csak a szükséges minimumon konfiguráljuk a rendszert, majd az A–Z példában ugyanezt a szemléletet egy működő Product alkalmazáson végigkövetjük. Utána a könyv minden egyes részét külön mélyíti el.

2. fejezet

Telepítés, binárisok és futtatási topológiák

2.1. Build a forrásból

A kiinduló release forráscsomag Rust workspace. A két telepíthető program előállításához a szokásos release build:

```
cargo build --locked --release
```

A repository saját teljes öellenőrzése a `./verify.sh`; ezt akkor futtasd, amikor magát az RWLang forrásfát validárod vagy release-eled. Egy alkalmazás használatához nem ez az első lépés.

A `-locked` biztosítja, hogy a build a release-hez tartozó `Cargo.lock` függőséggráfot használja. Ha csak a két futtatható program kell:

```
cargo build --locked --release -p rclang-server -p rclang-cli
```

A build után a két fontos program:

```
target/release/rclang-server  
target/release/rclang-cli
```

- **rclang-server**: az RWLang alkalmazás fordítását és HTTP(S) kiszolgálását végző hosszú életű service process;
- **rclang-cli**: operátori/adminisztratív program, például auth- és migrációs műveletekhez.

A két bináris szerepe szándékosan különbözik: a reverse proxy kizárólag a **rclang-server** processzhez kapcsolódik. A **rclang-cli** nem webserver és nem kell publikus vagy proxy mögötti listener.

Fejlesztés közben elegendő lehet:

```
cargo check --workspace  
cargo test --workspace
```

2.2. A binárisok telepítési helye

Production gépen ne a Cargo `target/` könyvtárából futtasd tartósan a szolgáltatást. Az RWLang helyben telepített, nem az operációs rendszer csomagkezelője által kezelt program, ezért a könyv következetesen a `/usr/local` hierarchiát használja: a futtatható programok a `/usr/local/bin`, a géphez tartozó konfiguráció pedig a `/usr/local/etc/rwlang` alá kerül. Ez elválasztja az RWLang telepítését a disztribúció által kezelt `/usr/bin` és `/etc` fájloktól.

Egy egyszerű, jól auditálható elrendezés:

```
/usr/local/bin/rwlang-server
/usr/local/bin/rwlang-cli
/usr/local/etc/rwlang/server.toml
/srv/rwlang/current/
/srv/rwlang/data/
/run/secrets/rwlang/
/var/log/rwlang/
```

Telepítés például:

```
sudo install -d -m 0755 /usr/local/bin
sudo install -d -m 0755 /usr/local/etc/rwlang
sudo install -m 0755 target/release/rwlang-server /usr/local/bin/rwlang-server
sudo install -m 0755 target/release/rwlang-cli /usr/local/bin/rwlang-cli
sudo install -m 0644 config/server.toml.sample \
  /usr/local/etc/rwlang/server.toml
```

A `/usr/local/bin` a szokásos rendszer-PATH része, ezért külön symlinkre nincs szükség. A könyv példáiban a valódi binárisneveket, `rwlang-server` és `rwlang-cli`, használjuk.

Debian csomag esetén szándékosan más az elrendezés

A fenti elrendezés a kézi/forrásból telepítésre vonatkozik. Ha az RWLang Debian csomagból kerül a gépre, a fájlokat a `dpkg` kezeli, ezért a Debian filesystem-konvenció érvényes, nem a lokális adminisztrátori `/usr/local` prefix:

```
/usr/bin/rwlang-server
/usr/bin/rwlang-cli
/etc/rwlang/server.toml
/usr/lib/systemd/system/rwlang.service
```

A csomag Debian/Ubuntu build gépen így készíthető és telepíthető:

```
make deb
sudo dpkg -i dist/rwlang_1.0.0-1_amd64.deb
```

A csomag szándékosan nem indítja el automatikusan a szolgáltatást: előbb telepítsd az alkalmazást és a secreteket, állítsd be az `/etc/rwlang/server.toml` fájlt, validáld, és csak ezután engedélyezd az `rwlang.service` unitot. Ez nem ellentmondás: kézi, adminisztrátor által birtokolt telepítéshez a `/usr/local` helyes, csomagkezelő által birtokolt fájlokhoz pedig a `/usr` és `/etc`. A teljes folyamatot a `docs/hu/36-debian-csomag.md` írja le.

2.3. Első alkalmazás

Ajánlott minimális projekt:

```
myapp/  
|-- main.rw  
|-- pages.rw  
`-- public/
```

main.rw:

```
mod pages;
```

pages.rw:

```
page fn home(ctx: PageContext) -> Result<Html, PageError> {  
    return Ok(html {  
        <main><h1>Hello RWLang</h1></main>  
    });  
}  
  
route home GET "/" => home;
```

Fejlesztői indítás közvetlenül a workspace-ből:

```
cargo run -p rwlang-server -- \  
  --app /abs/path/myapp/main.rw \  
  --listen 127.0.0.1:8080 \  
  --insecure-dev-cookies
```

Ugyanez a lefordított release binárisal:

```
/usr/local/bin/rwlang-server \  
  --app /abs/path/myapp/main.rw \  
  --listen 127.0.0.1:8080 \  
  --insecure-dev-cookies
```

Az `-insecure-dev-cookies` csak lokális developmentre való.

2.4. A konfigurációból elsőre csak a minimum kell

A könyv ezen pontján még nem cél a teljes `server.toml` megtanulása. Kezdként elég három dolgot szétválasztani:

- az *alkalmazáskód* mondja meg, milyen modellek, queryk, oldalak és route-ok léteznek;
- a *szerverkonfiguráció* mondja meg, hol hallgat a process, milyen külső erőforrásokat érhet el, és milyen operátori limitek érvényesek;
- a *secret fájlok* tartalmazzák azokat az érzékeny értékeket, amelyek nem kerülhetnek sem az alkalmazáskódba, sem plaintext TOML-ba.

Az A–Z fejezet egy minimális development konfigurációt ad. A production-specifikus kulcsokat akkor vezetjük be, amikor már van mit üzemeltetni; a teljes kulcs/default/sample referencia a könyv végén található. Első olvasáskor tehát ne próbáld a teljes konfigurációs felületet

memorizálni.

2.5. Production process és reverse proxy

Apache vagy Nginx mögött a tipikus adatút:

```
Internet
|
| HTTPS :443
v
Apache vagy Nginx
|
| HTTP 127.0.0.1:8080
v
/usr/local/bin/rwlang-server
|
+-- /usr/local/etc/rwlang/server.toml
+-- /srv/rwlang/current/main.rw
+-- DB / Redis / AppFs
```

A proxy nem indítja és nem helyettesíti az RWLang processzt. A `rwlang-server` külön service-ként fusson, például `systemd` alatt, a proxy pedig a loopback listenerre továbbítja a kéréseket. A `rwlang-cli` csak operátori parancsok idejére indul el.

Egyszerű `systemd` `ExecStart` részlet:

```
ExecStart=/usr/local/bin/rwlang-server \\  
--config /usr/local/etc/rwlang/server.toml
```

A process felhasználója ne legyen `root`; csak a szükséges alkalmazás-, secret-, data- és log-fájlokhoz kapjon jogosultságot. Ha a TLS-t Apache vagy Nginx terminálja, az RWLang listener maradjon loopbacken, például `127.0.0.1:8080`.

2.6. Futtatási topológiák részletesen

A binárisok helye és a processzek szerepe ugyanaz marad, akár az RWLang terminálja a TLS-t, akár Apache vagy Nginx áll előtte. A különbség a listener, a TLS-termináció és a trusted proxy határ konfigurációja.

2.7. Közvetlen, egydomaines HTTPS

Ha az RWLang szerver maga terminálja a TLS-t, a publikus listener lehet például 443:

```
[server]
app = "/srv/rwlang/current/main.rw"
listen = "0.0.0.0:443"
insecure_dev_cookies = false

[tls]
cert_file = "/run/secrets/rwlang/tls-fullchain.pem"
key_file = "/run/secrets/rwlang/tls-key.pem"
handshake_timeout_ms = 5000
http_redirect_listen = "0.0.0.0:80"
```

```
public_host = "www.example.com"
```

A `public_host` trusted konfigurációból jön. HTTP-ről HTTPS-re csak GET/HEAD kérés redirectelhető.

Mivel a közvetlen minta 80/443 portot használ, unprivileged systemd service esetén a processnek célzott `CAP_NET_BIND_SERVICE` capability kell. Ne futtasd emiatt rootként. Reverse proxy mögött, például 127.0.0.1:8080 listenerrel ez a capability nem szükséges.

2.8. Apache reverse proxy mögött

Ebben a felállásban az Apache kezeli a publikus TLS-t, az RWLang pedig csak loopback interfészen hallgat. Példa RWLang config:

```
[server]
app = "/srv/rwlang/current/main.rw"
listen = "127.0.0.1:8080"
insecure_dev_cookies = false

[tls]
public_host = "www.example.com"

[web]
trusted_proxy_cidrs = ["127.0.0.1/32", "::1/128"]
allow_missing_origin = false
cors_origins = []
cors_allow_credentials = false
```

Reverse proxy módban nincs RWLang oldali certificate/key, de a `tls.public_host` továbbra is szükséges: ez a böngésző által látott canonical host trusted forrása, amelyhez a Host és az Origin/Referer ellenőrzés kötődik. A production plain HTTP upstream csak loopback listeneren, explicit trusted proxy listával és ilyen public hosttal engedett. Minden normál proxied kérésnek trusted proxytól származó `X-Forwarded-Proto: https` (vagy ekvivalens szabványos `Forwarded: proto=https`) metadata-t kell hordoznia.

Az Apache VirtualHost minimális mintája:

```
<VirtualHost *:443>
    ServerName www.example.com

    SSLEngine on
    SSLCertificateFile /etc/letsencrypt/live/www.example.com/fullchain.pem
    SSLCertificateKeyFile /etc/letsencrypt/live/www.example.com/privkey.pem

    ProxyPreserveHost On
    ProxyPass          / http://127.0.0.1:8080/
    ProxyPassReverse   / http://127.0.0.1:8080/

    RequestHeader set X-Forwarded-Proto "https"
</VirtualHost>
```

Ehhez tipikusan a `proxy`, `proxy_http`, `headers`, `ssl` modulok szükségesek. A forwarding headereket az RWLang csak a `trusted_proxy_cidrs` tartományból fogadja el; ezért ne tegyél tág, például teljes internetet lefedő trusted proxy CIDR-t a configba.

A port 80-as VirtualHost feladata lehet kizárólag az Apache-s HTTPS redirect:

```
<VirtualHost *:80>
    ServerName www.example.com
    Redirect permanent / https://www.example.com/
</VirtualHost>
```

2.9. Nginx reverse proxy mögött

Az Nginx topológia ugyanazt a trust modellt használja: a publikus TLS-t az Nginx kezeli, az RWLang pedig loopbacken hallgat. Az RWLang oldali `server.toml` megegyezhet az Apache példában látottal.

Minimális Nginx server block:

```
server {
    listen 443 ssl;
    server_name www.example.com;

    ssl_certificate      /etc/letsencrypt/live/www.example.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/www.example.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header X-Forwarded-For $remote_addr;
    }
}
```

A példában egyetlen közvetlen reverse proxy van, ezért az `X-Forwarded-For` értéke a proxy által látott klienscím, az `X-Forwarded-Proto` pedig `https`. Ne engedd, hogy az internet felől érkező tetszőleges forwarding header authority legyen. Az RWLang csak a pontos `trusted_proxy_cidrs` tartományból érkező forwarding headert fogadja el.

A HTTP listener feladata lehet csak a HTTPS redirect:

```
server {
    listen 80;
    server_name www.example.com;
    return 301 https://$host$request_uri;
}
```

2.10. Ajánlott production könyvtárstruktúra

```
/usr/local/bin/rwlang-server
/usr/local/bin/rwlang-cli
/srv/rwlang/releases/2026-09-04.1/
/srv/rwlang/current -> /srv/rwlang/releases/2026-09-04.1
/srv/rwlang/data/
/usr/local/etc/rwlang/server.toml
/run/secrets/rwlang/
```

```
/var/log/rwlang/
```

A bináris, a release és a konfiguráció legyen runtime read-only. Az alkalmazás csak a deklarált data/log helyekre írjon.

2.11. Első ellenőrzés

Mielőtt forgalmat adnál a szerverre:

```
/usr/local/bin/rwlang-server \\  
--config /usr/local/etc/rwlang/server.toml \\  
--check-config
```

Ez listener nélkül ellenőrzi többek között a konfigurációt, a secret-file referenciákat, az alkalmazás fordíthatóságát és a TLS fájlokat.

3. fejezet

A-tól Z-ig: egy minimális webalkalmazás

Az előző fejezetekben felépítettük az RWLang model- és capability-szemléletét, majd lefordítottuk és telepítettük a rendszert. A teljes szerverkonfigurációt még nem kell ismernünk: ehhez a példához csak a minimális development beállításokat használjuk. Mielőtt a nyelv egyes részeit külön-külön részleteznénk, készítsünk el egy teljes, de szándékosan kicsi alkalmazást. A cél nem egy kész webshop, hanem annak megmutatása, hogyan áll össze egy RWLang webalkalmazás a forrásfájltól az adatbázison és a HTTP kérésen át a production telepítésig.

A példa egy mini terméklista. A kezdőoldal kilistázza az adatbázisban lévő termékeket, és ugyanott egy formmal új terméket is fel lehet venni. Ezzel egyetlen példában megjelenik a model, typed SQL, GET page, POST action, form validation, CSRF, transaction, migration, konfiguráció és a reverse proxy mögötti futtatás.

A repositoryban a kész példa itt található:

```
examples/a-z-products/  
  main.rw  
  migrations/0001_create_products.sql  
  server.toml.example  
  README.md
```

3.1. A könyvtár létrehozása

Fejlesztéshez legyen egy külön alkalmazáskönyvtárunk:

```
mkdir -p my-products/migrations  
cd my-products
```

Ebben a példában minden RWLang kód egyetlen `main.rw` fájlban marad. Nagyobb alkalmazásnál később `mod` fájlokra bontjuk.

3.2. Az adatbázis-séma

A `migrations/0001_create_products.sql` tartalma SQLite esetén:

```
CREATE TABLE products (
```



```
id INTEGER PRIMARY KEY AUTOINCREMENT,  
name TEXT NOT NULL,  
price INTEGER NOT NULL CHECK (price >= 0)  
);
```

Ez a migration hozza létre azt a táblát, amelyet az alkalmazás typed SQL lekérdezései használnak. Productionban a migráció külön deploy lépés; a server nem módosítja automatikusan a sémát induláskor.

3.3. A model és a lekérdezések

A `main.rw` elején a modellközpontú fejezetben megismert módon először a typed üzleti adatokat írjuk le:

```
model Product {  
  id: Int  
  name: String  
  price: Int  
}
```

A lista lekérdezése:

```
query fn listProducts(db: Db) -> Result<List<Product>, DbError> sql {  
  SELECT id, name, price  
  FROM products  
  ORDER BY id DESC  
  LIMIT 50  
}
```

A `List<Product>` azt mondja ki, hogy a query nulla vagy több `Product` sort adhat vissza. A `SELECT` oszlopai pontosan a model mezőinek nevét és sorrendjét követik.

Az `INSERT` transaction capabilityt kap:

```
query fn createProduct(  
  tx: Transaction,  
  name: String,  
  price: Int  
) -> Result<Void, DbError> sql {  
  INSERT INTO products(name, price)  
  VALUES (:name, :price)  
}
```

A `:name` és `:price` bind paraméter. Nem szövegkonkatenációval építünk SQL-t felhasználói inputból.

3.4. A GET oldal

A kezdőoldal lekéri a termékeket, megjelenít egy formot és kilistázza az eredményt:

```
page fn home(ctx: PageContext, db: Db) -> Result<Html, PageError> {  
  let products = listProducts(db)?;
```

```

return Ok(html {
    <main>
        <h1>Mini terméklista</h1>

        <form method="post" @action(create)>
            <input type="hidden" name="_csrf" value="{{ csrfToken }}">
            <label>
                Nev
                <input name="name" maxlength="100">
            </label>
            <label>
                Ár
                <input name="price">
            </label>
            <button type="submit">Hozzaadás</button>
        </form>

        <ul>
            @for product in products {
                <li>
                    #{{ product.id }} - {{ product.name }} - {{ product.price }}
                </li>
            }
        </ul>
    </main>
});
}

```

A dinamikus szövegek HTML-escape-elve kerülnek a válaszba. A form action nem kézzel írt URL, hanem az `@action(create)` helperből származik. A rejtett `_csrf` mező a POST kérés CSRF-védelméhez kell.

3.5. A POST action

A form sikeres beküldése transactionben ír az adatbázisba, majd redirectel vissza a kezdőoldalra:

```

action fn create(
    ctx: ActionContext,
    db: Db,
    name: String,
    price: Int
) -> Result<Redirect, PageError> {
    transaction db {
        createProduct(tx, name, price)?;
    }
    return Ok(redirect("/"));
}

```

Ez a Post/Redirect/Get minta. Ha a felhasználó frissíti a visszakapott oldalt, a böngésző nem küldi be újra automatikusan az INSERT-et.

3.6. A route-ok és a validáció

A fájl végén két route köti össze a HTTP felületet a handlerekkel:

```
route home GET "/" => home;

route create POST "/products"
  form name<String> price<Int>
  validate name length 1 100 price range 0 100000000
  => create;
```

A POST route már a handler előtt typed inputtá alakítja a két mezőt. Üres vagy 100 karakternél hosszabb név, negatív ár, túl nagy ár vagy nem egész szám esetén a kérés nem jut el érvényes üzleti műveletként az INSERT-ig.

3.7. A teljes alkalmazás egy helyen

Az előző részek együtt alkotják a teljes alkalmazást: `model` + két `query fn` + `page fn` + `action fn` + két `route`. Nem ismételjük meg még egyszer ugyanazt a hosszú listinget; a repositoryban a pontos, egyfájlos változat közvetlenül megnyitható:

```
examples/a-z-products/main.rw
```

Ez szándékos könyvszerkesztési döntés is: a következő fejezetekben az egyes nyelvi elemeket már részletesen tárgyaljuk, itt viszont az alkalmazás teljes útvonalán akarunk végigmenni.

3.8. SQLite kapcsolat developmenthez

A database URL-t ne írjuk a forrásba. Készítsünk külön fájlt, abszolút SQLite pathszal:

```
printf '%s\n' 'sqlite:///tmp/rwlang-a-z-products.db' > dev-db-url
chmod 600 dev-db-url
```

A fájl itt nem valódi titkot tartalmaz, de ugyanazt a `*_file` mintát használjuk, mint productionben a jelszavas database URL esetén.

3.9. Forrásellenőrzés és migration

Ha a release binárisokat már telepítettük például `/usr/local/bin` alá:

```
/usr/local/bin/rwlang-cli check main.rw

/usr/local/bin/rwlang-cli migrate verify \
  --dir migrations \
  --db-url-file "$PWD/dev-db-url"

/usr/local/bin/rwlang-cli migrate apply \
  --dir migrations \
  --db-url-file "$PWD/dev-db-url"
```

A `check` a nyelvi és statikus szerződést ellenőrzi. A migration CLI külön folyamat; ettől még az alkalmazásszerver nem indult el.

3.10. Minimális development konfiguráció

A `server.toml` legyen például:

```
[server]
app = "/ABS/PATH/my-products/main.rw"
listen = "127.0.0.1:8080"
insecure_dev_cookies = true

[database]
url_file = "/ABS/PATH/my-products/dev-db-url"
```

A két `/ABS/PATH` helyére valódi abszolút útvonal kell. Az `insecure_dev_cookies` csak loop-back developmenthez elfogadható; productionben ne kapcsolod be.

Először ellenőrizzük a konfigurációt:

```
/usr/local/bin/rwlang-server --config "$PWD/server.toml" --check-config
```

Majd indulhat a szerver:

```
/usr/local/bin/rwlang-server --config "$PWD/server.toml"
```

3.11. Az első HTTP kérés

Másik terminálból:

```
curl -i http://127.0.0.1:8080/
```

Böngészőben nyisd meg ugyanazt az URL-t. A form beküldése után a folyamat:

1. a böngésző POST-ot küld a `/products` route-ra;
2. a runtime ellenőrzi a request schema-t, a typed mezőket, a validációt és a CSRF tokenet;
3. a `create` action transactiont nyit;
4. a `createProduct` bind paraméterekkel INSERT-et futtat;
5. siker esetén commit történik;
6. a handler redirectet ad a `/` URL-re;
7. az új GET már az adatbázisból visszaolvasott sort rendereli.

Ez az egyszerű lánc a későbbi wiki és CMS alkalmazásban is ugyanaz marad; csak több model, query, route, jogosultság és komponens kerül köré.

3.12. Mit történik hibás inputnál?

Próbáld negatív árat vagy nem numerikus értéket megadni. A fontos elv az, hogy a böngésző HTML attribútumai csak felhasználói kényelmet adnak; a valódi szerződést a szerveroldali route schema és validation adja. Az SQL query csak már typed értéket kap.

Érdekes még a következőket kipróbálni:

- küldj hiányzó `name` mezőt;
- küldj ismeretlen extra form mezőt;
- módosítsd a migrationt alkalmazás után, majd futtasd újra a `migrate verify` parancsot;

- állítsd le az adatbázis elérését, és figyeld meg a readiness illetve a szervert log viselkedését.

3.13. Developmentből production felé

A példa alkalmazáskódja ugyanaz marad release környezetben is; nem kell újraírni csak azért, mert a szervert már telepített binárisal futtatjuk. A

refch:telepites. fejezetben bemutatott build és telepítés után a szerver tipikusan így indul:

```
/usr/local/bin/rwlang-server \\  
--config /usr/local/etc/rwlang/server.toml
```

Az alkalmazás entrypointja lehet például `/srv/rwlang/current/main.rw`. Apache vagy Nginx esetén a publikus TLS kapcsolatot a proxy terminálja, az RWLang pedig loopbacken hallgat. A teljes Apache/Nginx mintát nem ismételjük meg itt: a telepítési fejezet mutatja be a trust boundaryt, a könyv végi függelék pedig copy/paste-közi production konfigurációt ad.

3.14. Mit tanultunk ebből az egy példából?

Egy mindössze néhány tucat soros alkalmazásban már láttuk az RWLang teljes alap útvonalát:

```
HTTP GET  
-> route  
-> page  
-> typed query  
-> database  
-> escaped HTML  
  
HTTP POST  
-> route schema + validation + CSRF  
-> action  
-> transaction  
-> typed SQL bind  
-> commit  
-> redirect  
-> GET
```

A következő fejezetek ezt a képet bontják szét. Amikor egy-egy nyelvi vagy runtime részlet technikainak tűnik, érdemes visszagondolni erre a két kérésre: *hol lép be az adat a rendszerbe, és milyen typed/capability határokon halad át, mielőtt tartós állapot vagy HTTP válasz lesz belőle?*

II. rész

Alap webalkalmazás-fejlesztés

4. fejezet

Nyelvi alapok és HTTP-határok

Az A–Z példában már láttuk a fő építőelemeket működés közben. Ebben a fejezetben ezeket napi webfejlesztési munkafolyamattá rendezzük: a forrásfájlokat explicit modulgráfba szervezzük, a statementeket egyértelműen lezárjuk, typed értékekkel számolunk, a szöveget a határokon normalizáljuk, a domainlogikát handlerekben tartjuk, majd typed route-on és biztonságos renderelésen keresztül lépünk át HTTP/HTML világba.

Hasznos munkaszabály: *a bemenetet a határon parse-old és normalizáld, a domainben typed értékekkel dolgozz, explicit query-ken keresztül perzisztálj, és csak a lehető legkésőbb renderelj.* A fejezet sorrendje ezt követi.

4.1. Forrásstruktúra és statement-határok

Modulok

A `main.rw` a modulgráf belépési pontja:

```
mod models;  
mod queries;  
mod components;  
mod pages;  
mod actions;
```

A V1-ben a `mod application-root` relatív modult tölt be, de a deklarációit nem önti a globális névtérbe. Például a `models.rw` deklarációi más modulból `models::Article` alakban érhetők el. A saját modulon belüli lokális deklaráció rövid névvel is hivatkozható. Nincs általános Rust-szerű `pub export`-rendszer.

Modul-névterek és modulok közötti hívások

A moduldeklaráció egy `application-root` relatív forrásmodult vesz fel az explicit source graphba; nem másolja annak deklarációit az aktuális scope-ba. A kanonikus leképezés közvetlen:

<code>models.rw</code>	<code>-> models</code>
<code>catalog/queries.rw</code>	<code>-> catalog::queries</code>
<code>catalog/pages.rw</code>	<code>-> catalog::pages</code>

Egy belépési pont például több modult tölthet be:

```
mod models;
mod catalog::queries;
mod catalog::pages;

route catalogIndex GET "/catalog"
  => catalog::pages::index;
```

Modulhatáron át kvalifikált nevet használj. Például:

```
let recent = catalog::queries::recent(db);
// típus modulhataron at: models::Article
```

A `catalog/queries.rw` tehát a `catalog::queries` névtérhez tartozik. A deklaráció saját modulján belül a rövid lokális név továbbra is használható. Így a helyi kód rövid marad, a forrásmodulok közötti dependency viszont látható.

A modulútvonal mindig az application rootból indul. Nincs `mod.rw` fallback, automatikus könyvtár-discovery, `../`, `./`, `self::`, `super::`, `crate::` vagy wildcard import. Minden forrásmodult explicit deklarálj, még a normál top-level deklarációk előtt.

A nyelvi builtinok – például `trim(...)`, `sin(...)` és `len(...)` – nem alkalmazásmodul-deklarációk, ezért továbbra is kvalifikáció nélkül használhatók.

Statement terminátorok

Az RWLangban az egyszerű statementeket explicit pontosvessző zárja. A sortörés whitespace, nem statement-határ; nincs automatikus semicolon insertion. Ez determinisztikussá teszi a többsoros kifejezéseket és egyértelműbbé a generált, illetve auditált kódot.

```
let subtotal = price
  * quantity
  + shipping;
set retries = retries + 1;
authorize article owner authorUsername or role Publisher;
return Ok(json(subtotal));
```

A blokkos statementeket és blokkos deklarációkat a kapcsos zárójel zárja, utánuk nincs pontosvessző:

```
if ready {
  set state = 1;
}
while state < 3 {
  set state = state + 1;
}
```

Ugyanez érvényes tranzakción belül is: az önálló mutating query-hívás és a business-audit statement egyszerű statement, ezért `;` zárja. A top-level blokkos deklarációk – például `model`, `page fn`, `action fn` – és a control-flow blokkok után nincs `;`. A blokk nélküli top-level deklarációk viszont pontosvesszősök: a `mod path;` és a `route ... => handler;` is explicit ; terminátort kap. Többsoros route esetén is csak ez a záró terminátor fejezi be a deklarációt.

4.2. Typed számítás webalkalmazás-kódban

Kifejezések és aritmetika

Az RWLang explicit operátorprecedenciát használ. Szorosabbtól lazább felé: unáris `!`; szorzás, osztás és maradékképzés; összeadás és kivonás; shiftok; összehasonlítások; bitenkénti AND/XOR/OR; logikai AND; logikai OR. Ha javítja az olvashatóságot, használj zárójelet.

```
let total = 12 + 3 * 4;
let remainder = 17 % 5;
let shifted = 3 << 2;
let masked = 12 & 10;
let allowed = ready && authorized;
let fallback = cached || fresh;
let denied = !allowed;
```

Az operátorszerződés szigorúan typed:

Operandusok	Operátorok	Eredmény
Int, azonos típus	<code>+ - * / \%</code>	checked Int.
F32, azonos típus	<code>+ - * / \%</code>	F32; véges eredmény szükséges.
Decimal, azonos típus	<code>+ - * / \%</code>	checked Decimal.
Int, azonos típus	<code>< > \% \^{} </code>	shift/bitművelet, Int.
Bool, azonos típus	<code>\&\& </code>	Bool; short-circuit kiértékelés.
Bool	<code>!</code>	logikai negáció.
String, azonos típus	<code>+</code>	budgetelt String-konkatenáció.
Numerikus, azonos típus	<code>< <= > >=</code>	Bool.
Támogatott azonos skalártípus	<code>== !=</code>	Bool.

Nincs implicit keverés Int, F32 és Decimal között. Lebegőpontos számításhoz az integert explicit `toF32(value)` hívással alakítsd át. Overflow, nullával osztás/maradékképzés, hibás shift vagy nem véges F32 eredmény fail-closed hibát ad.

A numerikus builtinok:

Függvény	Típus	Feladat
<code>sin(x), cos(x)</code>	F32 -> F32	Trigonometrikus függvények.
<code>sqrt(x)</code>	F32 -> F32	Négyzetgyök; hibás tartomány elutasítva.
<code>abs(x)</code>	Int -> Int vagy F32 -> F32	Abszolútérték.
<code>ln(x), log10(x)</code>	F32 -> F32	Természetes/tízes alapú logaritmus.
<code>log(x, base)</code>	F32, F32 -> F32	Explicit alapú logaritmus.
<code>exp(x)</code>	F32 -> F32	Exponenciális függvény.
<code>pow(x, y)</code>	F32, F32 -> F32	Lebegőpontos hatványozás.
<code>round/floor/ceil(x)</code>	F32 -> F32	Kerekítési műveletek.
<code>toF32(x)</code>	Int -> F32	Explicit integer-F32 konverzió.
<code>monotonicNanos()</code>	() -> Int	Process-lokális monoton nanoszekundum-számláló.

Minden F32 eredménynek végesnek kell maradnia. A `sqrt(-1.0f32)` és `ln(-1.0f32)` jellegű domainhibák NaN/végtelen helyett hibát adnak. A `monotonicNanos()` eltelt idő mérésére való, nem falióra-időbélyeg; a tőle függő public-cache kimenetet a compiler elutasítja.

Gyakori webes aritmetikai minták

Webalkalmazásban ritkán kell mindenhol haladó matematika, néhány numerikus minta viszont folyamatosan visszatér. Ezeket typed értékekkel és látható kóddal kezeld, ne string-konverziókba vagy SQL-részletekbe rejtisd.

Lapozásnál validált integerekből számolj offsetet:

```
let pageSize = 25;
let offset = (page - 1) * pageSize;
```

Integer flag/mask esetén használhatók a bitműveletek, de üzleti feltételekhez az olvasható boolean logika az alap. A `&&` és `||` short-circuit módon működik:

```
let mayPublish = authenticated
    && ownsArticle
    && !locked;
```

Pénzhez `Decimal`-t használj. `F32`-t akkor válassz, ha maga a feladat lebegőpontos: például geometria, jelfeldolgozás, statisztika vagy explicit matematikai függvény. Pénzösszeget ne alakíts `F32`-re csak azért, hogy `math` builtint lehessen rá alkalmazni.

String- és szövegfüggvények

A string műveletek typed, Unicode-tudatos builtinok a korlátozott runtime-on belül:

```
let cleaned = trim(text);
let left = trimStart(text);
let right = trimEnd(text);
let normalized = lower(cleaned);
let chars = stringLen(cleaned);
let rewritten = replace(normalized, " ", "-");
let part = substring(cleaned, 1, 3);
let first = indexOf(cleaned, "rw");
let last = lastIndexOf(cleaned, "a");
let ch = charAt("RWLang", 1);
let repeated = repeat("rw", 3);
let parts = split(rewritten, "-");
```

Függvény	Szignatúra
<code>stringLen(text)</code>	<code>String -> Int</code>
<code>trim/trimStart/trimEnd(text)</code>	<code>String -> String</code>
<code>lower/upper(text)</code>	<code>String -> String</code>
<code>contains(text, needle)</code>	<code>String, String -> Bool</code>
<code>startsWith/endsWith(text, part)</code>	<code>String, String -> Bool</code>
<code>replace(text, search, replacement)</code>	<code>String, String, String -> String</code>
<code>split(text, delimiter)</code>	<code>String, String -> List<String></code>
<code>substring(text, start[, length])</code>	<code>String, Int[, Int] -> String</code>
<code>indexOf/lastIndexOf(text, needle)</code>	<code>String, String -> Int</code>
<code>charAt(text, index)</code>	<code>String, Int -> String</code>
<code>repeat(text, count)</code>	<code>String, Int -> String</code>

A `stringLen`, `substring`, `indexOf`, `lastIndexOf` és `charAt` Unicode scalar value pozíciókat

használ, nem UTF-8 byte indexet. Sikertelen keresésnél `-1` az eredmény; negatív vagy érvénytelen index `fail-closed` hibát ad. A `replace` nem enged üres keresőszöveget, a `split` nem enged üres delimitert és legfeljebb 4096 elemet ad, a `repeat` pedig az eredmény létrehozása előtt bekerül az allocation budgetbe. A `List<String>` typed compute collection marad `len(parts)` és ellenőrzött `parts[index]` használattal.

Korlátozott reguláris kifejezéses feldolgozáshoz elérhető a `regexMatch`, `regexReplace` és `regexCaptures`; ezek továbbra is a validációs és resource limitek alatt futnak.

Gyakori szövegkezelési minták

A szövegmanipulációt kezeld boundary-feladatként. Keresési és filter inputot érdemes összehasonlítás előtt normalizálni, de az eredeti értéket tartsd meg, ha a domainben számít a pontos írásmód.

```
let queryText = trim(q);
let normalizedQuery = lower(queryText);
let hasRwlang = contains(normalizedQuery, "rwlang");
```

A `replace` determinisztikus helyettesítésre való, nem általános parser helyett:

```
let displayTag = replace(trim(tag), "_", " ");
```

Szeletelésnél és pozíciókeresésnél az RWLang Unicode scalar-value pozíciókat használ UTF-8 byte offset helyett. Így a `substring`, `charAt` és indexfüggvények alkalmazásszövegnél nem teszik ki a fejlesztőt encoding-részleteknek. Strukturált azonosítóknál – URL, slug, email, dátum, UUID – viszont a dedikált RWLang típusokat és validátorokat használd, ne string függvényekből építsd újra a szabályokat.

Melyik eszközt válaszd alkalmazáskódban?

Webfejlesztési feladat	Preferált RWLang megoldás
Keresési/filter szöveg normalizálása	<code>trim</code> , majd <code>lower/upper</code> csak akkor, ha az összehasonlítás case-insensitive.
Ismert token helyettesítése	<code>replace</code> ; regexet csak valódi mintakereséshez használj.
Email, URL, slug, dátum, UUID	Dedikált typed érték és validáció, ne ad-hoc string parsing.
Pénz/összegzés	<code>Decimal</code> .
Lapozás/számlálók	<code>Int</code> checked aritmetikával.
Üzleti feltételek	<code>Bool</code> short-circuit <code>&&</code> , <code> </code> , <code>!</code> operátorokkal.
Tudományos/statisztikai matematika	<code>F32</code> és explicit math builtin; előbb dönts el, hogy a lebegőpont valóban illik-e a domainhez.
Eltelt idő mérése	<code>monotonicNanos()</code> , soha ne falióra-időbélyegként.

4.3. Domainértékek és handlerkód

Modellek és üzleti típusok

```
model Article {
  id: Int
  slug: Slug
```

```

    title: String
    body: String
}

```

A gyakori típusok: `String`, `Int`, `Bool`, továbbá `Date`, `DateTime`, `Uuid`, `Decimal` és `Slug`. Pénzhez `Decimal`-t használj, ne lebegőpontos számot.

Page és action

A page GET-szerű megjelenítési logika:

```

page fn show(ctx: PageContext, db: Db, id: Int)
  -> Result<Html, PageError> {
  let article = articleById(db, id)?;
  return Ok(html { <h1>{{ article.title }}</h1> });
}

```

Az action állapotváltoztató kérés kezelője:

```

action fn create(ctx: ActionContext, db: Db, title: String)
  -> Result<Redirect, PageError> {
  transaction db {
    insertArticle(tx, title)?;
  }
  return Ok(redirect("/admin/articles"));
}

```

Változók és hibaterjedés

A `let` typed lokális értéket hoz létre. Általános `mut` nincs a V1-ben; `compute` kódban explicit `set name = expr` újraértékdadás használható az eredeti statikus típus megtartásával és instruction budget alatt. A `?` a typed `Result` hibáját továbbítja.

Enum

Zárt választási halmazhoz ne szabad szöveget használj, hanem enumot. Ez form-, route-, JSON- és domain-contractként is erősebb.

Object

Összetartozó model/query/page műveletek domain object alatt csoportosíthatók. Ez különösen nagyobb alkalmazásban segít a domain API körülhatárolásában.

4.4. HTTP- és prezentációs határ

Route

```

route article GET "/cikk/:slug<Slug>" => article;

```

A path paraméter típusa a route-ban deklarált. Hibás input nem jut el a handlerig typed értéként.

HTML escape

```
return Ok(html {
    <h1>{{ article.title }}</h1>
});
```

A `{ ... }` HTML-escaped. Adatbázisból érkező string sem kivétel. Raw HTML API nincs.

Lista és optional érték

A listát iterálni kell, az optional modelt pedig explicit jelenlétvizsgálat után lehet mezőzni. A compiler nem engedi, hogy `List<Model>` vagy ellenőrizetlen `Model?` úgy viselkedjen, mintha egyetlen rekord lenne.

```
@for article in articles {
    <li>{{ article.title }}</li>
}

@if article {
    <h1>{{ article.title }}</h1>
}
```

A query-cardinality pontos futásidejű szerződését – például hogy a `Model?` több sor esetén hibát jelent – az adatbázis-fejezet tárgyalja.

Typed URL

```
<a @href(article, article.slug)>Megnyitas</a>
<form method="post" @action(deleteArticle, article.id)>
```

Preferáld a route helpereket

Saját alkalmazás-route-ok URL-jét ne másold kézzel több helyre. HTML-ben POST formhoz az `@action(...)` és linkhez az `@href(...)` a preferált minta, mert a route neve és typed paraméterei maradnak a kapcsolat forrásai. A beépített runtime endpointok – például a `/__rw/auth/logout` – természetesen nem alkalmazás-route helperek.

Ne építs dinamikus `href="{{ value }}"` linket. A route helper ismeri a célt és a paraméter-típust.

Component és layout

```
layout fn Main(title: String) -> Html {
    html {
        <html>
            <head><title>{{ title }}</title></head>
            <body><main>@content</main></body>
        </html>
    }
}
```

Page-ben:

```
return Ok(html {  
    @layout(Main, article.title) {  
        <article><h1>{{ article.title }}</h1></article>  
    }  
});
```

A layout pontosan egy `@content` slotot tartalmazhat; component/layout csak explicit typed paramétereket lát.

5. fejezet

Adatbevitel: typed SQL, CRUD, formok és PRG

Egy klasszikus webalkalmazás alapciklusa: adatot olvasunk az adatbázisból, HTML formon typed inputot fogadunk, validálunk, tranzakcióban módosítunk, majd redirect után újra GET-tel olvasunk. Ebben a fejezetben ezt a teljes ciklust egy helyen kezeljük.

5.1. Typed SQL és CRUD

Lekérdezés

```
query fn articleBySlug(db: Db, slug: Slug)
  -> Result<Article, DbError> sql {
    SELECT id, slug, title, body
    FROM articles
    WHERE slug = :slug
  }
```

Az SQL szöveg statikus és compiler-owned. Felhasználói érték csak typed bindként, például `:slug`, kerülhet bele.

Mutáció transactionben

```
query fn updateArticle(
  tx: Transaction,
  id: Int,
  title: String,
  body: String
) -> Result<Void, DbError> sql {
  UPDATE articles
  SET title = :title, body = :body
  WHERE id = :id
}
```

Az action:

```
action fn update(
  ctx: ActionContext,
  db: Db,
```

```

    id: Int,
    title: String,
    body: String
  ) -> Result<Redirect, PageError> {
    transaction db {
      updateArticle(tx, id, title, body)?;
    }
    return Ok(redirect("/admin/articles"));
  }

```

Siker esetén commit, hiba esetén rollback. A `Transaction` capability nem escape-elhet a blokkból.

Portable adatbázis-séma

A V1 SQLite, PostgreSQL és MariaDB irányban is hordozható mintát céloz. `Date`, `DateTime`, `Uuid` és `Decimal` portable reprezentációja canonical text lehet.

Migration

A szerver nem migrál automatikusan startupkor. Ajánlott folyamat:

```

migrate verify
migrate apply
migrate verify

```

A migration credential legyen elkülönítve a normál application runtime credentialtól.

5.2. Formok, validáció és PRG

Named form schema

```

form ArticleForm {
  title<String>
  body<String>
  published<Bool>
  validate title length 2 200 body length 1 200000
}

```

Route:

```

route articleCreate POST "/admin/articles"
  form ArticleForm
  auth role Editor
  => create;

```

A handler paramétersorrendje és típusa kövesse a form mezőit.

Hibás beküldés

Named formnál a hiányzó kötelező mező, a mezőszintű typed decode hiba vagy a deklarált validation hibája 422 *Unprocessable Content*. A runtime a mezőértékeket escape-elve tölti

vissza, mezőhibát ad és CSRF tokenet illeszt a formba. Ismeretlen vagy duplikált mező viszont 400, mert az request-schema megsértése.

Ez szándékosan más, mint az inline form- vagy JSON-schema: ott a typed decode/validation hibája request-contract sértésként 400. Named formot akkor használj, amikor a hibát ugyanazon felhasználói úrlapon, mezőszintű visszajelzéssel akarod kezelni.

Sikeres POST és PRG

Sikeres módosítás után redirectet adj vissza; így a böngésző GET-tel tölti be a céloldalt. Ez a Post/Redirect/Get minta megszünteti a véletlen újraküldés tipikus problémáját.

Egyszerű teljes form-action minta

```
action fn create(  
  ctx: ActionContext,  
  db: Db,  
  title: String,  
  body: String,  
  published: Bool  
) -> Result<Redirect, PageError> {  
  let articleSlug = slug(title);  
  transaction db {  
    insertArticle(tx, articleSlug, title, body)?;  
  }  
  return Ok(redirect("/admin/articles"));  
}
```

A checkbox-szerű hiányzó Bool mező named form esetén `false`.

6. fejezet

Adatbázis mélyebben: tranzakció, migráció és konkurencia

Az előző fejezetben már láttuk a typed SQL és a CRUD alapmintáját. Egy valódi wiki, CMS vagy ügyviteli alkalmazás azonban hamar eljut oda, ahol már nem az a kérdés, hogy *hogyan futtatunk egy SELECT-et*, hanem az, hogy *hogyan* őrizzük meg az adat integritását több kérés, több szerverpéldány és több release között.

Ez a fejezet erre ad fejlesztői modellt. A cél: az adatbázist ne rejtett globális szolgáltatásként, hanem explicit capabilityként, a sémát pedig verziózott release-komponensként kezelni.

6.1. Három külön felelősség

RWLang alkalmazásnál érdemes három réteget szétválasztani:

alkalmazáskód statikus typed SQL queryket deklarál, bindokat ad át és tranzakciót kér;
runtime kapcsolatot pooloz, bindol, timeoutot és eredménykorlátot alkalmaz, majd a visszakapott sorokat a deklarált típushoz ellenőrzi;
operator DB credentialt, TLS-t, adatbázis-kapacitást, backupot és migrációs folyamatot birtokol.

Az alkalmazás tehát nem nyit tetszőleges socketet az adatbázishoz és nem épít saját connection poolt. A **Db** és a **Transaction** capability az a felület, amelyen keresztül adatbázis-művelet történhet.

A V1 data layer secure defaultjai jelenleg többek között 32 maximális poolkapcsolatot, 5 másodperces connection-acquire timeoutot, 10 másodperces query timeoutot, 10 000 soros és 16 MiB-os eredménykorlátot használnak. Ezeket ne alkalmazáslogikaként kezelni: a helyes query ettől függetlenül legyen rövid, paginált és indexelhető.

Több szerverpéldánynál a DB connection budgetet az operatornak összesítve kell méreteznie:

```
össz DB kapcsolat ~= RWLang instance-ok szama
                    * instance-onkenti pool
                    + admin/maintenance tartalek
```

6.2. A query visszatérési típusa szerződés

Egy lekérdezés cardinalityje már a függvény típusában látszik:

```
query fn articleById(db: Db, id: Int)
  -> Result<Article, DbError> sql {
    SELECT id, slug, title, body, version
    FROM articles
    WHERE id = :id
  }

query fn articleBySlug(db: Db, slug: Slug)
  -> Result<Article?, DbError> sql {
    SELECT id, slug, title, body, version
    FROM articles
    WHERE slug = :slug
  }

query fn recentArticles(db: Db, limit: Int, offset: Int)
  -> Result<List<Article>, DbError> sql {
    SELECT id, slug, title, body, version
    FROM articles
    ORDER BY id DESC
    LIMIT :limit OFFSET :offset
  }
```

A három forma jelentése:

Típus	Elvárt eredmény	Cardinality sérülése
<code>Article</code>	pontosan egy sor	0 sor: 404; több sor: DB-hiba
<code>Article?</code>	nulla vagy egy sor	több sor: DB-hiba
<code>List<Article></code>	nulla vagy több sor	a runtime eredménylimitei érvényesek

Ez több egyszerű kényelmi jelölésnél: a lekérdezés eredményének alakja része a program futás közben is kikényszerített szerződésének. Az `Article?` nem azt jelenti, hogy „vedd az első sort, ha van”: ha a query kettő vagy több sort ad, az invariáns sérült, ezért a runtime fail-closed DB-hibát ad. Ugyanezért egy `Article` query nulla sor esetén 404 `Not Found`, több sor esetén pedig nem választ önkényesen rekordot.

A row shape legyen explicit

A `SELECT` mezőinek neve és sorrendje egyezzen a modellel. Ne használj `SELECT *`-ot alkalmazásqueryben.

```
model Article {
  id: Int
  slug: Slug
  title: String
  body: String
  version: Int
}

// jo: ugyanaz az ot mezo, ugyanebben a sorrendben
SELECT id, slug, title, body, version
```

```
FROM articles
```

Az explicit projection két okból fontos. Először is a fordító ellenőrizni tudja a modell és a query kapcsolatát. Másodszor egy későbbi schema-bővítés nem változtatja meg észrevétlenül egy régi query result shape-jét.

6.3. Bindolj, ne SQL-t építs

A kliensből származó érték typed bindként kerüljön az SQL-be:

```
query fn findByEmail(db: Db, email: String)
  -> Result<User?, DbError> sql {
    SELECT id, email, display_name
    FROM users
    WHERE email = :email
  }
```

A `:email` nem string-helyettesítés. A runtime backend-specifikus placeholderre fordítja és paraméterként bindolja.

Amit ne próbálj meg:

```
// rossz gondolkodasmod:
// "SELECT ... ORDER BY " + userOrder
// "WHERE " + userFilter
```

Oszlopnév, ORDER BY irány, SQL operator vagy teljes WHERE-fragment ne érkezzen user inputból. Ha több rendezést akarsz támogatni, az alkalmazáskódban válassz néhány statikus query közül.

6.4. Olvasás és mutáció authority-határa

Olvasó query Db capabilityt kap. Adatmódosító query Transaction-t:

```
query fn renameArticle(
  tx: Transaction,
  id: Int,
  title: String
) -> Result<Void, DbError> sql {
  UPDATE articles
  SET title = :title
  WHERE id = :id
}
```

A hívó action explicit tranzakciós blokkot nyit:

```
action fn rename(
  ctx: ActionContext,
  db: Db,
  id: Int,
  title: String
) -> Result<Redirect, PageError> {
  transaction db {
```

```

        renameArticle(tx, id, title)?;
    }
    return Ok(redirect("/admin/articles"));
}

```

A tranzakció sikeres blokk után commitol; hiba esetén rollbackel. A `Transaction` nem escape-elhet a blokkból, ezért nem lehet véletlenül egy lezárt tranzakció handle-jét később felhasználni.

Egy üzleti művelet legyen egy tranzakció

Több, összetartozó módosítást ne külön actionök vagy külön tranzakciók végezzenek:

```

transaction db {
    Article.publishChanged(tx, id, version)?;
    audit Article id action publish;
        from article.status to ArticleStatus.Published
}

```

Ebben a mintában a cikk publikálása és a business audit trail együtt commitol. Ha az audit nem írható, a publikálás is rollbackel. Így nem jön létre olyan állapot, amelyben az üzleti módosítás megtörtént, de a hozzá kötelező audit rekord hiányzik.

6.5. Optimistic locking: ne írd felül más munkáját

CMS-ben és wikiben gyakori helyzet, hogy két szerkesztő ugyanazt a rekordot nyitja meg. A klasszikus

```
UPDATE articles SET title = :title WHERE id = :id
```

az utoljára mentő felhasználót teszi nyertessé, és csendben elveszhet az első szerkesztő módosítása.

Ehhez használható verziómező és `Changed`:

```

model Article {
    id: Int
    title: String
    version: Int
}

query fn updateArticle(
    tx: Transaction,
    id: Int,
    title: String,
    version: Int
) -> Result<Changed, DbError> sql {
    UPDATE articles
    SET title = :title,
        version = version + 1
    WHERE id = :id AND version = :version
}

```

Az edit form elküldi azt a **version** értéket, amelyet a felhasználó az oldal megnyitásakor látott. A **Changed** szerződése szigorú: a mutációnak **pontosan egy sort** kell módosítania. Egy módosított sor siker; nulla módosított sor **409 Conflict**; kettő vagy több módosított sor DB-hiba, mert a query a saját invariánsát sértette meg. **Changed** csak mutáló query visszatérési típusa lehet, és V1-ben nem használható **RETURNING** klauzussal.

Optimistic lockingnál, ha időközben más már mentett, a **WHERE id = :id AND version = :version** nulla sort talál, ezért a kérés konfliktusra fut. A helyes UX ilyenkor nem az automatikus újrapróbálás. Mutasd meg, hogy az adat közben változott, töltsd be az új verziót, és a felhasználó döntsön az összeolvasztásról.

Optimistic locking tipikus helyei:

- wiki oldalak és CMS cikkek;
- ticket vagy ügyfélrekord;
- workflow-állapotot tartalmazó üzleti objektum;
- ritkán módosított admin konfiguráció.

Append-only eseményekhez vagy adatbázis-oldali atomi számlálókhoz általában nincs rá szükség.

6.6. Indexet a hozzáférési mintához tervezz

A jó index nem attól jó, hogy sok van belőle, hanem attól, hogy a tényleges queryt szolgálja ki.

Egy CMS-ben például természetes hozzáférési minták lehetnek:

```
CREATE UNIQUE INDEX articles_slug_uq
  ON articles(slug);

CREATE INDEX articles_status_id_idx
  ON articles(status, id);
```

Ha a publikus oldal slug alapján kér le egy cikket, a slug legyen unique/indexelt. Ha az admin lista státusz szerint szűr és azon belül azonosító szerint lapoz, ehhez illeszkedő összetett index kellhet.

Néhány szabály:

- minden nagy listát paginálj;
- a rendszeresen használt lookup mező legyen indexelhető;
- az indexek sorrendje kövesse a **WHERE/ORDER BY** mintát;
- ne indexelj automatikusan minden oszlopot: az index írási és storage költség;
- teljesítményproblémát valós backend **EXPLAIN** tervével vizsgálj.

Az RWLang typed SQL-ja a query alakját védi; az adatbázis optimalizálóját nem helyettesíti.

6.7. Migration: a séma a release része

Productionban a server nem futtat automatikus migrációt startupkor. Ez szándékos: egy schema-változás külön operátori művelet, külön credentialdel és külön hibahatárral.

Tipikus könyvtár:

```
migrations/
```

```
0001_create_articles.sql
0002_articles_slug_index.sql
0003_articles_add_version.sql
```

A fájlnev alakja:

```
NNNN_name.sql
```

Már alkalmazott migration fájlt ne módosíts. A CLI SHA-256 checksumot tárol, és eltérésnél megáll.

A production parancsok a telepített CLI-val:

```
/usr/local/bin/rwlang-cli migrate status \
--dir /srv/myapp/current/migrations \
--db-url-file /run/secrets/migration-db-url

/usr/local/bin/rwlang-cli migrate verify \
--dir /srv/myapp/current/migrations \
--db-url-file /run/secrets/migration-db-url

/usr/local/bin/rwlang-cli migrate apply \
--dir /srv/myapp/current/migrations \
--db-url-file /run/secrets/migration-db-url \
--lock-timeout-secs 30
```

A migration credential magasabb jogosultságú lehet, mint az alkalmazás DML credentialje. Emiatt a server process ne kapja meg.

Mit garantál a migration CLI?

- ellenőrzi az alkalmazott fájl nevét és checksumát;
- nem enged észrevétlenül régi verziószám alá új migrationt beszúrni;
- adatbázis-szintű migration lockot kér;
- a state csak sikeres alkalmazás után halad tovább.

Backend-specifikus különbség viszont marad. PostgreSQL migrationonként tranzakciót használ; SQLite **BEGIN IMMEDIATE** writer lockkal a batch-et tranzakcióban tartja; MariaDB DDL nem minden esetben tranzakcionális, ezért részleges DDL-hibánál operator repair is szükséges lehet.

6.8. Expand/contract: tartsd meg a rollback lehetőségét

Egy gyors application rollback csak akkor egyszerű, ha az új schema a régi programmal is kompatibilis.

Ezért nagyobb változtatásnál használj expand/contract mintát:

1. **expand**: adj hozzá új nullable/defaultolt mezőt vagy új táblát úgy, hogy a régi app még működjön;
2. deployold az új alkalmazást;
3. szükség esetén végezz kontrollált backfillt;
4. csak későbbi release-ben **contract**: töröld a régi mezőt vagy constraintet, amikor a rollback window lezárult.

Az azonnali rename/drop, új kötelező mező vagy inkompatibilis adattípus-váltás könnyen forward-only release-t eredményez. Ezt release review során explicit döntésként kezeld.

6.9. Adatbázis-hiba kezelése

A `DbError` technikai hiba. A végfelhasználónak ne jeleníts meg driver message-et, SQL szöveget, hostnevet vagy connection stringet.

Alkalmazási szinten különítsd el legalább ezeket a helyzeteket:

nincs ilyen rekord normál domain-állapot, opcionális queryvel kezelhető;

409 konkurens módosítás optimistic-locking konfliktus, a felhasználó újratöltéssel/összevetéssel oldja fel;

adatbázis nem elérhető infrastruktúrahiba; readiness is jelezheti, a logban request ID-val kereshető;

constraint/domain hiba inkább validációval előzd meg, de az adatbázis constraint maradjon utolsó integritási védővonal.

Ne építs automatikus, korlátlan retry loopot egy teljes mutáló action köré. Egy megismételt kérésnek mellékhatása lehet, és a tranzakciós konfliktus sem feltétlen átmeneti hiba.

6.10. SQLite, PostgreSQL vagy MariaDB?

RWLang V1 mindhárom backendet támogatja, de ez nem jelenti azt, hogy minden SQL dialektusfeature hordozható.

SQLite kiváló egyszerű vagy egygépes telepítéshez; az application DB fájl backupját SQLite-aware módszerrel készítsd, ne élő fájl egyszerű másolásával.

PostgreSQL erős alap többpéldányos és nagyobb production rendszerekhez; TLS-t használj remote kapcsolathoz.

MariaDB szintén támogatott, de különösen migrációknál számolj a DDL tranzakciós különbségeivel.

Portable alkalmazásnál a queryk közös SQL-részhalmozra épüljenek. Backend-specifikus funkciót csak tudatosan használj, és akkor a projekt dokumentációjában rögzítsd, hogy az alkalmazás mely backendhez kötött.

6.11. Backup szempont adatbázis-fejlesztőként

Adatbázis-fejlesztőként itt egy dolgot kell rögzíteni: a migráció és a backup két külön életciklus. A migration leírja, hogyan változik a séma; a backup/restore azt bizonyítja, hogy a tartós állapot visszaállítható. PostgreSQLnél, MariaDB-nél és SQLite-nál a backend saját konzisztens mentési eszközét használd, és ne tekints egy egyszerű fájlmásolást automatikusan érvényes backupnak.

A teljes alkalmazásállapot – adatbázis, AppFs, local-auth store, recovery manifest, RPO/RTO és restore drill – a

refch:recovery. fejezet témája. Itt azért utalunk rá, mert egy schema-változtatás tervezésekor már tudni kell, hogy rollbackhez vagy restore-hoz milyen állapot áll rendelkezésre.

6.12. Egy CMS adatbázis-rétegének mintája

Egy kisebb CMS-nél jó kiindulás:

```
Article
  id
  slug          UNIQUE
  title
  body
  status
  hero_image
  version
  created_at
  updated_at

migrations/
  0001_create_articles.sql
  0002_article_indexes.sql
  0003_add_version.sql
```

Az alkalmazási minták:

- publikus olvasás: `Article?` slug alapján;
- admin lista: paginált `List<Article>`;
- szerkesztés: `Changed + version`;
- publish/archive: egy transactionben üzleti módosítás + audit;
- schema-változás: külön CLI migration;
- deploy előtt: migration verify + backup/restore readiness;
- deploy után: readiness, log, metrics és smoke test.

Ezzel a modellel a database layer nem különálló SQL-gyűjtemény lesz, hanem a webalkalmazás típus-, konkurencia-, release- és recovery-szerződésének része.

6.13. Fejezet végi ellenőrzőlista

Mielőtt egy adatbázisos funkciót késznek tekintesz, ellenőrizd:

- minden user érték typed bindként kerül SQL-be;
- nincs `SELECT *`; a projection egyezik a modellel;
- nagy lista paginált;
- a fontos lookup és rendezési minta mögött van megfelelő index;
- minden mutáció `Transaction` capabilityvel fut;
- több összetartozó módosítás egy tranzakcióban van;
- ember által párhuzamosan szerkeszthető rekordnál van optimistic locking;
- már alkalmazott migration immutable;
- a migration credential nincs az application servernél;
- destructive schema-változásnál ismert a rollback következménye;
- a DB hiba nem szivárogtat SQL-t vagy credentialt a kliensnek;
- a backuphoz tartozik rendszeresen bizonyított restore.

7. fejezet

Autentikáció, session és jogosultságkezelés

Az autentikáció (authentication) annak bizonyítása, hogy ki a felhasználó; az authorization/-jogosultságkezelés pedig annak eldöntése, hogy mit tehet. A session a sikeres bejelentkezés után ezt az identityt viszi tovább a kérések között. Az autentikáció ezért nem egyetlen login form. Egy production webalkalmazásban identity backend, session-életciklus, második faktor, jogosultság, CSRF, rate limit, audit és proxy trust együtt alkot egy rendszert. Az RWLang ezeket a felelőségeket szétválasztja: a `.rw` alkalmazás route-policyt deklarál, az identity és session policy pedig a szerver konfigurációjából jön.

7.1. A teljes login útvonal

A beépített HTML login endpoint:

```
GET  /__rw/auth/login
POST /__rw/auth/login
POST /__rw/auth/logout
```

Ezek szerver-endpointok, nem az alkalmazás által definiált route-ok. A GET login oldal egy sessionhöz kötött CSRF token-t ad. A POST login form pontosan a `_csrf`, `username`, `password` és `totp` mezőket várja.

A sikeres login magas szinten:

1. ellenőrzi a form szerkezetét és a CSRF token-t;
2. kanonizálja a felhasználónevet;
3. alkalmazza a login rate limitet;
4. ellenőrzi a local-auth vagy LDAP credentialt;
5. ha szükséges, ellenőrzi a TOTP-t vagy local auth esetén a recovery kódot;
6. új, autentikált sessionre rotál;
7. audit eseményt ír;
8. `303 See Other` válasszal a gyökérre irányít.

A session rotation fontos: sikeres autentikáció után nem marad használatban az anonim session azonosítója. Ez session-fixation elleni alapvédelmi elem.

7.2. Local auth: külön identity adatbázis

Single-node alkalmazásnál használható a local auth SQLite store. Ez szándékosan nem az application DB. Az RWLang program typed SQL queryjei így nem férnek hozzá a credential táblákhoz.

Secret file például:

```
/run/secrets/rwlang/local-auth-db-url
```

Tartalma:

```
sqlite:///srv/rwlang/auth/users.db?mode=rwc
```

Inicializálás:

```
/usr/local/bin/rwlang-cli auth init \  
--db-url-file /run/secrets/rwlang/local-auth-db-url
```

User létrehozásnál a jelszó nem CLI argumentumként szerepel, hanem fájlból olvasódik:

```
printf '%s\n' 'a-very-long-random-password' \  
> /run/secrets/rwlang/new-user-password  
chmod 600 /run/secrets/rwlang/new-user-password  
  
/usr/local/bin/rwlang-cli auth user-add \  
--db-url-file /run/secrets/rwlang/local-auth-db-url \  
--username alice \  
--password-file /run/secrets/rwlang/new-user-password \  
--role User \  
--role Editor
```

A local store Argon2id jelszóhash-t használ, egyedi random salttal. A jelenlegi implementáció legalább 12 és legfeljebb 1024 karakteres jelszót fogad el. Ezt ne keverd össze alkalmazásoldali form-validációval: a credential-policy az identity backend felelőssége.

7.3. Account lifecycle operátori CLI-val

A felhasználó állapotát nem kell közvetlen SQL-lel módosítani. A CLI erre explicit műveleteket ad:

```
rwlang-cli auth password-set --db-url-file ... \  
--username alice --password-file ...  
  
rwlang-cli auth disable --db-url-file ... --username alice  
rwlang-cli auth enable --db-url-file ... --username alice  
rwlang-cli auth roles-set --db-url-file ... --username alice \  
--role User --role Publisher
```

A disable nem törli a usert. Kívülről a sikertelen belépés nem árulja el, hogy rossz jelszó, nem létező user vagy tiltott account áll-e a háttérben: az ilyen különbségek account-enumeration csatornát nyithatnának.

7.4. Session invalidation: auth_generation

Local auth esetén a credential- és authorization-változások növelik a user `auth_generation` értékét. A session eltárolja azt a generációt, amely mellett létrejött.

A következő requestnél a szerver összeveti a session generációját a local auth store aktuális értékével. Ha eltér, a régi session érvénytelenné válik és új anonim session készül.

Ezért például az alábbi operátori műveletek nem csak a következő loginra hatnak:

- password change;
- role change;
- account disable/enable;
- TOTP enrollment vagy disable.

Ez különösen fontos role-visszavonásnál: nem kell megvárni a korábbi session természetes lejáratát.

7.5. Session store: memory vagy Redis

Development/test környezetben használható in-memory session store. Productionban, különösen több server instance esetén, Redis legyen a shared session backend.

```
[redis]
url_file = "/run/secrets/rwlang/redis-url"
allow_insecure = false
```

Redis az RWLangban több rövid életű infrastruktúra-állapotot is hordozhat:

- shared session;
- TOTP replay védelem;
- login rate limiting;
- public cache.

Redis nem üzleti source of truth. Disaster recovery után elfogadható az üres session/cache state: a felhasználók újra belépnek, a cache újraépül.

7.6. Cookie policy

Normál production módban a session cookie neve:

```
__Host-rw_session
```

A szerver `Path=/`, `HttpOnly`, `Secure` attribútumokat használ. Alapesetben `SameSite=Lax`; credentialed cross-origin konfiguráció esetén `SameSite=None`; `Secure`.

Developmentben az explicit `-insecure-dev-cookies` escape hatch miatt a cookie neve `rw_session`, és nincs `Secure`. Ezt internetre kitett production deploymentben ne használj.

7.7. Logout

A logout szándékosan POST művelet:

```
<form method="post" action="/_rw/auth/logout">
  <input type="hidden" name="_csrf" value="{ { csrfToken } }">
  <button type="submit">Logout</button>
</form>
```

A szerver ellenőrzi a CSRF token, invalidálja a régi sessiont, létrehoz egy friss anonim sessiont, új cookie-t küld, majd auditálja a logout eseményt. Így a kijelentkezés nem egyszerű kliensoldali cookie-törlés.

7.8. LDAP / LDAPS vállalati identityhez

Shared vagy vállalati identity esetén használható LDAP backend. Productionban LDAPS URL-t használj, a service bind credential pedig secret file-ből jöjjön.

Példa konfiguráció:

```
[auth]
ldap_url = "ldaps://ldap.example.com:636"
ldap_search_base = "ou=people,dc=example,dc=com"
ldap_username_attribute = "uid"
ldap_service_bind_dn_file = "/run/secrets/rwlang/ldap-bind-dn"
ldap_service_bind_password_file = "/run/secrets/rwlang/ldap-bind-password"
require_totp = true
```

Local auth és LDAP ne legyen egyszerre elsődleges identity backend ugyanabban a processben. Ezzel nincs bizonytalan backend-prioritás vagy azonos username-ek közötti névütközés.

LDAP használatakor a role mapping trusted szerveroldali konfigurációból jöjjön. A kliens által küldött `role`, `username` vagy hasonló mező soha nem authorization authority.

7.9. TOTP és recovery code

Local auth userhez TOTP enrollment:

```
/usr/local/bin/rwlang-cli auth totp-enroll \
  --db-url-file /run/secrets/rwlang/local-auth-db-url \
  --username alice
```

A CLI megadja az enrollmenthez szükséges secret/URI adatot és recovery kódokat. A recovery kódok a local auth DB-ben csak hashként maradnak meg, és egy sikeresen felhasznált recovery kód elfogy.

TOTP kikapcsolása:

```
/usr/local/bin/rwlang-cli auth totp-disable \
  --db-url-file /run/secrets/rwlang/local-auth-db-url \
  --username alice
```

Ha `require_totp = true`, az a user sem léphet be, akinek egyébként helyes a jelszava, de nincs használható második faktora.

A login form `totp` mezőjébe hatjegyű TOTP vagy local auth esetén recovery code kerülhet. Redis backend mellett a TOTP replay állapot shared, ezért több server instance is ugyanazt a

replay-védelmet látja.

7.10. Route authorization

Az alkalmazás route szinten három gyakori policyt deklarálhat:

```
route account GET "/account" auth user => account;
route admin GET "/admin" auth role Admin => admin;
route security GET "/security" auth mfa => security;
```

Jelentésük:

- **auth user:** bármely autentikált principal;
- **auth role Admin:** autentikált principal a trusted sessionben szereplő Admin role-lal;
- **auth mfa:** autentikált principal sikeresen igazolt második faktorral.

HTML route-nál a nem autentikált kérés a login oldalra irányítható. JSON API-nál ugyanez nem HTML redirectként jelenik meg, hanem 401 Unauthorized válaszként; ez API kliensnél kiszámíthatóbb viselkedés.

7.11. A principal használata az oldalon

Az autholt oldal trusted sessionadatot kap:

```
page fn account(ctx: PageContext) -> Result<Html, PageError> {
  return Ok(html {
    <main>
      <h1>Account</h1>
      <p>Signed in as {{ authPrincipal }}</p>
      <form method="post" action="/__rw/auth/logout">
        <input type="hidden" name="_csrf" value="{{ csrfToken }}">
        <button type="submit">Logout</button>
      </form>
    </main>
  });
}

route account GET "/account" auth user => account;
```

A `authPrincipal`, a session role-jai és az MFA állapot szerveroldali trusted state. Ne próbáld ezeket hidden form mezőből vagy query parameterből visszaépíteni.

7.12. Route role nem mindig elég

A route-level role azt mondja meg, hogy a felhasználó beléphet-e egy műveletbe. Objektum-szintű authorizationnál ennél több kellhet.

Példa: egy szerkesztő csak a saját draftját módosíthatja. Ilyenkor nem elég az, hogy **auth role Editor**: az actionnek a DB-ből olvasott objektum tulajdonosát is össze kell vetnie a trusted session principaljával.

Az alapelv:

identity a sessionből, objektumállapot a DB-ből, döntés szerveroldalon

A kliens által küldött `owner`, `actor` vagy `role` mező legfeljebb adat, nem `authority`.

Az object authorization typed guard

Az `authorize guard` csak már betöltött, **nem optional** model recordon használható. A compiler azt is ellenőrzi, hogy az `owner` mező létezik és `String` típusú. Ezért egy `Result<Article?, DbError>` eredményt előbb explicit módon kezelni kell; `authorization` nem értelmez „nincs rekord” állapotot.

```
let article = loadArticle(db, id)?;  
authorize article owner authorUsername or role Publisher;
```

Az ilyen guardot tartalmazó handler route-ja nem lehet `public`, és `public cache` sem kerülhet object authorization elé. A tulajdonos vagy a felsorolt `trusted role` engedélyez; minden más autentikált `actor` `403 Forbidden` választ kap.

7.13. Login rate limiting

A login külön `rate-limit` policyt kap. Például:

```
[auth]  
login_max_attempts = 8  
login_window_secs = 300
```

A szerver a `trusted proxy` szabályok után meghatározott `effective client IP`-t és a `kanonikus usernevet` használja a `limiter` kulcsához. Ezért `reverse proxy` mögött a `trusted_proxy_cidrs` `auth security` szempontból is fontos.

Sikertelen `credential` vagy második faktor nem ad részletes hibát a kliensnek. `Rate-limit` esetén `429 Too Many Requests`, `auth backend elérhetetlenség`nél `503 Service Unavailable` a megfelelő kategória.

7.14. CSRF, Origin és Fetch Metadata

`Session-cookie` alapú autentikációnál a böngésző automatikusan elküldi a `cookie`-t, ezért `state-changing` kérésnél `CSRF-védelem` kell. Az `RWLang defense-in-depth` modellje több réteget használ:

- `session-bound CSRF token`;
- `same-origin Origin/Referer policy`;
- `Fetch Metadata kontroll`;
- szigorú `request parsing`;
- `trusted proxy policy`.

A `CORS` nem helyettesíti a `CSRF`-et. Ha külön `frontend origin`hez `credentialed CORS`-t engedélyezel, az külön `security policy` és `HTTPS`-et igényel.

7.15. Auth audit

A `login/logout` események `audit log`ba kerülnek. Hasznos mezők például a `request ID`, `actor`, `outcome` és `effective client IP`. Az `audit` célja eseményrekonstrukció, nem `credential-debugging`.

Soha ne `logold`:

- a jelszót;
- TOTP secretet vagy recovery code-ot;
- session ID-t vagy teljes Cookie headert;
- CSRF tokenet;
- LDAP bind vagy DB/Redis credentialt.

A login eredménye lehet például siker, invalid credential, invalid second factor vagy rate limited; a kliensnek mutatott üzenet ettől szándékosan lehet kevésbé részletes.

7.16. Local auth vagy LDAP?

	Local auth	LDAP/LDAPS
Identity store	külön SQLite DB	vállalati directory
Tipikus deployment	single-node	shared/vállalati identity
User admin	RWLang CLI	directory admin workflow
Role forrás	local auth store	trusted mapping
TOTP	local enrollment + recovery	server-side TOTP policy/secrets
Multi-instance identity	nem ez a cél	igen

Ha az alkalmazásnak saját publikus regisztráció, email-es password reset vagy önkiszolgáló MFA enrollment kell, azt ne próbáld a meglévő operátori CLI köré vékony route-tal odabarkácsolni. Ezek token-életciklust, email securityt, rate limitet és külön recovery threat modelt igényelnek.

7.17. Rich text és autentikált CMS

Az autholt CMS-szerkesztői felület sem indokolja raw HTML tárolását. A szerkesztői jogosultság és a content sanitization két külön védelmi réteg. Tartalomként Markdownot tárolj, megjelenítéskor pedig a biztonságos renderert használd:

```
<article class="article-body">
  @markdown(article.body)
</article>
```

Az @markdown nem enged raw HTML-t, és a link targeteket allowlist szerint kezeli.

7.18. Production ellenőrzőlista

Mielőtt autentikációt engedsz productionben, ellenőrizd:

- pontosan egy elsődleges identity backend aktív;
- local auth DB elkülönül az application DB-től;
- jelszó és service credential csak secret file-ból jön;
- production sessionhoz Redis van, ha több server instance fut;
- a cookie Secure/HttpOnly és nincs `-insecure-dev-cookies`;
- a `trusted_proxy_cidrs` csak a valódi Apache/Nginx proxy címét fedi;
- a login limiter be van állítva;
- a state-changing formok CSRF-et használnak;
- role helyett objektumszintű authorization is van, ahol szükséges;
- credential vagy session secret nem kerül logba;
- account disable/role/TOTP változás session-visszavonását tesztelted.

8. fejezet

Fájlok, feltöltés és médiatár

Egy webalkalmazásban a fájlkezelés nem egyszerűen annyi, hogy a kliens által küldött fájlnevet bemásoljuk egy könyvtárba. A feltöltés egyszerre érinti a request feldolgozását, az erőforrás-korlátokat, a fájlrendszer-biztonságot, az adatbázist, a jogosultságkezelést, a backupot és – képek esetén – a HTML-kimenetet is.

Az RWLang ezért két külön szintet ad. Az általános `Upload` feltöltési metaadatot ad egy `AppFs`-be streamelt fájlhoz, az `Image` pedig ennél szigorúbb, validált képtípus, amelyet CMS- és wiki-felületen érdemes használni.

8.1. A mentális modell: három külön dolog

Feltöltésnél tartsd külön a következőket:

1. **kliens-metaadat**: eredeti fájlnev és a kliens által állított MIME-típus;
2. **tárolás**: a runtime által választott, `AppFs`-en belüli szerveroldali fájl;
3. **üzleti referencia**: amit az alkalmazás adatbázisban tárol és később felhasznál.

A kliens fájlneve nem storage path, a kliens `Content-Type` fejléce pedig nem biztonsági bizonyíték. Ezt a két értéket legfeljebb megjelenítési vagy audit-metaadatként kezeld.

8.2. Általános fájlfeltöltés: `Upload`

A route deklarációja megmondja a runtime-nak, hogy multipart fájlt vár, és melyik statikus `AppFs` alkönyvtárba helyezze azt:

```
route uploadFile POST "/upload"
  upload file<Upload> to "uploads"
  => uploadFile;
```

Az action már nem a teljes fájlt kapja meg memóriában. Az Upload a feltöltés metaadatait adja át:

```
action fn uploadFile(ctx: ActionContext, file: Upload)
  -> Result<Redirect, PageError> {
    let savedPath = file.path;
    let originalName = file.filename;
    let contentType = file.contentType;
    let byteCount = file.bytes;
    return Ok(redirect("/uploaded"));
  }
```

Ez fontos erőforrás-védelmi tulajdonság: a runtime streameli a multipart tartalmat, ezért egy nagyobb fájl nem válik automatikusan ugyanekkora VM heap-allokációvá.

8.3. AppFs: az alkalmazás írható fájltere

Az AppFs az alkalmazás számára kijelölt, korlátozott fájlrendszer-rész. Productionban külön data rootot adj neki; ne keverd a program binárisával, a statikus assetek könyvtárával vagy a rendszer más írható területeivel.

Példa szerverkonfiguráció:

```
[storage]
data_root = "/srv/rwlang/data"
fs_mode = "rwc"
max_upload_bytes = 16777216
max_image_pixels = 40000000
```

A rövid capability-jelölések jelentése:

r olvasás;
w írás;
c létrehozás és az upload commitjához szükséges műveletek.

Feltöltéshez tipikusan **rwc** szükséges. A byte-limit operátori hard limit: az alkalmazáskód nem emelheti meg tetszőlegesen egy route kedvéért.

Linuxon az AppFs confinement célja, hogy az alkalmazás a kijelölt root alól path traversal, symlink vagy magic-link trükkel se tudjon kitörni. Ettől még a normál Unix ownership és permission beállítások is szükségesek: a service user csak azt a könyvtárat kapja meg írhatónak, amelyre valóban szüksége van.

8.4. Mi történik egy feltöltés során?

A biztonságos feltöltési útvonal lépései fejlesztői szemmel:

1. a runtime ellenőrzi a request méretkorlátaikat és a route upload-sémáját;
2. a fájlt streaming módon egy confined staging helyre írja;
3. a storage kulcsot a szerver generálja, nem a kliens fájlnevéből képezi;
4. typed feltöltésnél további szerveroldali tartalomellenőrzés fut;
5. sikeres feldolgozás után a fájl atomikusan commitolható;
6. handler- vagy runtime-hiba esetén a runtime cleanupot kísérel meg.

Ezért ne építs saját „*sanitize filename és írjuk ki valahová*” mechanizmust az AppFs mellett. Az a nyelv által biztosított biztonsági határt kerülné meg.

8.5. Képfeltöltés: preferáld az Image típust

CMS-ben, wikiben, hírportálon és adminfelületen a képekhez az általános Upload helyett az Image a megfelelő surface:

```
route saveHero POST "/admin/hero"  
  upload hero<Image> to "media"  
  auth role Editor  
=> saveHero;
```

Az Image ugyanazt a streaming multipart és AppFs útvonalat használja, de a szerver a tényleges fájl tartalmát is ellenőrzi. V1-ben PNG és JPEG fogadható el. A kliens által küldött MIME-típus nem authority.

Az SVG szándékosan nem általános képfeltöltési formátum: aktív tartalmat és script/XSS felületet hordozhat. A GIF, WebP vagy AVIF támogatását se feltételezd, ha az adott runtime-verzió dokumentációja nem mondja ki.

8.6. Byte-limit és pixel-limit nem ugyanaz

Képnél két külön erőforrás-probléma van. Egy fájl lehet kevés byte, miközben a dekódolt kép óriási pixelszámú. Ezért a `max_upload_bytes` mellett külön `max_image_pixels` limit is van.

```
max_upload_bytes = 16777216  
max_image_pixels = 40000000
```

A második limit a `width * height` értéket korlátozza. A production konfigurációban mindkettőt tudatosan állítsd be az alkalmazás feladatához; a „16 MB-os upload limit” önmagában nem kép-dekódolási védelem.

8.7. Kép referencia az adatbázisban

Az Image first-class scalar, ezért model mezőként és adatbázisban is tárolható canonical reprezentációval. Az alkalmazásnak nem kell – és nem is érdemes – a belső storage path részeit kézzel összeraknia vagy szétszednie.

Tipikus üzleti modell:

```
model Article {  
  id: Int  
  slug: Slug  
  title: String  
  body: String  
  hero: Image  
}
```

A lényeg az, hogy az adatbázisban *referencia* legyen, ne kliens fájlnev és ne kézzel gyártott publikus URL. Így a tárolási és a HTTP-kiszolgálási szabályok a runtime kontrollja alatt maradnak.

8.8. Biztonságos megjelenítés HTML-ben

Egy validált képet nem közvetlen string-interpolációval jelenítünk meg, hanem az erre szolgáló renderelővel:

```
page fn article(ctx: PageContext, article: Article)
  -> Result<Html, PageError> {
    return Ok(html {
      <article>
        <h1>{{ article.title }}</h1>
        @image(article.hero, article.title)
        @markdown(article.body)
      </article>
    });
  }
```

Az `@image(image, alt)` a validált kép-referenciából készít HTML-t. Az `alt` szöveget escape-eli, a validált méreteket felhasználja, és a beépített read-only media endpointot célozza. Az `Image` közvetlen `{ image }` interpolációja compiler error, így a typed referencia nem csúszhat át véletlenül raw URL vagy markup szerepbe.

8.9. A beépített media endpoint

A runtime fenntartja a következő URL-prefixet:

```
/__rw/media/
```

Az alkalmazás ezzel ütköző route-ot nem deklarálhat. A media endpoint nem teszi HTTP-n böngészhetővé a teljes `data_root` könyvtárat: csak olyan AppFs könyvtárból szolgál ki képet, amelyet a program valamely `upload ...<Image> to "..."` route-ja deklarált.

Kiszolgáláskor a szerver a tartalmat ismét a képdetektor alapján kezeli, nem a kliens eredeti MIME-jára hagyatkozik. A válaszhoz biztonsági és cache headerek társulnak; a média kiszolgálása nem futtat alkalmazás-route-ot és nem hoz létre sessiont.

8.10. CMS-példa: hero kép feltöltése

Egy szerkesztői felületnél a gyakorlati folyamat általában két üzleti lépésből áll:

1. a szerkesztő feltölti a validált képet;
2. az alkalmazás az így kapott typed képreferenciát az adott cikkhez rendeli.

A route legalább autentikált legyen, admin/szerkesztő funkcionál pedig role policy is indokolt:

```
route saveHero POST "/admin/articles/:id<Int>/hero"
  upload hero<Image> to "media"
  auth role Editor
  => saveHero;
```

Ne abból indulj ki, hogy „csak kép, tehát veszélytelen”. Az upload route ugyanúgy state-changing POST: auth, CSRF/origin védelem, request-limit és auditálható üzleti művelet tartozhat hozzá.

8.11. Adatbázis és AppFs együtt alkothat üzleti állapotot

Ha egy adatbázissor egy AppFs-ben tárolt képre hivatkozik, akkor a két tároló együtt alkotja a helyreállítandó állapotot. Ez közvetlenül érinti a backup-stratégiát.

Például nem tekinthető automatikusan konzisztens mentésnek az, ha délelőtt készül egy DB dump, délután pedig külön tar a data rootról. Közben létrejöhetett vagy törlődhetett olyan referencia, amely csak az egyik snapshotban szerepel.

Production baseline-ként kontrollált stop/drain melletti, összerendelt application DB + local-auth DB + AppFs snapshotot kezelj egy recovery pontként. Redis session/cache/rate-limit állapota ezzel szemben nem üzleti source of truth.

8.12. Statikus asset és feltöltött fájl nem ugyanaz

A programhoz tartozó CSS, JavaScript, ikon és egyéb verziózott asset más életciklusú, mint a felhasználói feltöltés. A static root legyen read-only deploy-artifact; az AppFs data root pedig külön, kontrolláltan írható runtime adat.

Ez a szétválasztás több hibát megelőz:

- deploy nem írja felül a felhasználói médiát;
- upload nem módosíthat alkalmazáskódot vagy verziózott assetet;
- backupnál egyértelmű, mi üzleti adat és mi újratelepíthető release-artifact;
- a reverse proxy szabályai is tisztábban kialakíthatók.

8.13. Mit ne csinálj?

- Ne használd a kliens fájlnevét közvetlen filesystem pathként.
- Ne bízz a multipart **Content-Type** értékében.
- Ne tedd publikussá a teljes AppFs data rootot Apache/Nginx aliasszal.
- Ne interpolálj **Image** értéket kézzel URL-be vagy HTML-be.
- Ne kezeld a DB-t és az AppFs-t egymástól független recovery state-ként, ha vannak köztük referenciák.
- Ne kerüld meg az operátori upload- és pixel-limiteket alkalmazáskódból.

8.14. A jelenlegi media surface határai

A V1 media felülete tudatosan kicsi. Ne építs dokumentálatlan feltételezésekre. A jelenlegi alap nem ígér automatikus thumbnail/resize pipeline-t, WebP/AVIF konverziót, EXIF-normalizálást vagy metadata strippinget, teljes admin media browsert, S3/object-storage backendet vagy általános Markdown image syntaxot.

Ha az alkalmazásnak ezek valamelyike szükséges, azt külön architekturális feladatként kezeld, saját erőforrás-, security- és lifecycle-szabályokkal.

8.15. Fejlesztői ellenőrzőlista feltöltéshez

- | | |
|--------------------------------|---|
| 1. Auth/authorization rendben? | 2. AppFs root elkülönített és nem publikus? |
| 3. Ésszerű byte-limit? | 4. Képnél pixel-limit is? |
| 5. Képhez Image ? | 6. Filename/MIME csak metaadat? |
| 7. DB-ben typed referencia? | 8. HTML-ben @image ? |
| 9. DB + AppFs együtt mentve? | 10. Nincs reverse-proxy data-root alias? |

9. fejezet

JSON API, CORS és böngészős kliensintegráció

Az RWLang nem kényszerít választásra a klasszikus server-renderelt HTML és a JSON API között. Ugyanabban az alkalmazásban lehetnek HTML-t visszaadó oldalak, formot feldolgozó actionök és typed JSON endpointok. Ez hasznos akkor is, ha csak egy kisebb JavaScript komponens kér adatot, és akkor is, ha külön frontend alkalmazás használja az RWLang szerveret API-ként.

A fejezet célja három működési modell tiszta szétválasztása:

1. **same-origin HTML + API**: a böngésző ugyanarról az originről tölti az oldalt és hívja az API-t;
2. **cross-origin publikus API**: külön origin hívja az API-t, de nincs session-cookie;
3. **cross-origin session auth**: külön frontend origin használ cookie-alapú sessiont, ezért CORS és CSRF policy együtt szükséges.

A legfontosabb alapelv: **a CORS nem autentikáció és nem CSRF-védelem**. A CORS böngészőoldali hozzáférési policy; az auth azt dönti el, ki a felhasználó, a CSRF pedig azt védi, hogy egy autentikált session ne legyen más originről jogosulatlanul felhasználható állapotmódosításra.

9.1. Az első typed JSON endpoint

Egy JSON endpoint handlerének visszatérési típusa `Json`. A `json(...)` a támogatott runtime értéket JSON reprezentációvá alakítja.

```
page fn statusApi(ctx: PageContext) -> Result<Json, PageError> {
  let healthy = true;
  return Ok(json(healthy));
}

route statusApi GET "/api/status" => statusApi;
```

A sikeres válasz reprezentációja JSON, például:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

true
```

A `json(value)` nem tetszőleges objektum-serializáló. A támogatott értékek kötöttek: scalar értékek, model record, optional model és modell-lista. Ez illeszkedik az RWLang typed boundary szemléletéhez: a response alakja nem dinamikus, requestenként változó adatszerkezetből épül fel.

9.2. Adatbázisból JSON lista

A typed SQL és a JSON response közvetlenül összekapcsolható. A query továbbra is pontos model shape-et ad vissza, a JSON réteg pedig ezt reprezentálja.

```
model Product {
  id: Int
  name: String
  price: Int
}

query fn listProducts(db: Db, limit: Int, offset: Int)
  -> Result<List<Product>, DbError> sql {
  SELECT id, name, price
  FROM products
  ORDER BY id
  LIMIT :limit OFFSET :offset
}

page fn productsApi(ctx: PageContext, db: Db,
  limit: Int, offset: Int)
  -> Result<Json, PageError> {
  let products = listProducts(db, limit, offset)?;
  return Ok(json(products));
}

route productsApi GET "/api/products"
  query limit<Int> offset<Int>
  validate limit range 1 100 offset range 0 1000000
  => productsApi;
```

A fontos rész itt nem a JSON, hanem az, hogy az input ugyanúgy typed és validált, mint HTML route esetén. A kliens nem tudja például egy korlátlan `limit` értékkel megkerülni az alkalmazás route policy-ját.

9.3. Typed JSON POST body

JSON request bodyhoz a route `json` input módot deklarál.

```
action fn echoApi(ctx: ActionContext,
                  name: String,
                  age: Int,
                  active: Bool)
  -> Result<Json, PageError> {
  return Ok(json(name));
}

route echoApi POST "/api/echo"
  json name<String> age<Int> active<Bool>
  validate name length 1 100 age range 0 150
  auth user
  => echoApi;
```

Egy megfelelő kérés:

```
POST /api/echo HTTP/1.1
Content-Type: application/json
Accept: application/json
X-CSRF-Token: <session CSRF token>

{"name":"Alice","age":42,"active":true}
```

A JSON input schema **zárt**. A runtime hibának tekinti többek között az ismeretlen vagy hiányzó kulcsot, a duplikált kulcsot, a rossz scalar típust, valamint a nem támogatott nested object, array, float vagy `null` inputot. Ezek request-contract hibák, ezért **400 Bad Request** választ adnak. Hibás vagy hiányzó **Content-Type** JSON route-on **415 Unsupported Media Type**. A mező- és request-méretlimiterek ugyanúgy érvényesek, mint más request body esetén.

A `json`, `form` és `upload` body módok ugyanazon POST route-on kölcsönösen kizárják egymást, és ezt már a compiler ellenőrzi. JSON route `application/json`, egyszerű form route `application/x-www-form-urlencoded`, upload route pedig `multipart/form-data` requestet vár. Egy endpointnak legyen egyértelmű request contractja.

9.4. Content negotiation és a 406 válasz

A kliens az `Accept` headerrel jelezheti, milyen reprezentációt tud fogadni. JSON route-nál támogatott például:

```
Accept: application/json
Accept: application/*
Accept: */*
```

Ha a handler JSON választ adna, de a kliens kizárólag `text/html` típust fogad el, a szerver **406 Not Acceptable** választ ad. Ez jobb annál, mintha a szerver csendben olyan reprezentációt küldene, amelyet a kliens explicit elutasított.

9.5. API hibák

JSON route-on a runtime által klasszifikált alkalmazáshibák ugyanazt az egyszerű envelope-ot használják: egyetlen stabil **error** kódot adnak vissza, miközben a részletes diagnosztika a szerveroldali logban marad. A kliens ne angol hibaüzenetek szövegét parse-olja, hanem a HTTP státuszt és ezt a kódot kezelje.

JSON-return route felismerése után a hibák JSON error envelope-ban érkeznek. Például:

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json; charset=utf-8

{"error": "unauthorized"}
```

Tipikus error kódok:

- `bad_request;`
- `unauthorized;`
- `forbidden;`
- `csrf_failed;`
- `unsupported_media_type;`
- `database_unavailable;`
- `resource_limit;`
- `internal_error.`

Ez különösen fontos böngészős kliensnél. Egy autentikált JSON route névtelen kérésre 401-et ad, nem HTML login redirectet. A frontend így státuszkód alapján tud dönteni, nem kell HTML választ API hibaként értelmeznie.

9.6. Same-origin: az alapértelmezett és legegyszerűbb modell

Ha a HTML és az API ugyanazon scheme + host + port kombinációról érhető el, a böngésző szempontjából same-origin alkalmazásról beszélünk.

```
https://app.example/
https://app.example/api/products
```

Ilyenkor általában nincs szükség CORS konfigurációra. Egy egyszerű kliens:

```
const response = await fetch("/api/products?limit=20&offset=0", {
  headers: { Accept: "application/json" }
});

if (!response.ok) {
  throw new Error(`HTTP ${response.status}`);
}

const products = await response.json();
```

Ez legyen az alapértelmezett architektúra akkor, ha nincs valós üzleti igény külön frontend originre. Kevesebb trust boundaryt, egyszerűbb cookie policy-t és kevesebb deployment-konfigurációt jelent.

9.7. CORS: exact origin allowlist

Cross-origin böngészős hozzáférés alapból nincs engedélyezve. A trusted server configban konkrét origineket kell felsorolni; wildcard origin nincs.

```
[web]
cors_origins = ["https://frontend.example"]
cors_allow_credentials = false
```

A CLI megfelelője:

```
rwlang-server --cors-origin https://frontend.example ...
```

Több originhez több explicit allowlist elem tartozzon. Ne építs olyan szabályt, amely requestből származó `Origin` értéket vakon visszatükröz.

9.8. Preflight

Bizonyos cross-origin requestek előtt a böngésző `OPTIONS` preflight kérést küld. Az RWLang csak akkor enged preflightot, ha:

- az `Origin` konfigurált;
- a cél GET/POST route létezik;
- a kért request header támogatott.

A támogatott browser API headerek között szerepel a `Content-Type`, az `Accept` és az `X-CSRF-Token`. A preflight tehát nem általános „mindent engedő” `OPTIONS` endpoint, hanem a tényleges route- és origin-policy része.

9.9. Credentialed CORS session-cookie-val

Ha a frontend külön originről használ RWLang session authot, credentialed CORS szükséges:

```
[web]
cors_origins = ["https://frontend.example"]
cors_allow_credentials = true
```

Productionben ez csak HTTPS mellett engedélyezett. A session cookie ilyen konfigurációban `SameSite=None; Secure` tulajdonságot kap, mert a böngészőnek cross-site requestben is el kell küldenie.

A JavaScript oldalon a fetch kérésnek is explicit credential mód kell:

```
const response = await fetch("https://api.example/api/account", {
  credentials: "include",
  headers: { Accept: "application/json" }
});
```

A `credentials: "include"` nem jogosultságot ad. Csak azt mondja a böngészőnek, hogy a megfelelő cookie-kat a CORS policy által megengedett requesthez elküldheti.

9.10. CSRF JSON API-n

Cookie-alapú sessionnél a böngésző a session cookie-t automatikusan kezeli, ezért állapotmódosító JSON requestnél ugyanúgy CSRF-védelem kell, mint HTML formnál.

Form POST esetén a token body mező:

```
_csrf=<token>
```

JSON POST esetén header:

```
X-CSRF-Token: <token>
```

Külön frontend originhez készíthető autentikált token endpoint:

```
page fn csrfApi(ctx: PageContext) -> Result<Json, PageError> {  
  return Ok(json(csrfToken));  
}  
  
route csrfApi GET "/api/csrf" auth user => csrfApi;
```

A token ugyanahhoz a sessionhöz tartozik. A CORS allowlist önmagában nem helyettesíti ezt az ellenőrzést.

9.11. Teljes böngészős POST folyamat

Egy külön `frontend.example` originről futó session-auth frontend tipikus lépései:

1. a felhasználó autentikálódik, a böngésző megkapja a session cookie-t;
2. a frontend lekéri a sessionhöz tartozó CSRF token;
3. a POST kérés `credentials: "include"` módban megy;
4. a request JSON bodyja megfelel a route zárt typed sémájának;
5. az `X-CSRF-Token` header a session tokenjét tartalmazza;
6. a runtime ellenőrzi az origint, authot, CSRF-et és input-validációt;
7. a handler csak ezután hajt végre üzleti műveletet.

Példa kliens:

```
const csrf = await fetch("https://api.example/api/csrf", {  
  credentials: "include",  
  headers: { Accept: "application/json" }  
}).then(async r => {  
  if (!r.ok) throw new Error(`CSRF HTTP ${r.status}`);  
  return r.json();  
});  
  
const result = await fetch("https://api.example/api/echo", {  
  method: "POST",  
  credentials: "include",  
  headers: {  
    "Content-Type": "application/json",  
    Accept: "application/json",  
    "X-CSRF-Token": csrf  
  },  
  body: JSON.stringify({ name: "Alice", age: 42, active: true })  
});
```

```
}).then(async r => {
  if (!r.ok) throw new Error(`API HTTP ${r.status}`);
  return r.json();
});
```

9.12. Reverse proxy és origin

Apache vagy Nginx mögött különösen fontos, hogy a public scheme/host és a trusted proxy konfiguráció összhangban legyen. Az alkalmazás ne a kliens által szabadon küldött forwarded headerekből találja ki, hogy milyen origin tekinthető biztonságosnak. A trusted proxy boundary alapját a telepítési fejezet mutatja be; a teljes `server.toml` kulcsreferencia a könyv végén található.

A CORS allowlistbe mindig a **böngésző által látott frontend origin**t írd, például:

```
https://frontend.example
```

nem pedig a belső proxy vagy container címet.

9.13. Mit ne tegyél?

- Ne engedélyezz CORS-t csak azért, mert „API-ról van szó”.
- Ne tükrözd vissza tetszőleges request `Origin` értékét.
- Ne tekintsd a CORS-t auth- vagy CSRF-mechanizmusnak.
- Ne küldj session credentialt HTTP-n productionben.
- Ne keverd ugyanazon POST route-on a form, JSON és upload body módot.
- Ne hagyd korlátlanul a listázó query paramétereit; az API input is validált input.
- Ne építs klienslogikát HTML login redirect köré JSON route esetén; kezeld a 401/403 státuszt.

9.14. Melyik architektúrát válaszd?

Helyzet	Ajánlott modell
szerver-renderelt oldal + kisebb JS	same-origin HTML + JSON API
azonos domainen futó SPA	same-origin API, CORS nélkül
külön publikus frontend, nincs session	exact-origin CORS credentials nélkül
külön frontend + session auth	exact-origin credentialed CORS + HTTPS + CSRF
server-to-server kliens	CORS nem releváns; az auth és hálózati policy a lényeg

9.15. Kipróbálható példa

A repository teljes mintája: `examples/json-api/app.rw`.

Elsőként ugyanazon originről érdemes kipróbálni. Ezután csak akkor vezess be külön frontend origin

10. fejezet

Statikus assetek és frontend-integráció

A webalkalmazásnak nem csak dinamikus HTML- és JSON-válaszai vannak. CSS, ikon, kép, font, source map vagy egy külön frontend build artifactjai más életciklusú fájlok: a release részeként érkeznek, tartalmuk ideális esetben immutable, és a böngésző agresszíven cache-elheti őket.

Az RWLang ezért a statikus asseteket külön, **read-only static rootból** szolgálja ki. Ez nem ugyanaz a terület, mint az AppFs írható data rootja, és nem ugyanaz a fogalom, mint a felhasználó által feltöltött média.

A fejezet célja négy kérdés tisztázása:

1. hol legyen a CSS, kép, font és egyéb build artifact;
2. hogyan kapjon hosszú cache-élettartamot anélkül, hogy régi verzió ragadna a kliensnél;
3. mikor szolgálja ki az RWLang, és mikor érdemes Apache/Nginx elé tenni;
4. mit jelent a V1 CSP-modell a JavaScript szempontjából.

10.1. Három külön fájléletciklus

Egy production alkalmazásban legalább három fájlcsoportot érdemes külön kezelni:

	Példa	Írhatóság	Életciklus
release	CSS, font, logo	read-only	deploykor cserélődik
runtime data	export, generált fájl	AppFs policy szerint	alkalmazás futása közben
user media	feltöltött kép	kontrolláltan írható	üzleti adattal együtt

Tipikus könyvtárszerkezet:

```
/usr/local/bin/rwlang-server
/usr/local/bin/rwlang-cli
/srv/myapp/releases/2026-09-04/
  main.rw
  migrations/
  public/
    css/
    img/
```

```

    fonts/
  /srv/myapp/data/
  /run/secrets/rwlang/

```

A `public/` legyen deployment artifact. A `data/` ezzel szemben runtime állapot. Ha a kettőt összekevered, egy upload felülírhat release-fájlt, vagy egy rollback véletlenül felhasználói adatot cserélhet vissza.

10.2. A static root konfigurálása

A server configban a statikus namespace külön szekció:

```

[static_assets]
root = "/srv/myapp/current/public"
url_prefix = "/assets/"
max_asset_bytes = 8388608
max_age_secs = 300
immutable_max_age_secs = 31536000
precompressed = true

```

Ugyanez CLI-ből is beállítható. A fontosabb opciók:

```

--static-root
--static-url-prefix
--max-static-asset-bytes
--static-max-age-secs
--static-immutable-max-age-secs

```

A static prefix rezervált route namespace. Ha az URL prefix `/assets/`, az application ne deklaráljon ezzel ütköző dinamikus vagy statikus route-ot. Az ütközést a szerver fail-closed módon kezelje: jobb startupkor hibázni, mint requestenként bizonytalan route-választást végezni.

10.3. Fingerprint: a hosszú cache kulcsa

A statikus asset tartalma és URL-je legyen összekötve. Például:

```

public/
  css/app.01234567.css
  img/logo.a91f028c.svg
  fonts/ui-f1e2d3c4.woff2

```

Az RWLang fingerprinteltnek tekinti azt a fájlnevet, amelyben legalább 8 hexadecimális karakterből álló, ponttal, kötőjellel vagy underscore-ral határolt token szerepel. Ilyen fájlnál a response hosszú immutable cache-t kaphat:

```
Cache-Control: public, max-age=31536000, immutable
```

Nem fingerprintelt fájlnál a biztonságosabb alap rövid és revalidálható:

```
Cache-Control: public, max-age=300, must-revalidate
```

A lényeg: **ne ugyanazon URL mögött cseréld le a tartalmat egyéves cache mellett.** Új CSS buildhez új fingerprintelt fájlnev tartozzon. A régi fájl akár maradhat is a release-ben addig, amíg a régi HTML-reprezentációk kifuthatnak.

10.4. Egy minimális asset build pipeline

Az RWLang nem akar frontend build rendszert előírni. A pipeline feladata egyszerűen az, hogy determinisztikus outputot készítsen a `public/` könyvtárba.

Egy kis alkalmazásnál a forrás lehet például:

```
assets-src/  
  app.css  
  logo.svg  
public/
```

A release build lépései:

1. tiszta `public/` könyvtár létrehozása;
2. CSS/font/kép build vagy másolás;
3. content hash képzése;
4. fingerprintelt fájlnev létrehozása;
5. opcionális Brotli/gzip siblingek készítése;
6. az alkalmazásban a fingerprintelt URL használata.

A hash nem security token. A feladata az, hogy a fájlnev a tartalom verzióját azonosítsa.

10.5. ETag és conditional GET

Minden statikus reprezentáció saját ETag értéket kap. A böngésző később küldhet:

```
If-None-Match: "rw-..."
```

Ha a reprezentáció nem változott, a szerver `304 Not Modified` választ adhat body nélkül. GET mellett HEAD is támogatott.

A static request nem hoz létre sessiont és nem küld session cookie-t. Ez fontos teljesítmény- és security tulajdonság: egy CSS vagy font letöltése ne váljon session-lifecycle eseménnyé.

10.6. Precompressed Brotli és gzip

Az RWLang server request közben nem tömöríti a statikus fájlt. A release pipeline opcionálisan előre elkészítheti:

```
app.01234567.css  
app.01234567.css.br  
app.01234567.css.gz
```

A kliens `Accept-Encoding` értéke alapján a szerver a Brotli, majd gzip, végül az eredeti reprezentáció közül választ. Tömörített válasznál a header például:

```
Content-Encoding: br  
Vary: Accept-Encoding
```

Ennek két előnye van: nincs requestenkénti compression CPU-költség, és a web request nem nyit dinamikus tömörítési erőforrás-felületet.

10.7. MIME, nosniff és path confinement

A server ismert webes kiterjesztésekhez MIME típust rendel, ismeretlen extensionnél pedig `application/octet-stream` típust használ. A válasz mindig kap:

```
X-Content-Type-Options: nosniff
```

A static path szándékosan szigorú. Jó:

```
/assets/css/app.01234567.css
```

Tiltott többek között a `...`, encoded traversal, backslash, üres path komponens és control karakter. A V1 static pathban percent-encoding sincs; a build pipeline használjon URL-safe fájlneveket.

Linuxon a static root ugyanahhoz a confinement szemlélethez igazodik, mint az AppFs: a kiszolgálás nem válhat tetszőleges host filesystem olvasássá symlink vagy path traversal segítségével.

10.8. CSS, kép és font az RWLang HTML-ben

A V1 CSP same-origin style, image, font és media asseteket enged. Egy release-ben ezért természetes minta:

```
/assets/css/app.01234567.css  
/assets/img/logo.a91f028c.svg  
/assets/fonts/ui-f1e2d3c4.woff2
```

A HTML-ben mindig a release-hez tartozó fingerprintelt URL jelenjen meg. A sablon ne felhasználói inputból állítson elő asset URL-t; az assetnév a build része, nem request adat.

10.9. Fontos V1-határ: JavaScript kiszolgálható, de RWLang HTML-ben nem fut

A static server ismeri a `.js` és `.mjs` MIME típust, ezért ilyen build artifact fizikailag kiszolgálható. Ez azonban **nem jelenti azt, hogy az RWLang által renderelt HTML scriptet futtathat.**

A V1 HTML security modellben:

- a CSP nem nyit `script-src` engedélyt;
- a compiler a `<script>` markupt elutasítja;
- nincs inline-script vagy „unsafe” escape hatch.

Ez tudatos security határ. A könyv előző fejezetének JavaScript `fetch` példái ezért kétféleképpen értelmezendők:

1. külön frontend alkalmazás fut a saját originjén, és RWLang JSON API-t hív;

2. a kód a browser API kliens viselkedését szemlélteti, nem azt, hogy V1 RWLang HTML-be script tag írható.

Ha az alkalmazás klasszikus RWLang szerver-renderelt oldal, építs HTML + CSS + typed form/action modellre. Ha teljes JavaScript frontend szükséges, kezeld azt külön frontendként, és az RWLang legyen typed JSON backend.

10.10. RWLang szolgálja ki vagy Apache/Nginx?

Két működő deployment modell van.

1. RWLang-owned static namespace

Ez legyen az alapértelmezett. A reverse proxy minden requestet az RWLang szervernek ad, és az RWLang kezeli az `/assets/` namespace-t is:

```
Internet
|
Apache / Nginx :443
|
127.0.0.1:8080 RWLang
|-- application routes
`-- /assets/*
```

Előnye, hogy a fingerprint-, ETag-, precompressed-, nosniff- és path-policy egy helyen marad. Kis és közepes alkalmazásnál ez általában bőven elég.

2. A reverse proxy szolgálja ki közvetlenül

Nagy forgalomnál dönthetsz úgy, hogy az Apache vagy Nginx közvetlenül a read-only release public/ könyvtárból szolgál. Ekkor viszont **neked kell reprodukálnod** a fontos contractot:

- read-only, külön static root;
- traversal/symlink elleni biztonságos path-kezelés;
- helyes MIME + **nosniff**;
- fingerprintelt fájlokra hosszú immutable cache;
- nem fingerprintelt fájlokra rövid/revalidálható cache;
- Brotli/gzip esetén helyes **Vary**;
- a namespace ne ütközzön application route-tal.

Ez már operátori optimalizáció, nem szükséges ahhoz, hogy az alkalmazás helyes legyen.

10.11. Nginx minta közvetlen static kiszolgálásra

Egy egyszerű kiindulás:

```
server {
    listen 443 ssl;
    server_name app.example;

    location /assets/ {
        alias /srv/myapp/current/public/;
        try_files $uri =404;
        add_header X-Content-Type-Options nosniff always;
    }

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

Ez csak topológiai alaplanta. A production konfigurációban a cache policy-t, precompressed fájlokat és symlink/release szabályokat is ugyanahhoz az immutable deployment modellhez kell igazítani.

10.12. Apache minta közvetlen static kiszolgálásra

Apache esetén a gondolat ugyanaz:

```
Alias /assets/ /srv/myapp/current/public/

<Directory /srv/myapp/current/public/>
    Require all granted
    Options -Indexes
    AllowOverride None
    Header always set X-Content-Type-Options "nosniff"
</Directory>

ProxyPass          / http://127.0.0.1:8080/
ProxyPassReverse   / http://127.0.0.1:8080/
```

Ha Apache közvetlenül szolgálja az asseteket, az `Alias` szabálynak a proxy catch-all előtt kell érvényesülnie, és a cache headereket is explicit konfigurálni kell.

10.13. Deploy: asset és alkalmazáskód ugyanahhoz a release-hez tartozzon

A legkönnyebben elkerülhető assethiba az, amikor az új HTML már új CSS fájlnevet hivatkozik, de a static rootban még a régi release van – vagy fordítva.

Ezért a helyes sorrend:

```
build application
-> build/fingerprint assets
-> optional .br/.gz
-> immutable release directory
-> preflight
```

```
-> atomic current switch  
-> controlled restart/reload
```

Az alkalmazáskód és a `public/` könyvtár ugyanazon release artifact része legyen. Így rollback-kor mindkettő együtt áll vissza.

10.14. Ellenőrzés curl-lel

Deploy után legalább ezeket érdemes ellenőrizni:

```
curl -I https://app.example/assets/css/app.01234567.css  
curl -I -H 'Accept-Encoding: br' \  
  https://app.example/assets/css/app.01234567.css  
curl -I -H 'If-None-Match: "rw-..."' \  
  https://app.example/assets/css/app.01234567.css
```

Ellenőrizd a `Content-Type`, `Cache-Control`, `ETag`, `X-Content-Type-Options`, valamint tömörített reprezentációnál a `Content-Encoding` és `Vary` headert.

10.15. Gyakorlati döntési szabály

Helyzet	Ajánlott megoldás
szerver-renderelt RWLang oldal	RWLang static root + CSS/image/font
kis/közepes production	reverse proxy mindent RWLangnak továbbít
nagy statikus forgalom	Apache/Nginx direct static, azonos policy-vel
teljes JavaScript frontend	külön frontend deployment + RWLang JSON API
user upload	AppFs/media pipeline, nem static root

A legfontosabb szabály: **a statikus asset deployment artifact, nem alkalmazásadat**. Ha ezt a határt megtartod, a cache, rollback, backup és security modell egyszerre lesz egyszerűbb.

11. fejezet

Nagyobb alkalmazások szervezése

Egy néhány route-os alkalmazásban még kényelmes lehet minden modellt, lekérdezést és handlert egyetlen `main.rw` fájlban tartani. Wiki, CMS vagy ügyviteli rendszer esetén azonban gyorsan fontosabbá válik a *szerkezet*, mint az egyes nyelvi elemek szintaxisa. Az RWLang ehhez két egymást kiegészítő eszközt ad: a `mod` a forrásfájlok szervezésére, az `object` pedig a domain API névterének kialakítására szolgál.

11.1. Három külön kérdés

A nagyobb alkalmazás szerkezetét érdemes három külön réteggént kezelni:

Eszköz	Feladata
<code>mod</code>	Melyik fájlban, illetve forrásmodulban található a kód?
<code>object</code>	Melyik üzleti fogalomhoz tartozik a model, query, page vagy action?
<code>route</code>	Milyen HTTP felület és milyen auth/cache/rate-limit policy teszi elérhetővé?

Ez a szétválasztás tudatos. Egy domain object nem kap automatikusan adatbázist, current usert, fájlrendszert vagy hálózati hozzáférést, és egy modul sem jelent HTTP policy-t.

11.2. A `mod` nem szöveges `include`

Egy kisebb, többfájlos projekt már így nézhet ki:

```
myapp/  
|-- main.rw  
|-- models.rw  
|-- queries.rw  
|-- components.rw  
|-- pages.rw  
|-- actions.rw  
|-- migrations/  
`-- public/
```

A belépési pont:

```
mod models;  
mod queries;  
mod components;
```

```
mod pages;  
mod actions;
```

A `mod` nem C-szerű szövegbeillesztés, és nem önti a deklarációkat globális névtérbe. A compiler az explicit source graphot tölti be, ellenőrzi és egyetlen programmá fordítja. Egy modul saját névtérrel bír: például a `queries.rw` deklarációira más modulból `queries::name` alakban kell hivatkozni. A modulútvonal application-root relatív; nested forrásfájlt explicit, például `mod article::admin;` deklarációval kell felvenni. Nincs automatikus almappa-discovery vagy relatív `../` modulútvonal.

11.3. Mikor válts feature-alapú szerkezetre?

A `models.rw`, `queries.rw`, `pages.rw` típus szerinti bontás jó első lépés, de nagyobb alkalmazásnál könnyen azt eredményezi, hogy egyetlen feature módosításakor öt-hat fájlt kell párhuzamosan megnyitni. Ilyenkor jobb üzleti terület szerint bontani:

```
main.rw  
article.rw  
article/  
|-- admin.rw  
`-- components.rw  
account.rw  
account/  
`-- profile.rw  
invoice.rw  
invoice/  
`-- admin.rw  
routes.rw  
migrations/  
public/
```

A `main.rw` ilyenkor inkább alkalmazás-kompozíció:

```
mod article;  
mod article::admin;  
mod article::components;  
mod account;  
mod account::profile;  
mod invoice;  
mod invoice::admin;  
mod routes;
```

A cél nem a lehető legtöbb fájl. A cél az, hogy egy üzleti változtatás kódja többnyire egy feature könyvtáron belül maradjon.

11.4. Domain object: üzleti névtér, nem OOP-osztály

A globális nevek számát az `object` csökkenti. Például:

```

object Article {
  model {
    id: Int
    slug: Slug
    title: String
    body: String
    authorUsername: String
  }

  query fn bySlug(db: Db, slug: Slug)
    -> Result<Article, DbError> sql {
    SELECT id, slug, title, body, authorUsername
    FROM articles
    WHERE slug = :slug
  }

  page fn show(ctx: PageContext, db: Db, slug: Slug)
    -> Result<Html, PageError> {
    let article = Article.bySlug(db, slug)?;
    return Ok(html {
      <article>
        <h1>{{ article.title }}</h1>
        @markdown(article.body)
      </article>
    });
  }
}

```

A használat így beszédes:

```
let article = Article.bySlug(db, slug)?;
```

Később ugyanilyen természetesek a rövid domainnevek: `Invoice.approve`, `Invoice.cancel`, illetve `UserAccount.disable`.

Mit tartalmazhat egy object?

A V1 domain object egy model köré rendezheti a hozzá tartozó műveleteket. Az alábbi blokk csak szerkezeti váz, nem önmagában fordítható RWLang forrás:

```

object Article {
  model { ... }
  query fn bySlug(...) ...
  page fn show(...) ...
  action fn publish(...) ...
  component fn Card(...) ...
  layout fn EditorLayout(...) ...
}

```

Pontosan egy model blokk tartozik hozzá. Az object nem klasszikus objektumorientált osztály: nincs öröklés, mutable `self`, constructor-mágia, virtual dispatch vagy reflection.

11.5. A dependency továbbra is explicit

A domain namespace nem rejt el capabilityket. Ez jó:

```
query fn byId(db: Db, id: Int)
  -> Result<Article, DbError> sql {
    SELECT id, slug, title, body, authorUsername
    FROM articles
    WHERE id = :id
  }
```

Az action ugyanilyen explicit módon kapja meg a szükséges capabilityket. A következő blokk szándékosan rövidített szerkezeti példa; az állapotváltás konkrét typed queryjét a CRUD és üzleti-audit fejezetek mutatják be:

```

action fn publish(ctx: ActionContext, db: Db, id: Int)
  -> Result<Redirect, PageError> {
  let article = Article.byId(db, id)?;
  authorize article owner authorUsername or role Publisher;
  // ... állapotváltás tranzakcióban ...
  return Ok(redirect("/admin/articles"));
}

```

A handler aláírásából továbbra is látszik, hogy adatbázishoz fér hozzá. Ugyanez igaz AppFs-re, authra és később hálózati egressre is: a domain object nem válhat rejtett service locatorrá.

11.6. Miért maradjon a route top-level?

A route tudatosan nem olvad bele teljesen az objectbe:

```

route articleShow GET "/cikk/:slug<Slug>"
  cache public ttl 60
  => article::Article.show;

route articlePublish POST "/admin/cikk/:id<Int>/publish"
  auth role Publisher
  invalidate cache articleShow
  => article::Article.publish;

```

Így egy route-registry vagy egy code review során egy helyen látszik:

- az URL és a HTTP method;
- az authentication/authorization követelmény;
- a cache policy és invalidáció;
- a rate limit vagy resource profile;
- a logokban és auditban megjelenő globális route-név.

A domain member neve lehet rövid és helyi jelentésű, például `Article.publish` a saját modulján belül; külön `routes.rw` fájlból `article::Article.publish` szükséges. A route neve viszont maradjon globálisan beszédes, például `articlePublish`.

11.7. Ajánlott névkonvenció

Elem	Forma	Példa
object	PascalCase üzleti főnév	Article, Invoice, UserAccount
member	camelCase, domainen belül rövid	bySlug, publish, approve
route	globálisan auditálható név	articleShow, invoiceApprove
module	feature vagy technikai egység	article, account, routes

11.8. Egy CMS lehetséges szerkezete

Egy közepes CMS-nél jó kompromisszum lehet:


```
main.rw
routes.rw
article.rw          # Article object + alap queryk
article/
|-- admin.rw        # szerkesztes/publikalas
`-- components.rw   # Card, listak
media.rw
media/
`-- admin.rw
account.rw
migrations/
public/
`-- app.css
```

A publikus és admin HTTP policy továbbra is a route-rétegben marad. Az article feature viszont együtt tartja a cikk üzleti modelljét és műveleteit.

11.9. Mit ne csinálj?

Ne készíts minden deklarációból külön fájlt

A túl apró modulok több navigációt okoznak, mint amennyi szerkezetet adnak. Egy ötven soros feature-nek nem kell nyolc fájl.

Ne építs rejtett „service” réteget

Ha egy művelet DB-t, AppFs-t vagy auth contextet használ, az maradjon látható a typed aláírásban. Az RWLang egyik értéke éppen az, hogy a capability-határ explicit.

Ne duplikáld a route policy-t a handlerben

A route-level auth, cache, rate limit és resource policy deklaratív szerződés. A domain-specifikus objektumszintű authorization természetesen a handlerben maradhat, mert az üzleti rekordtól függ.

Ne használj szabad Stringet üzleti állapotnak

Ha egy domain Draft, Review, Published vagy Cancelled állapotokból áll, használd az enumot. A projektstruktúra akkor segít igazán, ha a domain contract is typed.

11.10. Refaktorálási út: egy fájlból nagyobb alkalmazás

Nem kell a projekt első napján végleges könyvtárhierarchiát tervezni. Jó evolúciós út:

1. Kezdj egy `main.rw` fájlal, amíg az alkalmazás kicsi.
2. Válaszd külön a `model/query/page/action` csoportokat, amikor a fájl már nehezen áttekinthető.
3. Ha egy feature több technikai fájlra szétszóródik, válts feature-alapú modulokra.
4. Ha sok globális üzleti név keletkezik, rendezd őket `object` namespace-be.

III. rész

Gyakorlati alkalmazások

12. fejezet

Gyakorlati alkalmazások: wiki és CMS

A következő két minta ugyanazokra az alapokra épül, de eltérő termékigényeket mutat. A minimális wiki a stabil slugot, Markdown tartalmat és autentikált szerkesztést kapcsolja össze; az érettebb wikihez később optimistic locking és audit trail adható. A CMS-minta ehhez többfájlos projektstruktúrát, szerkesztői jogosultságot és public cache-t ad, majd külön felsoroljuk a production CMS további rétegeit.

A wiki minimális, fordítási fixture-ként is ellenőrzött változata az `examples/wiki/main.rw`; a CMS többfájlos mintája az `examples/news-site/`. Így az alábbi két alkalmazásminta nem csak könyvbeli pszeudokód.

12.1. Egyszerű wiki

Egy wiki minimális domainje oldalból áll: stabil azonosító, slug, cím és Markdown törzs. Az alábbi minta a nyelvi elemek összekapcsolását mutatja.

Modell

```
model WikiPage {
  id: Int
  slug: Slug
  title: String
  body: String
}
```

Lekérdezés

```
query fn wikiBySlug(db: Db, slug: Slug)
-> Result<WikiPage, DbError> sql {
  SELECT id, slug, title, body
  FROM wiki_pages
  WHERE slug = :slug
}
```

Publikus oldal

```
page fn wikiShow(ctx: PageContext, db: Db, slug: Slug)
  -> Result<Html, PageError> {
    let page = wikiBySlug(db, slug)?;
    return Ok(html {
      @layout(Main, page.title) {
        <article>
          <h1>{{ page.title }}</h1>
          @markdown(page.body)
        </article>
      }
    });
  }

route wikiShow GET "/wiki/:slug<Slug>" => wikiShow;
```

Szerkesztő form

```
form WikiForm {
  title<String>
  body<String>
  validate title length 2 200 body length 1 300000
}
```

A módosító route legyen autentikált, például `auth role Editor`. A write query `Transaction` capabilityt kapjon. A cím alapján generált slugot ne írd át vakon minden szerkesztéskor, ha a régi URL-ek stabilitása fontos.

Canonical slug és alias

Érettebb wikiben külön slug-alias/history tábla tartja a régi címeket. A runtime canonical route-ból tud 301-es redirectet készíteni, így nem kell tetszőleges redirect URL-t építeni.

Mit adj hozzá egy valódi wikihez?

- optimistic locking a konkurens szerkesztésekhez;
- üzleti audit trail a módosításokhoz;
- objektumszintű authorization, ha nem minden oldal azonos jogosultságú;
- backup/restore policy az adatbázisra és az esetleges AppFs adatra.

12.2. CMS és híroldal

A repository `examples/news-site/` példája már egy kis CMS magját mutatja: model, typed SQL, layout, Markdown, szerkesztői action és public cache.

Projektstruktúra

```
main.rw
models.rw
queries.rw
components.rw
pages.rw
```

```
actions.rw
```

Cikkmodell

```
model Article {
  id: Int
  slug: Slug
  title: String
  body: String
}
```

A szerkesztői input külön named form-sémát kap:

```
form ArticleForm {
  title<String>
  body<String>
  validate title length 2 200 body length 1 300000
}
```

Publikus cikkoldal

```
page fn article(ctx: PageContext, db: Db, slug: Slug)
-> Result<Html, PageError> {
  let article = articleBySlug(db, slug)?;
  return Ok(html {
    @layout(Main, article.title) {
      <article>
        <h1>{{ article.title }}</h1>
        @markdown(article.body)
      </article>
    }
  });
}

route article GET "/cikk/:slug<Slug>"
  cache public ttl 60
  => article;
```

Public cache csak olyan oldalra való, amely nem session- vagy userfüggő.

Szerkesztői létrehozás

```
action fn create(ctx: ActionContext, db: Db,
  title: String, body: String)
-> Result<Redirect, PageError> {
  let articleSlug = slug(title);
  transaction db {
    insertArticle(tx, articleSlug, title, body)?;
  }
  return Ok(redirect("/admin/articles"));
}

route articleCreate POST "/admin/articles"
```

```
form ArticleForm
auth role Editor
invalidate cache article
=> create;
```

A fenti invalidáció route-szintű generationváltás: nem csak az új slug egyetlen cache-kulcsát törli, hanem az `article` route korábbi generationjét teszi elérhetetlenné. Egyszerű mintában ez helyes és kiszámítható, de nagy forgalmú CMS-nél érdemes tudatosan mérlegelni, mely publikus route-okat kell valóban invalidálni egy adott módosítás után.

CMS-ből production rendszer

A következő lépések adják a gyakorlati production CMS gerincét:

- publikálási és domain-invariánsok külön validációja;
- draft/published állapot enumként;
- canonical slug history;
- optimistic locking szerkesztéskor;
- business audit trail;
- media library és AppFs a képekhez;
- role-based szerkesztő/publisher policy;
- rate limit és resource profile a drágább route-okra;
- backup, restore drill és kontrollált deployment.

A CMS-ben az eredeti Markdown legyen a source of truth; ne tárolj alkalmazásból származó raw renderelt HTML-t.

IV. rész

Haladó integráció és minőség

13. fejezet

Külső szolgáltatások és egress policy

Egy valódi webalkalmazás ritkán marad teljesen önmagában. Fizetési szolgáltatót, címtárat, térképes API-t, belső vállalati szolgáltatást vagy más HTTPS végpontot kell elérnie. Ez azonban más biztonsági határ, mint egy bejövő HTTP route: itt már maga az alkalmazás kezdeményez hálózati kapcsolatot.

13.1. Először a V1-határ

A V1-ben **nincs általános RWLang HttpClient vagy Secrets nyelvi API**. Az alkalmazáskód ezért nem kap tetszőleges hálózati hozzáférést. Az alábbi szándékosan nem RWLang szintaxis, csak azt mutatja, milyen kerülőutat nem szabad feltételezni:

```
// NINCS ilyen általános V1 surface:  
// let r = http.get(user_supplied_url);
```

A repository `integrations` crate-je viszont már tartalmaz egy trusted Rust adattérreget:

- policy-vezérelt outbound HTTPS klienst;
- névvel azonosított egress targeteket;
- host-, CIDR-, port- és TLS-korlátozást;
- küldési/fogadási méretlimitet és timeoutokat;
- fájlalapú `SecretsStore`-t.

Ez fontos tervezési döntés: **ha egy capability nincs a nyelvi surface-ben, ne építs alkalmazáskódban kerülőutat**. A külső integrációt trusted adapterként vagy későbbi, külön megtervezett nyelvi/runtime feature-ként kell kezelni.

13.2. Miért veszélyes a tetszőleges URL?

Tegyük fel, hogy egy route a felhasználótól kapott URL-t közvetlenül meghívhatná. Ezzel az alkalmazás könnyen belső vagy lokális szolgáltatásokat érhetne el:

```
http://127.0.0.1:...  
http://169.254.169.254/...  
http://internal-db.example/...  
http://[::1]/...
```


Ez az SSRF (Server-Side Request Forgery) klasszikus formája. A helyes modell ezért nem az, hogy a request megmondja, *hová* kapcsolódjunk, hanem az, hogy a trusted konfiguráció előre definiál egy névvel ellátott célpontot.

13.3. Named target: a hálózati capability konfigurációja

Egy célpont például így nézhet ki:

```
[[target]]
name = "payments"
hosts = ["api.payments.example"]
cidrs = ["203.0.113.0/24", "2001:db8:1200::/48"]
ports = [443]
tls_required = true
max_dns_answers = 8
max_sent_bytes = 262144
max_received_bytes = 2097152
connect_timeout_ms = 5000
total_timeout_ms = 15000
```

A név — itt **payments** — az alkalmazás/integráció által ismert capability. A hostname és a hálózati tartomány viszont trusted operátori policy.

A repositoryban ugyanez a minta megtalálható az `examples/egress/policy.toml` fájlban.

13.4. A policy több rétegen ellenőriz

Az egress policy nem csak a hostname-et nézi.

Host

A kért hostnak szerepelnie kell a target explicit **hosts** listájában. IP-literal hostname helyett névvel azonosított, TLS-sel ellenőrizhető hostot használunk.

Port és TLS

A portnak szerepelnie kell az engedélyezett portok között. A jelenlegi adapter TLS-t követel meg; a tipikus cél a 443-as HTTPS.

DNS és CIDR

A DNS-feloldás minden A és AAAA válaszát ellenőrzi. A szabály fail-closed: ha akár egy feloldott cím nincs az engedélyezett CIDR-ekben, a teljes kapcsolat elutasításra kerül.

Ez azért fontos, mert csak a hostname allowlist önmagában nem véd DNS-rebinding vagy hibás DNS-konfiguráció ellen.

Kapcsolódás utáni peer-ellenőrzés

A TCP kapcsolat létrejötte után az adapter újra ellenőrzi a tényleges peer IP-címet. Annak egyeznie kell a kiválasztott címmel és ugyanúgy át kell mennie a CIDR policy-n.

TLS identity

A tanúsítvány ellenőrzése az eredeti hostname alapján történik. A CIDR tehát nem helyettesíti a TLS identity verificationt; a két ellenőrzés egymást erősíti.

13.5. IPv4 és IPv6 ugyanahhoz a policyhoz tartozik

Az outbound policy dual-stack. A targetben IPv4 és IPv6 CIDR is megadható:

```
cidrs = [
    "203.0.113.0/24",
    "2001:db8:1200::/48"
]
```

Az IPv4-mapped IPv6 címeket a runtime normalizálja. A `::ffff:203.0.113.9` például IPv4-ként kerül policy-ellenőrzésre, ezért ezzel nem lehet megkerülni az IPv4 szabályt.

A loopback, link-local, ULA, multicast és unspecified IPv6 tartományok sem kapnak implicit bizalmat. Ha ilyen címhez való hozzáférésre ténylegesen szükség van, azt is explicit CIDR-rel kell engedélyezni.

13.6. Méret- és időkorlátok

A külső szolgáltatás ugyanúgy lehet hibás vagy ellenséges, mint a kliens. Ezért a target policy nem csak címet korlátoz:

Beállítás	Mit korlátoz?
<code>max_dns_answers</code>	elfogadható DNS-válaszok száma
<code>max_sent_bytes</code>	elküldhető request maximális mérete
<code>max_received_bytes</code>	fogadható response maximális mérete
<code>connect_timeout_ms</code>	egy kapcsolódási kísérlet ideje
<code>total_timeout_ms</code>	a teljes outbound művelet ideje

A lényeg ugyanaz, mint az RWLang más részein: **a felhasználói alkalmazáskód ne döntse el saját erőforrás-limitjeit.**

13.7. Secret nem konfigurációs string

Egy külső API gyakran credentialt igényel. A trusted adapter ehhez `SecretsStore`-t használhat, amely egy elkülönített secret rootból olvas név alapján.

Például az operátori elrendezés lehet:

```
/run/secrets/rwlang/
payments-bearer
```

A secret neve nem tartalmazhat útvonalat, `..`-ot vagy tetszőleges szeparátort. Az adapter symlinket nem követ, fájlméretet korlátoz, és Unix rendszeren ellenőrzi, hogy a secret ne legyen world-accessible.

A `SecretString` debug reprezentációja redacted, és dropkor a tárolt byte-ok felülírásra kerülnek. Ettől még továbbra is alapelv, hogy credentialt nem írunk logba, error message-be

vagy audit payloadba.

13.8. GET és JSON POST a trusted adapterben

A jelenlegi Rust adapter két tipikus műveletet ad: HTTPS GET-et és JSON POST-ot opcionális bearer secrettel. A használat **trusted Rust kód**, nem **.rw** szintaxis.

```
let policy = EgressPolicy::from_toml_file(policy_path)?;
let client = OutboundHttpClient::new(policy);
let secrets = SecretsStore::new("/run/secrets/rwlang"?);
let token = secrets.get("payments-bearer")?;

let response = client
    .post_json(
        "payments",
        "api.payments.example",
        443,
        "/v1/charge",
        body,
        Some(&token),
    )
    .await?;
```

Itt a **payments** target és a konkrét host egyszerre kell, hogy megfeleljen a trusted policy-nak. A requestből származó tetszőleges host nem válhat új célponttá.

13.9. Hibamodell

A külső integrációnál fontos különbséget tenni a hibák között:

- **Policy:** a kért cél vagy DNS/peer nem engedélyezett;
- **Secret:** a credential nem olvasható vagy szabályt sért;
- **DNS:** névfeloldási hiba;
- **Connect/TLS:** hálózati vagy identity hiba;
- **Timeout:** az upstream túl lassú;
- **Protocol:** hibás HTTP-válasz;
- **ResponseTooLarge/SendTooLarge:** policy limit sérült.

Ezeket ne alakítsd automatikusan ugyanazzá az üzleti hibává. Egy policy denial például konfigurációs/security esemény, egy upstream 503 pedig rendelkezésre állási probléma.

13.10. Retry és idempotencia

A jelenlegi adapter nem ad általános automatikus retry-réteget. Ez helyes alapállás: egy POST újraküldése mellékhatást ismételhet meg.

Ha egy későbbi integráció retry-t kap, előbb tisztázni kell:

1. idempotens-e a művelet;
2. támogat-e az upstream idempotency key-t;
3. mely hibák retry-zhatók;
4. mekkora a teljes timeout-budget;
5. hogyan kerülhető el a retry storm.

Fizetési vagy webhook-szerű műveletnél az idempotency kulcs üzleti adat, nem pusztán hálózati tuning.

13.11. Szinkron kérés vagy háttér munka?

A V1 surface-ben nincs általános background-job nyelvi API. Ezért ne rejts hosszú külső workflow-t egy normál page/action request mögé csak azért, mert technikailag megoldható egy adapterrel.

Rövid, jól bounded integráció esetén a request-válasz ciklus elfogadható lehet. Hosszú vagy megbízható újrapróbálást igénylő folyamatnál külön, explicit job/outbox feature-re van szükség; ezt nem szabad ad hoc threaddel vagy subprocessszel pótolni.

13.12. Mit jelent ez az RWLang alkalmazásfejlesztőnek?

A napi szabály egyszerű:

1. Ne tervezz tetszőleges URL-t fogadó outbound feature-t.
2. Ne olvass secretet raw filesystem vagy environment kerülőúttal.
3. Az integráció célpontja legyen névvel azonosított capability.
4. Host/CIDR/port/TLS/timeout/méret trusted policy legyen.
5. A felhasználói input legfeljebb az üzleti payloadot válassza meg, ne a hálózati célpontot.
6. Ha a szükséges integráció nincs a V1 nyelvi surface-ben, trusted adaptert vagy új, review-zott language/runtime feature-t tervez.

13.13. Tesztelési minimum

Egress policy esetén a happy path kevés. Legalább ezeket érdemes bizonyítani:

- engedélyezett host + engedélyezett IPv4;
- engedélyezett host + engedélyezett IPv6;
- vegyes A/AAAA válaszban egy tiltott cím esetén fail-closed;
- tiltott port;
- túl sok DNS-válasz;
- peer IP újraellenőrzés;
- TLS hostname mismatch;
- IPv4-mapped IPv6 normalizáció;
- send/receive limit és timeout;
- secret path traversal/symlink/permission negatív esetek.

A repository unit tesztjei a policy-döntéseket bizonyíthatják, de a release evidence része legyen valódi dual-stack hálózati integration teszt is.

13.14. A fejezet lényege

A külső HTTPS nem egyszerű utility, hanem új trust boundary. Az RWLang V1 ezért szándékosan nem ad korlátlan hálózati capabilityt az alkalmazáskódnak. A jelenlegi trusted adapter megmutatja a kívánt modellt: **named target + explicit hálózati policy + bounded erőforrás + külön secret store + fail-closed ellenőrzés.**

14. fejezet

Tesztelés, hibamodellek és fejlesztői diagnosztika

Egy webalkalmazás attól még nem helyes, hogy lefordul. Az RWLang fordítója szándékosan sok hibát már a futtatás előtt elutasít, de nem tudja bizonyítani, hogy az üzleti folyamat helyes, az adatbázis valóban elérhető, a migráció kompatibilis, a jogosultsági szabály azt engedi, amit a termék kíván, vagy a production topológia megfelelően működik.

Ezért az RWLangban érdemes a tesztelést több egymásra épülő réteggént kezelni:

1. **source check:** a program nyelvileg és policy szempontból érvényes-e;
2. **negatív compile teszt:** a veszélyes vagy tiltott program valóban elutasításra kerül-e;
3. **alkalmazás- és adatbázisteszt:** az üzleti művelet a várt állapotot hozza-e létre;
4. **integration teszt:** a valódi DB, Redis, LDAP, TLS, AppFs vagy más dependency együtt helyesen működik-e;
5. **system/smoke teszt:** a telepített rendszer HTTP-n, a tényleges konfigurációval is működik-e.

A cél nem az, hogy minden teszt ugyanazt bizonyítsa, hanem az, hogy minden hibafajta a lehető legolcsóbb rétegben bukjon el.

14.1. Az első kapu: `rwlang-cli check`

Fejlesztés közben a leggyorsabb ellenőrzés:

```
rwlang-cli check main.rw
```

Siker esetén a CLI összefoglalja a felismert programot, például a modellek, query-k, page-ek, actionök és route-ok számát. A check nem indít HTTP servert és nem módosít adatbázist: a source contractot ellenőrzi.

A fordító többek között itt tudja megfogni:

- a típusegyeztési hibákat;
- a query eredményalak és a `model` eltérését;
- a hibás route vagy form deklarációt;
- a tiltott HTML/script konstrukciókat;
- a veszélyes SQL összeállítást vagy bind hibát;
- az auth/cache/policy egymással össze nem férő kombinációit;

- a capability-határok bizonyos megsértéseit.

Ez az egyik legfontosabb szemléleti különbség egy általános scripting megoldáshoz képest: a hibák jelentős részét nem HTTP kérés közben akarjuk felfedezni.

14.2. A negatív teszt ugyanolyan értékes, mint a pozitív

Biztonsági invariánsnál nem elég azt tesztelni, hogy a helyes program lefordul. Azt is bizonyítani kell, hogy a tiltott program *nem* fordul le.

A repository release-ellenőrzése ezért ilyen fixture-öket is futtat:

```
# Ennek sikeresen kell fordulnia:
rclang-cli check examples/auth/app.rw

# Ennek viszont hibával kell visszatérnie:
rclang-cli check \
  examples/security/sql-injection-rejected.rw
```

Tipikus negatív teszt:

- SQL injectionre alkalmas dinamikus SQL;
- raw/script jellegű XSS-kísérlet;
- optional érték ellenőrzés nélküli használata;
- hibás form/schema validáció;
- userfüggő route public cache-sel;
- public route objektumszintű authorization guarddal;
- tiltott upload path vagy JSON/form body-mode keverés.

Egy biztonsági szabály akkor erős, ha a projektben van rá **pozitív és negatív bizonyíték** is.

14.3. A hibák négy külön világa

Hibakereséskor először azt dönts el, melyik rétegben történt a hiba. Ugyanaz a felhasználói tünet teljesen más javítást kívánhat.

Compile-time hiba A source contract hiba. A `rclang-cli check` vagy `build` áll meg. Ezt kódban kell javítani.

Startup/config hiba A program érvényes, de a szerver konfigurációja, secretje, TLS kulcsa vagy egy hard policy hiba. Itt a `-check-config`, a server log és az effective config a fő eszköz.

Request/policy hiba A szerver fut, de egy kérés input-, auth-, authorization-, rate-limit- vagy content-negotiation szabály miatt elutasítódik. Itt az HTTP státusz, request ID, access és audit log a bizonyíték.

Dependency/runtime hiba DB, Redis, LDAP, filesystem, hálózat vagy resource limit okozza a hibát. Itt readiness, metrics, server log és valódi integration teszt kell.

A rossz diagnosztikai reflex az, hogy minden 500/503 körüli tünetre a source kódot kezdjük átírni. Először a réteget azonosítsd.

14.4. Konfigurációt külön tesztelj

A source check nem helyettesíti a server konfiguráció ellenőrzését:

```
rwlang-server \  
  --config /usr/local/etc/rwlang/server.toml \  
  --check-config
```

Ha deploy előtt a ténylegesen összeálló értékeket is látni akarsz:

```
rwlang-server \  
  --config /usr/local/etc/rwlang/server.toml \  
  --print-effective-config
```

A `-check-config` nem dependency integration teszt. Egy szintaktikailag helyes DB URL mellett a DB még lehet elérhetetlen; ezért van külön readiness és deployment smoke teszt.

14.5. Migráció: status, verify, apply, verify

Adatbázis-változásnál a tesztelési folyamat része maga a migrációs állapot is:

```
rwlang-cli migrate status \  
  --dir migrations \  
  --db-url-file /run/secrets/rwlang/migration-db-url  
  
rwlang-cli migrate verify \  
  --dir migrations \  
  --db-url-file /run/secrets/rwlang/migration-db-url
```

Deploymentben a jó sorrend:

```
source/config check  
-> migration verify  
-> migration apply  
-> migration verify  
-> application rollout  
-> readiness  
-> smoke test
```

A már alkalmazott migration fájlt ne módosítsd azért, hogy a teszt újra zöld legyen. Új változást új migrationben vezess be.

14.6. Üzleti teszt: állapotot bizonyíts, ne implementációt

Egy CRUD vagy CMS action tesztjének ne az legyen a lényege, hogy „meghívódott-e ez a belső függvény”. A kívülről megfigyelhető állapotot ellenőrizd.

Például cikkipublikálásnál:

1. hozz létre draft cikket;
2. autentikálj Publisher szerepkörrel;
3. küldd el a publish actiont;
4. ellenőrizd a redirectet;
5. olvasd vissza a cikket és ellenőrizd a `Published` állapotot;
6. ellenőrizd a business audit rekordot;
7. ellenőrizd, hogy a publikus oldal már az új állapotot adja;

8. cache-elt route esetén ellenőrizd az invalidációt is.

Ez a teszt akkor is értékes marad, ha később belül átszervezed a query-ket vagy a modulstruktúrát.

14.7. Adatbázisteszt: SQLite nem bizonyít mindent

A gyors fejlesztői tesztben a külön ideiglenes SQLite adatbázis kényelmes. De ha production PostgreSQL vagy MariaDB, legalább a release/integration körben a valódi backendet is teszteld.

Különösen érzékeny területek:

- tranzakciós viselkedés;
- constraint és unique conflict;
- konkurens módosítás és optimistic locking;
- index/lekérdezési terv;
- migration DDL;
- timeout és connection availability.

A backend-független typed query contract sok hibát kizár, de a backend működését nem teszi azonossá.

14.8. Auth és authorization tesztmátrix

Egy védett actionnál legalább a következő eseteket érdemes explicit tesztelni:

Eset	Elvárt eredmény
Nincs session	login/auth elutasítás
Érvényes user, nincs megfelelő role	authorization deny
Megfelelő role	sikeres művelet
MFA-köteles route, nincs MFA	MFA elutasítás
Lejárt vagy visszavont session	új autentikáció szükséges
Hibás CSRF token state change-nél	kérés elutasítva
Objektum nem a useré	object authorization deny
Admin/engedélyezett actor	sikeres művelet

LDAP/LDAPS vagy Redis használatakor legyen valódi dependency integration teszt is. Egy mock nem bizonyít TLS hostname validationt, hálózati timeoutot vagy Redis fail-closed viselkedést.

14.9. Fájl, média, API és cache tesztek

Ezeknél a happy path különösen kevés.

Upload és média

Teszteld legalább:

- a megengedett kis fájlt;
- byte-limit túllépést;
- kép pixel-limit túllépést;
- hibás/támadó fájlnevet;

- path traversal/symlink escape elutasítást;
- félbeszakadt upload cleanupot.

JSON/CORS

Teszteld a helyes JSON mellett a hibás content type-ot, duplikált kulcsot, ismeretlen mezőt, **Accept** negotiationt, preflightot és credentialed state change esetén a CSRF/CORS együttműködését.

Cache és rate limit

Teszteld, hogy userfüggő oldal nem válik public cache-é, módosítás után a cache invalidálódik, több instance esetén a shared state tényleg közös, és Redis kiesésnél a security szempontból választott fail-closed viselkedés érvényesül.

14.10. HTTP hibakód nem egyenlő programhibával

A kliensnek adott státusz a szerződés része. Tipikus esetek:

Helyzet	HTTP
Hibás request-contract vagy malformed input	400
Nincs érvényes autentikáció JSON/API route-on	401
Az actor/policy nem engedélyezi a műveletet	403
Nem található erőforrás	404
Rossz HTTP method	405
Nem elfogadható representation	406
Konkurens módosítás / Changed nulla sor	409
Túl nagy request/upload	413
Rossz vagy hiányzó body content type	415
Host/trust boundary mismatch	421
Named form mezőhiba, újrendereléssel	422
HTTPS szükséges	426
Rate limit	429
Túl nagy request header	431
Belső, nem klasszifikált serverhiba	500
Dependency vagy request resource átmenetileg nem elérhető	503

A kliensnek soha ne küldj SQL-t, secretet, fizikai filesystem pathot, session tokenet vagy stack trace-et. A részletes diagnosztika a szervertől logban marad, request ID-val korrelálva.

A státusz és a válasz formátuma külön szerződés

Ugyanaz az üzleti helyzet eltérő reprezentációt kaphat HTML és JSON felületen. Ez nem inkonzisztencia: a státusz a hibakategóriát, a reprezentáció pedig a klienssel kötött felületi szerződést fejezi ki.

Helyzet	HTML route	JSON route
Nincs session	303 See Other a beépített login oldalra	401 + JSON error envelope

Helyzet	HTML route	JSON route
Authorization tiltás	403	403 + JSON error envelope
Optimistic-lock konfliktus	409 konfliktusoldal, no-store	409 + JSON error=conflict
Named form validation	422, a form újrarendelve mezőhibákkal	nem JSON body-mód; a body-contractok kölcsönösen kizártak
Nem található record	404	404 + JSON error envelope

A sikeres mutáló HTML action tipikusan 303 `See Other` választ használ a Post/Redirect/Get mintához. A 303 redirect tehát nem hiba, hanem a böngészős vezérlési folyamat része.

14.11. Request ID-val hibakeresni

Ha egy HTTP kérés hibázik, a legjobb első diagnosztikai adat a szerver által generált request ID. A kliens válaszából vagy az access logból indulva:

```
grep 'rw-...' /var/log/rwlang/access.log
grep 'rw-...' /var/log/rwlang/server.log
grep 'rw-...' /var/log/rwlang/audit.log
```

A három log más kérdésre válaszol:

- access: mi érkezett és milyen státusszal végződött;
- server: milyen technikai/runtime esemény történt;
- audit: security vagy felhasználói policy döntés történt-e.

Ezt a diagnosztikai láncot érdemes automatizált teszt failure outputban is megőrizni.

14.12. Deployment smoke teszt

A CI zöld állapota nem bizonyítja, hogy a frissen telepített service ténylegesen elindult. Rollout után legalább:

```
curl --fail --silent \
  https://example.test/health/live

curl --fail --silent \
  https://example.test/health/ready

curl --fail --silent \
  https://example.test/
```

A smoke teszt legyen kicsi és nem destruktív. Ha state-changing végpontot is ellenőrzöl, használj erre elkülönített tesztadatot és idempotens cleanupot; ne a production tartalom kísérletezz.

14.13. Mit jelent a repository `verify.sh`?

Az RWLang saját forrásfájában a teljes release-evidence belépési pontja:

```
./verify.sh
```

Ez jóval több, mint egy alkalmazásfejlesztői source check. A workspace Rust tesztjeit, pozitív és negatív RWLang fixture-öket, migrációs/auth/storage/runtime ellenőrzéseket és release artifact invariánsokat is futtat.

Ha magát az RWLang implementációját módosítod, ez a release minimum. Ha csak egy saját `.rw` alkalmazást fejlesztesz, a napi ciklus kisebb lehet, de a release folyamatodnak akkor is tartalmaznia kell source checket, migration/config ellenőrzést, a releváns integration teszteket és deploy utáni smoke tesztet.

14.14. Ajánlott fejlesztői ciklus

Egy hétköznapi feature fejlesztésére jó alap:

```
1. domain/model valtozas megtervezese
2. migration megirasa, ha kell
3. rwlang-cli check main.rw
4. gyors alkalmazasteszt
5. negativ/security esetek
6. migration verify
7. relevans integration teszt
8. rwlang-server --config server.toml --check-config
9. deploy stagingre
10. readiness + HTTP smoke
11. production rollout
12. readiness + smoke + metrics/log ellenorzes
```

A sorrend lényege: a gyors és determinisztikus hibák bukjanak el korán, a drága környezeti tesztek csak azután fussanak.

14.15. Hibakeresési döntési sorrend

Amikor „nem működik”, haladj következetesen:

1. Lefordul? Futtasd a `rwlang-cli check` parancsot.
2. Érvényes a config? Futtasd a `-check-config` ellenőrzést.
3. Elindult a service? Nézd a service és server logot.
4. Él a process? Nézd a liveness endpointot.
5. Elérhetők a dependency-k? Nézd a readiness endpointot.
6. Eljut a kérés a route-ig? Nézd az access logot és request ID-t.
7. Policy dobja el? Nézd az audit logot és a trust-boundary státuszt (például forwarding 400, 401/403/421/426/429).
8. Runtime/dependency hibázik? Nézd a server logot és metrics-et.
9. Üzleti eredmény rossz? Reprodukálj izolált tesztadattal és ellenőrizd a tartós állapotot.

Ez a sorrend sokkal gyorsabb, mint egyszerre kódot, proxy-konfigurációt és adatbázist módosítani.

14.16. Minimum release checklist alkalmazáshoz

- `rwlang-cli check` zöld;
- migration history verify zöld;
- server config check zöld;
- a kritikus business flow-k pozitív és negatív tesztje zöld;
- auth/authorization/security regressziós esetek zöldek;
- productionnal azonos DB/Redis/TLS irányú integration teszt zöld;
- backup/restore readiness ismert;
- staging readiness és smoke zöld;
- rollout után readiness, smoke, log és metrics ellenőrzött.

A tesztelés célja nem a százszázalékos sorlefedettség, hanem az, hogy a rendszer fontos szerződéseire legyen ismételtető bizonyíték: **mit engedünk, mit tiltunk, milyen állapotot hozunk létre, és hogyan viselkedünk dependency-hiba esetén.**

V. rész

Production és üzemeltetés

15. fejezet

Cache, Redis, rate limiting és többpéldányos működés

Egyetlen RWLang szerverpéldánynál sok állapot természetesen lokálisnak látszik. Amint viszont két vagy több példány szolgálja ki ugyanazt az alkalmazást, azonnal felmerül három kérdés: hol él a session, ki számolja a rate limitet, és melyik példány cache-e tekinthető érvényesnek? A production válasz ezekre nem az, hogy az alkalmazáskód közvetlenül Redis kulcsokat kezel. Az RWLang a megosztott infrastruktúrát runtime-capabilityként használja, az alkalmazás pedig deklaratív policy-t kér.

Ez a fejezet négy, egymással összefüggő témát választ szét:

- **session és MFA replay state:** felhasználói biztonsági állapot;
- **route rate limit:** forgalomszabályozási állapot;
- **public page cache:** újrászámítható teljesítményállapot;
- **multi-instance deployment:** annak eldöntése, mely állapotnak kell shared backendben élnie.

A közös production backend jelenleg Redis, de az alkalmazás nem kap általános Redis API-t. Ez fontos biztonsági és hordozhatósági határ.

15.1. Miért nem általános Redis kliens?

A V1 nyelvben nincs olyan művelet, amellyel a program tetszőleges Redis kulcsot írhatna vagy olvashatna. A Redis compiler/runtime-owned infrastruktúra:

Felhasználás	Ki birtokolja a kulcsot és a szabályokat?
session	runtime/auth réteg
TOTP replay	runtime/auth réteg
login rate limit	runtime/auth réteg
route rate limit	runtime + trusted policy config
public page cache	compiler/runtime
üzleti adat	nem Redisben; application DB-ben/AppFs-ben

Ezért egy wiki cikkét, CMS workflow állapotát vagy számlázási rekordját ne Redisbe tedd. Az ilyen adat üzleti source of truth, ezért a typed SQL-lel elért application DB-be tartozik. Redis elvesztése után a rendszernek újraépíthető állapotból kell indulnia: a sessionök megszűnhetnek, a cache újraépülhet, a rate-limit ablakok újraindulhatnak.

15.2. Production Redis konfiguráció

A Redis URL secret, ezért fájlból érkezzon:

```
[redis]
url_file = "/run/secrets/rwlang/redis-url"
allow_insecure = false
```

A secret file például TLS-es URL-t tartalmazhat:

```
rediss://rwlang:SECRET@redis.internal.example/0
```

Productionben az `allow_insecure = false` legyen az alap. A Redis hálózati elérhetősége önmagában nem jogosultsági modell: korlátozd hálózati ACL-lel/firewallal is, és a credentialt csak az RWLang service user olvashassa.

A runtime külön namespace-eket használ a különböző célokra. A public cache például `rw-cache` namespace-ben él; az alkalmazás nem választ tetszőleges key prefixet.

15.3. Shared session több szerverpéldánnyal

Fejlesztéshez használható in-memory session store, de több application instance esetén ez hibás production modell lenne. Képzeld el két szervert:

```
          +--> RWLang A ---+
Browser -> proxy |          |--> Redis
          +--> RWLang B ---+
```

Ha a session csak A memóriájában élne, akkor egy következő request B-re kerülve a felhasználó autentikációja eltűnne. Sticky sessionnel ezt részben el lehetne fedni, de az failover, rollout és load balancing szempontból tökéleny. A helyes megoldás a shared session backend.

Redis production környezetben többek között az alábbi auth state-et osztja meg:

- sessionök;
- TOTP replay védelem;
- login rate-limit állapot.

Így nincs szükség arra, hogy ugyanaz a böngésző mindig ugyanarra a példányra kerüljön.

A local auth fontos kivétele

A session shared lehet, de a V1 local-auth identity store SQLite és single-node identity megoldás. Ezért több application instance esetén a jelenlegi shared identity backend az LDAP/LDAPS. Ne téveszd össze a két réteget:

Redis session megmondja, hogy egy már autentikált request milyen trusted principalhoz és auth state-hez tartozik;

identity backend a login során ellenőrzi a credentialt és az account/role állapotot.

Több instance-nál shared Redis session mellett is közös identity forrás kell; V1-ben erre az LDAP a production út.

15.4. Public page cache

A public cache célja, hogy nyilvános, felhasználótól független GET oldalnál ne fusson minden requestre ugyanaz a query és HTML render.

Deklaráció:

```
route frontPage GET "/"
  cache public ttl 60
=> frontPage;
```

A fejlesztő csak a cache-elési szándékot és a route TTL-jét adja meg. A kulcsot a runtime készíti canonical, hash-elt formában többek között a route nevéből, route generationból, pathból, rendezett typed query inputból és reprezentációból.

Az alkalmazás tehát nem írhat ilyet:

```
cache.set("homepage:" + userInput, rendered)
```

Ilyen általános cache API nincs. Ez megakadályozza a kulcsnév-konvenciók szétcsúszását, a véletlen PII-keyeket és több invalidációs stratégia keverését.

15.5. Mit szabad public cache-be tenni?

A `cache public` név szó szerinti. Csak olyan GET route legyen cache-elt, amelynek válasza nem függ a felhasználó sessionjétől vagy request-specifikus security state-től.

A compiler tiltja többek között az olyan függéseket, mint:

```
csrfToken
authPrincipal
authMfaVerified
```

Ez azért kritikus, mert sikeres cache-elt válasz HTTP szinten is public cache-elhető:

```
Cache-Control: public, max-age=<ttl>
```

Vagyis ugyanaz a response reverse proxy vagy CDN cache-be is kerülhet. Account oldal, admin felület, személyre szabott navigáció vagy flash üzenetet fogyasztó oldal ezért nem public cache.

15.6. Operator által szabott cache-határok

A route nem állíthat tetszőlegesen nagy TTL-t. A trusted server config kemény plafont ad:

```
[cache]
max_ttl_secs = 3600
max_entries = 10000
max_bytes = 67108864
allow_memory = false
```

A `max_ttl_secs` a route által kérhető TTL maximuma. A memória backend entry- és byte-limitjei bounded development fallbackot adnak, nem production skálázási stratégiát.

Ha az alkalmazás public cache-t használ, de Redis nincs konfigurálva, a server production default mellett nem indul el. Fejlesztéshez explicit engedélyezhető a memory cache, de ezt ne vidd át észrevétlenül productionbe.

15.7. Cache invalidation: generation, nem kulcskeresés

State-changing route megjelölheti, mely cache-elt route-ok válnak érvénytelenné:

```
route publish POST "/publish"  
  invalidate cache frontPage article  
  => publish;
```

Az invalidation nem Redis SCAN és nem kulcslista törlése. A runtime route-generationt növel. Az ezután érkező request már új generationból készíti a cache key-t, ezért a korábbi entry automatikusan elérhetetlenné válik az új forgalom számára.

Ennek két fontos következménye van:

- az invalidation $O(1)$ jellegű, nem függ a korábban létrejött cache key-ek számától;
- shared Redis esetén minden instance ugyanazt az új generationt látja, ezért az invalidáció cluster-safe.

A régi entry fizikailag egy ideig még Redisben maradhat, de logikailag már nem része az aktuális cache namespace-nek és TTL alapján kifut.

15.8. Stampede védelem és annak határa

Cache miss után több azonos request egyszerre érkezhetsz. Az RWLang ugyanazon instance-en ugyanarra a cache keyre single-flight jelleggel csak egy requestet enged újraépíteni; a többiek várnak, majd újra cache-t olvasnak.

Ez fontos, de a határát is ismerni kell: V1-ben nincs distributed single-flight lease. Két külön RWLang instance ugyanannak a lejárt kulcsnak az újraépítését egyszerre is elkezdheti. Ezért a cache-elhető page queryje önmagában is legyen ésszerű költségű és jól indexelt. A cache nem mentség drága, kontrollálatlan adatbázis-műveletre.

15.9. Redis hiba cache-elt request közben

A public cache backend hibája nem változhat csendben más konzisztenciamodellre. Cache backend elérhetetlenségnél az RWLang `cache_unavailable` hibával szolgáltatáshibát adhat a cache-elt útvonalon helyett, hogy implicit, korlátlan fallbackként minden requestet az adatbázisra engedne.

Ez productionben hasznos fail-closed viselkedés: Redis kiesés ne okozzon észrevétlen DB thundering herdöt. A readiness probe DB/Redis függőségi hibánál 503-at adhat, ezért a load balancer ki tudja venni a példányt a forgalomból.

15.10. Route rate limiting

A route nem numerikus limitet ír a forráskódba, hanem trusted policy nevet kér:

```
route api GET "/api/status"  
  rate publicApi
```

```
=> api;
```

A konkrét számok operátori configban élnek:

```
[policy.publicApi]
limit = 120
window_secs = 60
scope = "ip_route"
```

Ez tudatos elválasztás. Az alkalmazás megmondhatja, hogy az endpointot védeni kell a `publicApi` policy-val, de nem tudja önkényesen felülrni a production erőforrás- és abuse policy-t.

15.11. Rate-limit scope-ok

A V1 scope-ok:

Scope	Bucket alanya
<code>ip</code>	effective client IP
<code>route</code>	minden kliens közös route bucketje
<code>ip_route</code>	effective client IP + route
<code>user</code>	autentikált canonical principal
<code>user_route</code>	canonical principal + route

A `user` és `user_route` public route-on startup error, mert nincs trusted autentikált principal, amelyből a bucket képződhetne.

IP-scope esetén az RWLang nem vakon bíz az `X-Forwarded-For` fejlécben. Előbb a trusted proxy policy alapján állapítja meg az effective client IP-t. Emiatt Apache/Nginx mögött a proxy CIDR helyes konfigurálása rate-limit security kérdés is.

15.12. Fixed-window modell több instance-nál

A route limiter fixed-window bucketet használ. Redisben minden instance ugyanazt a bucketet növeli atomikusan, ezért a limit nem példányonként értendő.

Például két szerver mellett:

```
limit = 120 / 60 sec

nem: 120 request A + 120 request B
hanem: 120 request a shared bucketben
```

A subject stabil hashként kerül a Redis kulcsba, így a raw username vagy IP nem jelenik meg közvetlenül keyként.

Limit túllépésekor a válasz:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 60
```

Redis limiter hiba esetén a policy fail-closed:

```
503 rate_limiter_unavailable
```

Ez biztonságosabb annál, mintha Redis hiba idején automatikusan korlátlanra nyílna az endpoint.

15.13. Memory backend csak fejlesztési kivétel

A trusted config production baseline-ja:

```
[rate_limit]
policies_file = "/usr/local/etc/rwlang/rate-limits.toml"
allow_memory = false

[cache]
allow_memory = false
```

A memory rate limiter és memory cache explicit development escape hatch. Több process vagy több host esetén minden példány külön memóriát látna, tehát sem a limit, sem a cache-invalidation globális szemantikája nem lenne ugyanaz, mint Redis mellett.

Ha lokálisan memory backenddel tesztelsz, deployment előtt mindig fuss át a production configon:

```
rwlang-server --config /usr/local/etc/rwlang/server.toml --check-config
```

15.14. Egy tipikus többpéldányos topológia

```

      +---> RWLang A  --+---> application DB
Internet -> proxy --|      |
      +---> RWLang B  --+---> Redis
              |
              +-----> shared AppFs (*)
```

A csillag fontos: ha az alkalmazás feltöltött fájlokat/AppFs üzleti állapotot használ, akkor több instance esetén annak tárolását is olyan deploymenttel kell megoldani, amely minden példány számára konzisztensen elérhető. A V1 AppFs API confinementet és biztonságos fájlműveleteket ad; attól még az infrastruktúra feladata eldönteni, hogyan lesz a data root több példány számára helyesen megosztott.

A komponensek szerepe:

- **reverse proxy/load balancer:** TLS, routing és instance-választás;
- **RWLang instance:** statelesshez közelítő alkalmazásprocessz, lokális bounded runtime state-tel;
- **application DB:** tartós strukturált üzleti source of truth;
- **AppFs:** tartós fájl/mediális üzleti állapot, ha az alkalmazás használja;
- **Redis:** újraépíthető shared session/cache/rate-limit/replay infrastruktúra.

15.15. Rolling restart és cache generation

Új release rolloutjánál nem kell a régi cache kulcsokat kézzel törölni. A cache key része a route generation, ezért a release/compiler által meghatározott új generation új namespace-t eredményez. Explicit üzleti invalidáció továbbra is a state-changing route deklarációjából jön.

Rolling restartnál a proxy csak ready instance-nak küldjön forgalmat. A SIGTERM graceful drain útvonalat használd. Az orchestrator leállítási türelmi ideje legyen nagyobb a szerver konfigurált shutdown grace értékénél.

Sticky session nem követelmény, ha a shared session backend helyesen konfigurált.

15.16. Mit monitorozz?

A cache és rate limit önmagában nem láthatatlan optimalizáció. Minimum production monitoring:

- `rw_cache_hits_total` és `rw_cache_misses_total`;
- 429 válaszok és `rw_rate_limit_denials_total`;
- `rate_limiter_unavailable` események;
- readiness 503 események Redis/DB dependency hibánál;
- Redis latency, connection és memória állapot az infrastruktúra monitoringjában;
- az alkalmazás DB latency-je cache miss időszakokban.

Metric labelként továbbra se használj raw IP-t, username-et, session ID-t, tenantot, request ID-t vagy teljes URL-t. A későbbi observability fejezet bounded-cardinality szabálya itt is érvényes: metric label ne legyen felhasználónként vagy kérésenként korlátlanul változó.

15.17. Redis recovery szerepe

A Redis ebben az architektúrában runtime state: session, cache, rate-limit és replay-védelmi állapot élhet benne, de nem ez az üzleti source of truth. Emiatt recovery során a fő kérdés nem a cache visszatöltése, hanem az, hogy a Redis elvesztése után a rendszer biztonságosan és kiszámíthatóan építse újra ezt az állapotot.

A teljes backup/restore sorrendet és azt, hogy mely állapotot kell ténylegesen menteni, a `refch:recovery`. fejezet tárgyalja.

15.18. Gyakorlati példa: cache-elt híroldal publikálással

A publikus címlap userfüggetlen, ezért cache-elhető:

```
page fn frontPage(ctx: PageContext, db: Db)
-> Result<Html, PageError> {
  let articles = latestArticles(db)?;
  return Ok(html {
    <main>
      @for article in articles {
        <a href="/cikk/{{ article.slug }}">
          {{ article.title }}
        </a>
      }
    </main>
  });
}
```

```
}  
  
route frontPage GET "/"  
  cache public ttl 60  
  => frontPage;
```

A publikálás autentikált state change, ezért nem cache-elhető, viszont invalidálja a címlapot:

```
route publish POST "/admin/publish/:id<Int>"  
  auth role Publisher  
  invalidate cache frontPage  
  => publish;
```

A mentális modell így egyszerű marad:

1. az adatbázis a cikk igazságforrása;
2. a publikus HTML cache csak gyors másolat;
3. a publish tranzakció után az új generation miatt a következő GET friss oldalt épít;
4. ezt bármelyik RWLang instance megteheti;
5. az elkészült response shared Redis cache-be kerül.

15.19. Ellenőrzőlista

Többpéldányos production deployment előtt ellenőrizd:

- Redis URL secret file-ből jön, TLS/security policy megfelel;
- shared session miatt nincs szükség sticky sessionre;
- local auth SQLite-ot nem kezeled shared multi-instance identity store-ként;
- public cache csak valóban userfüggetlen GET route-on van;
- minden releváns state-changing route explicit cache invalidációt deklarál;
- a cache TTL nem lépi túl az operator maximumot;
- productionben `allow_memory = false` cache-hez és rate limiterhez;
- rate-limit scope trusted identityből/effective client IP-ből képződik;
- Redis outage esetére 503/readiness viselkedést tesztelted;
- cache hit/miss és rate-limit denial metrikák monitorozva vannak;
- Redis állapotot nem tekinted üzleti backup kötelező részének.

16. fejezet

Security hardening: a trust boundaryk összekapcsolása

Az előző fejezetek külön-külön megtanították az autentikációt, az objektumszintű authorizationt, a CSRF- és CORS-védelmet, a typed SQL-t, az AppFs-t, a megosztott runtime state alapjait és az outbound egress policyt. A production konfiguráció részletes kulcsreferenciája később jön; itt azt foglalnuk össze, milyen biztonsági határokat kell annak kikényszerítenie. Production átadáskor azonban ezek nem különálló feature-ként számítanak, hanem egyetlen biztonsági modellként.

A security hardening célja ezért nem új szintaxis bevezetése, hanem annak ellenőrzése, hogy minden külső adat és minden érzékeny művelet egyértelmű trust boundaryn haladjon át, és sehol ne legyen olyan kerülőút, amely megkerüli a fordítót, a runtime vagy a szerver policyját.

16.1. Az öt legfontosabb trust boundary

Egy tipikus RWLang alkalmazásban legalább az alábbi határokat érdemes külön kezelni:

Határ	Nem megbízható oldal	Authority / ellenőrzés
HTTP bemenet	path, query, header, form, JSON	typed route/input, parser limitek, CSRF/CORS
Identity	cookie, login mezők, kliens role	trusted auth backend, session, MFA
Adat	kliens érték, rekordazonosító	typed query, bind, transaction, authorization
Fájlrendszer	filename, MIME, upload byte-ok	AppFs confinement, server-owned destination, limitek
Külső hálózat	URL, DNS, peer, response	named egress target, CIDR/port/TLS/timeout policy

A legfontosabb szemléleti szabály: **az adat attól még nem lesz megbízható, hogy egyszer már átment egy korábbi rétegen**. Egy autentikált felhasználó formadata továbbra is user input; egy valid rekordazonosító nem jelent automatikus authorizationt; egy DNS-ben feloldott host nem jelenti azt, hogy a tényleges peer IP is engedélyezett.

16.2. HTTP edge: Host, proxy és parser

Közvetlen HTTPS esetén maga az RWLang server a publikus HTTP trust boundary. Apache vagy Nginx mögött két réteg van: a publikus proxy és az RWLang listener.

Productionban a forwarding headereket csak deklarált trusted proxy hálózathoz szabad elfogadni. A kliens által közvetlenül küldött `X-Forwarded-For`, `X-Forwarded-Proto` vagy hasonló header nem lehet authority.

A public host szintén trusted konfigurációból származzon. A redirect célját ne a tetszőleges kliens `Host` headeréből építsd fel.

Az RWLang HTTP parser szándékosan szigorú. A kétértelmű kérésformákat elutasítja. Ide tartozik például a több kritikus header, az obs-fold és a GET/HEAD body. A request `Transfer-Encoding` és az ismeretlen connection token szintén tiltott. A fejlesztői szabály egyszerű: **ne próbáld alkalmazáskódból lazítani az edge parser invariánsait.**

Fail-closed döntés és security audit

A trust-boundary tiltás ne csak a kliensválaszban jelenjen meg. A szerver ugyanazzal a saját request ID-val access logot és – ahol security-döntés történt – strukturált `security_audit` eseményt képez. A fontosabb osztályozás:

Határ	HTTP	Audit category/outcome
forwarding metadata	400	proxy/invalid_forwarding
Host pinning	421	host/mismatch
HTTPS policy	426	transport/https_required
auth hiány API-n	401	auth/unauthorized
CSRF	403	csrf/denied
CORS	403	cors/denied
Origin/Fetch Metadata	403	origin/denied
route/object policy	403	policy/forbidden
rate limit	429	rate_limit/denied

A security auditba ne kerüljön token, cookie, credential, teljes URL vagy request body. A category/outcome azt mondja meg, *melyik trust boundary döntött*, a részletes request-forenzikához pedig a request ID-val kapcsolódó access/server logot használd.

16.3. TLS és secret kezelés

A privát kulcs, adatbázis-credential, Redis URL, LDAP credential vagy más secret ne legyen:

- forráskódban;
- Git repositoryban;
- parancssori argumentumban;
- plaintext TOML secret értéként;
- access/server/audit logban.

A production minta a `*_file` hivatkozás:

```
[tls]
cert_file = "/run/secrets/tls/fullchain.pem"
key_file = "/run/secrets/tls/privkey.pem"
```

```
[database]
url_file = "/run/secrets/rwlang/database-url"
```

A config megmondja, *hol* található a secret; maga az érték külön jogosultsági határ mögött marad. A secret fájlok legyenek a service user számára olvashatók, más számára pedig a lehető legszűkebben hozzáférhetők.

16.4. Identity: authentication nem authorization

A sikeres login csak azt bizonyítja, hogy van trusted principal. Azt nem, hogy az adott cikket, számlát vagy admin műveletet elérheti.

A helyes rétegek:

1. authentication: ki a felhasználó;
2. route authorization: beléphet-e ebbe a handlerbe;
3. object authorization: hozzáférhet-e ehhez a konkrét rekordhoz;
4. üzleti invariáns: a művelet ebben az állapotban megengedett-e.

Kliens által küldött role, owner, tenant vagy ár soha ne legyen authority. Role csak trusted auth backendből, owner csak trusted rekordból, üzleti döntés pedig szerveroldali állapotból származzon.

Objektumszintű módosításnál a tipikus sorrend:

```
let article = Article.byId(db, id)?;
authorize article owner authorUsername or role Publisher;

transaction db {
  Article.publishChanged(tx, id, version)?;
  audit Article id action publish;
    from article.status
    to ArticleStatus.Published
}
```

Az authorization guard nem helyettesíti az optimistic lockingot, és az optimistic locking nem helyettesíti az authorizationt.

16.5. Session, CSRF és CORS együtt

A session cookie önmagában nem CSRF-védelem. State-changing form vagy JSON kérésnél session-bound CSRF védelem szükséges; JSON POST-nál a token külön headerben vihető.

A böngészőoldali védelmet defense-in-depthként érdemes kezelni:

```
session-bound CSRF token
+ same-origin Origin/Referer check
+ Sec-Fetch-Site cross-site deny
```

CORS csak explicit, pontos origin allowlistből nyíljon meg. Credentialed CORS esetén a wildcard különösen nem megfelelő. A CORS nem authentication és nem authorization: csak azt szabályozza, hogy a böngésző mely origin számára teszi olvashatóvá a választ.

16.6. SQL és üzleti mutáció

A compiler-owned SQL és typed bind paraméter a V1 egyik legerősebb biztonsági alapja. Felhasználói input ne kerüljön SQL string-összefűzésbe, dinamikus oszlop- vagy táblanévbe.

Mutáció transactionben történjen. A transaction itt nem csak konzisztenciaeszköz: security szempontból is fontos, mert az authorization után végzett több írás együtt sikerüljön vagy együtt bukjon el, az üzleti audit pedig ugyanahhoz az állapotváltozáshoz tartozzon.

A runtime DB credential legyen least privilege. A migration credential külön operátori capability: ne fusson a webserver folyamatosan olyan jogosultsággal, amellyel sémát is módosíthat.

16.7. Upload, média és AppFs

A feltöltött fájl minden byte-ja nem megbízható. A kliens filename és MIME csak metadata; storage kulcsnak, elérési útnak vagy file-type authoritynek ne használd.

A production baseline:

- server-owned célkönyvtár;
- byte limit;
- képnél pixel/decode limit és byte-alapú format validation;
- AppFs confinement;
- symlink/magic-link escape tiltása;
- runtime data külön a release/static fájloktól;
- csak a szükséges r/w/c capability.

A publikus static root és a user upload ne legyen ugyanaz a könyvtár csak azért, mert mindkettőt HTTP-n szeretnéd kiszolgálni. Más a lifecycle, a trust és a cache policy.

16.8. Redis és shared runtime state

Redis runtime-owned infrastructure, nem üzleti source of truth. Session, TOTP replay state, rate limit és public cache élhet benne, de az alkalmazás üzleti rekordjai ne azért kerüljenek oda, mert „gyorsabb”.

Több instance esetén shared session/rate-limit backend nélkül könnyen eltérő security döntés születne attól függően, melyik node kapta a kérést. Production clusterben ezért a shared state policy a topológia része.

Redis hibánál a security-releváns funkciók ne váljanak automatikusan megengedővé. Rate limit, replay protection vagy session validation esetén a fail-closed irány a biztonságos alap.

16.9. Outbound egress és SSRF

A külső hálózat külön trust boundary. V1-ben nincs korlátlan alkalmazásoldali `HttpClient`; a trusted integration adapter named targetet használ.

A célpolicy egyszerre korlátozza a hostot, címtartományt, portot, TLS-t, timeoutot és válasz-méretet. DNS feloldás után minden A/AAAA címet ellenőrizni kell, majd a tényleges peer címet kapcsolódás után újra kell validálni.

Tiltott minta:

```
user input URL -> generic server-side fetch
```

Ajánlott minta:

```

user/business choice
    |
    v
named integration target
    |
    v
trusted host/CIDR/port/TLS policy

```

Így a felhasználó üzleti választást tehet, de nem választhat tetszőleges hálózati célt.

V1-ben az egress a trusted Rust integration réteg capabilityje, nem általános `.rw` HTTP API. Emiatt az `IntegrationError::Policy` fail-closed tiltása nem automatikusan a webserver `security_audit` eseménye: az adapternek/operatori integrációnak kell a target nevét és a minimális outcome-ot naplóznia, ha erre forenzikus igény van. Secretet, credentialt, arbitrary teljes URL-t vagy response bodyt itt se logolj.

16.10. Resource limitek is security kontrollok

A biztonság nem csak confidentiality és authorization. Egy webalkalmazás akkor is sérülékeny, ha egyetlen kérés korlátlan memóriát, CPU-t, DB-időt vagy upload sávszélességet használhat.

A védelmi rétegek együtt dolgoznak:

- request/parser hard limit;
- upload és response méretlimit;
- query/result limit és timeout;
- rate limit;
- runtime resource profile;
- process/systemd/cgroup hard limit.

Az alkalmazás ne kérjen nagyobb resource profile-t „hátha kell” alapon. A nagyobb keret production policy-döntés és auditálható kivétel legyen.

16.11. Log hygiene és audit integrity

A logba kerülő információ maga is adatkiáramlási felület lehet. Ne logolj credentialt, cookie-t, CSRF/TOTP secretet, teljes form- vagy JSON-bodyt.

Security eseményhez inkább minimális, strukturált mezők kellenek: request ID, route, actor ahol indokolt, döntés típusa és státusz. Az üzleti audit pedig legyen tranzakciós és az alkalmazás állapotváltozásához kötött.

A request ID korrelációs azonosító, nem authority. A szerver sajátot generáljon; ne bíz a kliens által küldött azonosítóban.

16.12. Process és filesystem hardening

A production service unprivileged user alatt fusson. A release könyvtár és a konfiguráció legyen runtime szempontból read-only; írható csak az legyen, aminek valóban írhatónak kell lennie.

Ajánlott szétválasztás:

```

/usr/local/bin/rwlang-server
/usr/local/bin/rwlang-cli
/usr/local/etc/rwlang/      # trusted config, read-only
/srv/rwlang/releases/      # immutable releases
/srv/rwlang/data/          # declared persistent app data
/run/secrets/rwlang/       # secret files
/var/log/rwlang/           # logs

```

A systemd hardening, a cgroup limitek és az RWLang saját resource policyja ne egymás alternatívái legyenek; két külön védelmi réteget adnak.

16.13. Dependency és build supply-chain

Az RWLang deployed server bináris, ezért a dependency graph is release állapot. A committed Cargo.lock és a -locked build reprodukálhatóbbá és reviewzhatóvá teszi a dependency felületet.

Intentional dependency update után ne csak compile legyen:

```

./tools/refresh-lock.sh
./verify.sh
./tools/dependency-audit.sh

```

A RustSec audit ismert advisorykra ad bizonyítékot; nem helyettesíti a kódreview-t és az upstream changelog átnézését. Különösen parser, TLS, crypto, auth és database dependency major/minor migráció ne bújjon el más feature diffjében.

16.14. Production security gate

A production átadás előtti minimum ellenőrzés:

1. HTTPS és public host policy ellenőrzött.
2. Reverse proxy trust csak explicit CIDR-re nyitott.
3. Secret csak file-reference útvonalon jut a processbe.
4. Runtime DB credential least privilege; migration credential külön van.
5. Session/auth backend megfelel a single-node vagy multi-instance topológiának.
6. Minden state-changing böngészős művelet CSRF-védett.
7. CORS exact allowlist; credentialed wildcard nincs.
8. Object authorizationnál trusted principal/role és stabil owner authority van.
9. SQL csak compiler-owned statement + typed bind.
10. Upload server-owned destinationre és korlátozott AppFs capabilityvel kerül.
11. Redisből semmi üzleti source of truth nem hiányozna restore esetén.
12. Outbound integráció csak named egress policy-n keresztül ér el hálózatot.
13. Request, upload, DB, runtime és process resource limitek be vannak állítva.
14. Logokban nincs secret vagy teljes érzékeny request payload.
15. Security-negatív tesztek, config check és deploy smoke zöld.

16.15. Gondolkodási minta code reviewhoz

Minden új feature-nél öt kérdés sok security hibát korán megfog:

1. **Mi jön kívülről?** – Melyik mező, header, fájl, rekordazonosító vagy külső válasz nem megbízható?
2. **Hol válik typed adattá?** – Route parser, form/JSON schema, model vagy trusted adapter?
3. **Ki az authority?** – Session backend, adatbázisrekord, trusted config vagy egress policy?
4. **Mi történik hiba esetén?** – Fail closed, rollback, cleanup, 4xx/5xx és audit?
5. **Mi korlátozza az erőforrást?** – Méret, idő, darabszám, rate vagy process budget?

Ha ezekre nincs rövid, egyértelmű válasz, a feature security modellje valószínűleg még nincs kész.

17. fejezet

Naplózás, audit, metrics és hibakeresés

Egy production webalkalmazás nem attól üzemeltethető, hogy hiba esetén „kiír valamit a konzolra”. A fejlesztőnek és az üzemeltetőnek ugyanazokat az eseményeket kell tudnia összekapcsolni: melyik HTTP kérés futott, milyen route-ot ért el, milyen státusszal fejeződött be, történt-e biztonsági elutasítás, és maradt-e üzleti szempontból tartós auditnyom.

Az RWLang ehhez négy külön felületet ad:

- strukturált **server log** a technikai és lifecycle eseményekhez;
- strukturált **access log** a HTTP kérésekhez;
- strukturált **audit log** a security és felhasználói aktivitáshoz;
- külön listen socketen elérhető **Prometheus metrics** felületet.

Ehhez társul az alkalmazás adatbázisában tárolható, tranzakciós **business audit trail**. Fontos: az audit log és a business audit nem ugyanaz.

17.1. A három naplóosztály szerepe

Ajánlott production konfiguráció:

```
[observability]
metrics_listen = "127.0.0.1:9090"
allow_public_metrics = false
access_log = true

[logging]
server_file = "/var/log/rwlang/server.log"
access_file = "/var/log/rwlang/access.log"
audit_file = "/var/log/rwlang/audit.log"
stderr = true
```

A fájlok célja eltérő.

server.log Indulás, leállítás, dependency-hibák, technikai és belső runtime események. Ezt nézd először, ha a szerver nem indul, readiness hibás, vagy egy háttérkomponens problémát jelez.

access.log Egy rekord HTTP kérésenként. Ez adja a forgalmi és latency nyomot: method, route, státusz, időtartam, kliens IP, be- és kimenő byte-ok, request ID.

audit.log Biztonsági és felhasználói aktivitás: login/logout, auth vagy MFA failure, policy deny, rate-limit deny, resource-profile startup audit és autentikált state-changing action.

Az `access_log = false` csak az access logot kapcsolja ki. A security/user audit ettől még megmarad.

17.2. Request ID: egy kérés végigkövetése

Minden sikeresen parse-olt application, static vagy health request szerver-generált azonosítót kap:

```
X-Request-Id: rw-...
```

Az access log például ilyen lehet:

```
{"event": "http_request", "request_id": "rw-...",  
  "method": "GET", "route": "articleShow", "status": 200,  
  "duration_ms": 7, "client_ip": "203.0.113.5",  
  "bytes_in": 0, "bytes_out": 812}
```

A kliens által küldött `X-Request-Id` nem authority. Az RWLang saját request ID-t generál, így a támadó nem kényszerítheti rá a szerverre a saját korrelációs azonosítóját.

A hibakeresés alaplővelete ezért:

```
# 1. keresd meg a request ID-t az access logban  
grep 'rw-...' /var/log/rwlang/access.log  
  
# 2. keresd ugyanazt a technikai logban  
grep 'rw-...' /var/log/rwlang/server.log  
  
# 3. security esetben az audit logban is  
grep 'rw-...' /var/log/rwlang/audit.log
```

A három napló együtt adja a teljes technikai képet.

17.3. Mit szabad és mit tilos logolni?

A log nem debug adatlerakó. Productionban különösen tilos érzékeny request tartalmat automatikusan naplózni.

Az RWLang logfelületére nem kerülhet:

- **Authorization** header;
- Cookie vagy session ID;
- CSRF token;
- TOTP secret és recovery code;
- DB, Redis vagy LDAP credential;
- secret-file tartalma;
- teljes request body vagy form payload.

Ez adatvédelmi és biztonsági invariáns. Ha egy üzleti eseményhez mégis tartósan szükséges információ, ne a request bodyt dumpold: használj explicit, szűk business audit mezőket.

17.4. Security audit és HTTP elutasítások

A trust-boundary elutasításoknál az RWLang külön `security_audit` eseményt képez ugyanazzal a request ID-val. Ide tartozik a hibás/untrusted forwarding metadata 400-as elutasítása, a 401/403/421/426/429 security döntések, valamint a CSRF/CORS/origin tiltások finomabb kategorizálása. Így elkülöníthető például:

- `proxy/invalid_forwarding`: hibás vagy nem trusted forwarding metadata;
- `auth/unauthorized`: autentikáció hiánya API route-on;
- `csrf/denied`, `cors/denied`, `origin/denied`: böngészős trust-boundary tiltás;
- `policy/forbidden`: route- vagy object-authorization deny;
- `host/mismatch`, `transport/https_required`: Host/TLS boundary sérülése;
- `rate_limit/denied`: limiter miatti elutasítás.

A fejlesztő számára fontos különbség: a 403 nem feltétlen programhiba. Lehet helyes security-policy döntés is. Hibakereséskor mindig nézd meg az audit eseményt és a route policyt is.

17.5. Business audit: tartós üzleti változástörténet

Az access és security log operációs eseménynapló. Egy cikk publikálása, számla jóváhagyása vagy rendelés állapotváltása viszont üzleti állapotváltozás. Ezt az alkalmazás adatbázisában érdemes tranzakciósan rögzíteni.

Példa:

```
enum ArticleStatus {
    Review
    Published
}

transaction db {
    Article.publishChanged(tx, id, version)?;
    audit Article id action publish;
        from article.status
        to ArticleStatus.Published
}
```

Az audit ugyanabban a DB-tranzakcióban fut, mint a módosítás. Ha az audit rekord nem írható be, a business módosítás is rollbackel. Ez erősebb garancia, mint egy külön log sor kiírása.

Az RWLang a `_rw_business_audit` runtime táblába többek között az alábbiakat írja:

- runtime-generált eseményazonosító és UTC timestamp;
- HTTP request ID;
- canonical autentikált actor;
- source action;
- objektumtípus és objektumazonosító;
- statikus üzleti action név;
- opcionális előző és új domain állapot.

A business audit statement authorizationt nem helyettesít:

```
let invoice = Invoice.byId(db, id)?;
authorize invoice owner ownerUsername or role Accountant;

transaction db {
  Invoice.approve(tx, id, version)?;
  audit Invoice id action approve;
    from invoice.status
    to InvoiceStatus.Approved
}
```

A helyes sorrend: **azonosítás -> authorization -> módosítás -> ugyanazon tranzakción belüli audit.**

17.6. Prometheus metrics külön management socketen

A metrics nem application route. Külön listeneren fut:

```
[observability]
metrics_listen = "127.0.0.1:9090"
allow_public_metrics = false
```

Lekérdezés ugyanazon a hoston:

```
curl http://127.0.0.1:9090/metrics
```

A listener csak a `/metrics` GET/HEAD kérést szolgálja ki. Nem fut rajta alkalmazás-route, session vagy auth handler.

Tipikus metrikák:

```
rw_requests_total
rw_responses_5xx_total
rw_auth_failures_total
rw_csrf_failures_total
rw_policy_denials_total
rw_request_timeouts_total
rw_runtime_budget_exceeded_total
rw_rate_limit_denials_total
rw_readiness_failures_total
rw_request_bytes_in_total
rw_response_bytes_out_total
rw_active_connections
rw_log_fallback_total
rw_request_duration_ms_bucket
rw_route_requests_total{route="..."}
rw_route_5xx_total{route="..."}
rw_route_duration_ms_sum{route="..."}
```

A `rw_policy_denials_total` a security-policy tiltásokat követi: a 403/421/426 döntések mellett a kifejezetten forwarding metadata hibaként felismert 400-as elutasítás is ide számít. A normál malformed request 400 nem security policy denial.

A legfontosabb szabály a **bounded cardinality**. A `route` label compiler-owned route név vagy fix reserved név. Raw URL, path paraméter, IP, username, tenant, session ID és request ID soha ne legyen metric label. Ellenkező esetben egy támadó korlátlan idősor-számot hozhatna létre.

17.7. Apache vagy Nginx mellett ki scrape-elje a metricset?

A publikus webes proxy és a metrics listener külön trust boundary. Tipikus elrendezés:

```

Internet
  |
  v
Apache/Nginx :443
  |
  v
127.0.0.1:8080  RWLang server

Prometheus agent
  |
  v
127.0.0.1:9090  RWLang metrics

```

A metrics portot ne proxyd ki automatikusan ugyanarra a publikus domainre. Ha távoli Prometheus szervernek kell elérnie, használj privát management hálózatot, ACL-t vagy külön autentikált monitoring proxyt.

17.8. Health: liveness és readiness nem ugyanaz

Az RWLang beépített szerver endpointjai:

```

GET  /health/live
HEAD /health/live

GET  /health/ready
HEAD /health/ready

```

A liveness csak azt jelzi, hogy a szerver event loop él. Nem hív DB-t, Redis vagy LDAP-ot.

A readiness azt mutatja, hogy a kiszolgáláshoz szükséges konfigurált runtime dependency-k elérhetők. DB vagy Redis hiba/timeout esetén **503 Service Unavailable** lehet a válasz.

Ezért load balancernél:

- a **liveness** alapján ne vedd ki rögtön az instancet egy rövid DB kiesés miatt;
- a **readiness** alapján viszont ideiglenesen kivehető a forgalomból.

A health endpoint nem üzleti monitoring API: szándékosan minimális státuszt közöl.

17.9. Logrotate és SIGHUP

Az RWLang file logging append módban ír. Productionban a logrotációt az operációs rendszerre bíz. A repository példája: `examples/logrotate/rwlang`.

Egyszerűsített minta:

```
/var/log/rwlang/server.log /var/log/rwlang/access.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 rwlang rwlang
    sharedscripts
    postrotate
        systemctl kill -s HUP --kill-who=main rwlang.service || true
    endscript
}
```

A SIGHUP jelentése tudatosan korlátozott. Behind-proxy módban újraindítja a logfájlokat és tranzakciósan újraépíti a domain/application hosting állapotot; nem általános process-szintű `server.toml` rekonfiguráció. Az alkalmazásforrás változását normál esetben az automatikus source-reload supervisor kezeli.

Konfigurációmódosításnál:

```
/usr/local/bin/rwlang-server \
--config /usr/local/etc/rwlang/server.toml \
--check-config

systemctl restart rwlang.service
```

Ez auditálhatóbb és kiszámíthatóbb, mint egy részlegesen sikerülő hot reload.

17.10. Mi történik, ha a logfájl nem írható?

A file logging bounded writer queue-t használ. Ha a queue megtelik, a file sink kiesik vagy a writer nem érhető el, az RWLang stderr fallbacket használ és növeli a következő metrikát:

```
rw_log_fallback_total
```

Ezt érdemes riasztási feltételként kezelni. Ha növekszik, lehet:

- jogosultság- vagy filesystem-hiba;
- megtelt lemez;
- hibás logrotate/reopen;
- tartós log-processing nyomás.

A logfájl útvonalának parent directory-ja létezzen. Unix rendszeren a runtime a logfájlt 0640 móddal, symlink-követés nélkül nyitja meg; az operatornak ettől még helyesen kell kialakítania a könyvtárjogosultságokat.

17.11. Gyakorlati production hibakeresési folyamat

Egy 5xx vagy „lassú az oldal” bejelentésnél ne találgass. Használj reprodukálható sorrendet.

1. **Liveness:** fut-e maga a process?
2. **Readiness:** elérhető-e DB/Redis?
3. **Access log:** melyik route, státusz, duration és request ID érintett?
4. **Server log:** ugyanahhoz a request ID-hoz tartozik-e dependency/runtime hiba?
5. **Audit log:** security policy vagy rate limit utasította-e el?
6. **Metrics:** egyedi eset vagy trendszerű 5xx/latency növekedés?
7. **Reverse proxy:** a hiba az Apache/Nginx előtt, benne vagy az RWLang mögött keletkezik?

Példa gyors ellenőrzésre:

```
curl -i https://www.example.com/health/live
curl -i https://www.example.com/health/ready
# Proxy deploymentnel plusz belso upstream-proba:
curl -i http://127.0.0.1:8080/health/ready
curl -s http://127.0.0.1:9090/metrics | grep rw_responses_5xx_total
journalctl -u rwwlang.service --since "10 min ago"
```

Ha a loopback RWLang endpoint jó, de a publikus domain rossz, a következő vizsgálati réteg az Apache/Nginx, TLS vagy forwarded-header policy. Ha a loopback readiness is 503, előbb a backend dependency-ket vizsgáld.

17.12. Minimum monitoring baseline

Egy kisebb production telepítésnél is legyen legalább riasztás az alábbiakra:

- readiness tartósan nem ready;
- 5xx arány vagy számláló hirtelen növekedése;
- request latency érdemi romlása;
- auth/policy/rate-limit denial szokatlan kiugrása;
- `rw_runtime_budget_exceeded_total` növekedése;
- `rw_log_fallback_total` növekedése;
- process restart loop vagy váratlan shutdown.

A konkrét thresholdot nem a nyelvnek kell hardcode-olnia. A forgalom, SLO és üzleti kritikalitás alapján az üzemeltető állítja be.

17.13. Retention, SIEM és auditvédelem

A retention szervezeti és jogi policy. Az RWLang nem kényszerít rá univerzális megőrzési időt.

A helyi `audit.log` fontos operációs nyom, de önmagában nem erős tamper-evident archívum, mert a service host adminisztrátora vagy megfelelő filesystem jogosultságú szereplő módosíthatja. Magasabb assurance esetén továbbítsd centralizált, hozzáférés-védett vagy append-only log/SIEM rendszerbe.

A business audit adatbázis-tábla ugyancsak nem véd a DB-admin közvetlen módosítása ellen. A két réteg együtt ad jó alkalmazási nyomot, de compliance környezetben a külső, független logarchívum marad fontos.

17.14. Ellenőrzőlista

Production indulás előtt ellenőrizd:

- külön server/access/audit log útvonal van;
- a logkönyvtár jogosultságai szűkek;
- request body és secret nincs logolva;
- a metrics listener nem publikus véletlenül;
- a Prometheus labelk bounded cardinality-jűek;
- a liveness és readiness külön van kezelve;
- logrotate után SIGHUP történik, configmódosítás után restart;
- `rw_log_fallback_total` monitorozva van;
- üzleti workflow esetén a business audit ugyanabban a DB-tranzakcióban fut;
- az audit log központi továbbításáról és retentionjéről van dokumentált döntés.

18. fejezet

Production deployment és service lifecycle

Ez a fejezet nem ismétli meg a konfigurációs referencia és az observability részleteit. A production szempontból azt foglalja össze, hogyan lesz egy ellenőrzött release-ből futó service, hogyan történik a release-váltás, és hogyan marad kontrollált a process lifecycle. A teljes backup/restore és disaster recovery a következő fejezet önálló témája.

18.1. Release útvonal

Ajánlott sorrend:

```
./verify.sh
-> cargo build --locked --release
-> immutable release directory
-> rwlang-server --config ... --check-config
-> rwlang-cli migrate verify
-> approved rwlang-cli migrate apply
-> rwlang-cli migrate verify
-> atomic current symlink switch
-> controlled restart
-> /health/live
-> /health/ready
-> log / metric / audit review
```

A buildelt **rwlang-server** és **rwlang-cli** bináris, az alkalmazásforrás és a migration legyen ugyanahhoz a release-azonosítóhoz köthető. Ne egy élő **/srv/rwlang/current** könyvtár tartalmát módosítsd fájlanként; új immutable release könyvtárat készíts, majd atomikusan válts symlinket.

18.2. systemd service

A **rwlang-server** hosszú életű service process. Az **rwlang-cli** nem daemon: migration, auth-admin vagy más operátori művelet idejére indul és kilép.

```
[Service]
User=rwlang
Group=rwlang
```

```
ExecStart=/usr/local/bin/rwlang-server \
  --config /usr/local/etc/rwlang/server.toml
Restart=on-failure
NoNewPrivileges=true
PrivateTmp=true
```

A service fusson unprivileged user alatt. A pontos systemd hardening és cgroup limitek igazodjanak a `server.toml` resource policy-jához; a két réteg célja ugyanaz: egy hibás vagy drága kérés ne vehesse át a gépet.

18.3. Controlled restart és readiness

Process-szintű konfigurációmódosítás után előbb fusson `-check-config`, majd controlled restart. Behind-proxy módban a SIGHUP a logok újrainvitása mellett tranzakciósan újraépíti a domain/application hosting állapotot, de listener, DB/Redis/auth kapcsolat, cgroup policy, logging sink és más process-szintű beállítás továbbra is restartot igényel.

Az alkalmazás-forrás kiadásának van könnyebb útja. A közös source-reload supervisor minden betöltött alkalmazás entrypointját és teljes tranzitív modulgráfját `mtime + size` alapján figyeli. Konfigurálható debounce után candidate runtime-ot fordít és validál, majd csak siker esetén cseréli le atomikusan az adott domaint. Hibás feltöltésnél a korábbi generáció fut tovább, a reload-hiba pedig naplózódik. A logikai entrypoint útvonal is figyelt, ezért az immutable release-ek közötti atomikus `current` symlink-csere automatikusan aktiválja az új kódot. Sikeres kódreload a public-cache generationöket is előrelépteti, ezért a régi renderelt tartalom nem marad életben pusztán a korábbi TTL miatt.

A forgalomterelő csak ready instance-ra küldjön kérést. Rolling deploymentnél az új instance előbb induljon el, legyen ready, és csak utána kerüljön ki a régi. A health endpointok és a hozzájuk tartozó diagnosztika részletesen az observability fejezetben szerepel.

18.4. Migration és rollback nem ugyanaz

Az alkalmazás binárisának vagy forrásának visszaváltása egyszerű lehet, az adatbázis-séma visszaállítása viszont nem. Ezért production migrationnél preferáld az expand/contract mintát: az új release előbb backward-compatible sémát vezessen be, az inkompatibilis takarítás csak későbbi release-ben történjen.

Ne kapcsolj automatikus reverse SQL-t az alkalmazás symlink visszaállításához. Ha a migration adatot alakított át vagy törölt, a valódi recovery pont a bizonyított backup lehet.

18.5. Recovery gate a deployment előtt

Destruktív vagy schema-kompatibilitást érintő release előtt legyen ismert recovery point és friss restore-evidence. A backup tartalma, a DB/AppFs konzisztencia, az RPO/RTO, valamint a rollback és restore döntési fa a következő fejezetben szerepel. A deployment felelőssége itt az, hogy a release-váltás előtt ez a gate teljesüljön.

18.6. Deploy utáni minimum ellenőrzés

1. az új process fut és ready;
2. a publikus főoldal vagy smoke route válaszol;

3. nincs új error spike a server logban;
4. access logban a státuszkód-eloszlás normális;
5. audit log írható;
6. DB/Redis dependency metrikák nem romlottak;
7. egy rollback döntéshez ismert a release hash és az adatbázis migration állapota.

19. fejezet

Backup, restore és disaster recovery

A backup célja nem az, hogy legyen valahol egy másolat, hanem az, hogy egy ismert recovery pontról **bizonyíthatóan újraindítható legyen az alkalmazás**. Egy RWLang rendszer tartós állapota ráadásul nem feltétlenül egyetlen adatbázis: az application DB, a local-auth identity store és az AppFs együtt is alkothat üzleti állapotot.

Ezért a helyes kérdés nem az, hogy „mentjük-e az adatbázist?”, hanem az, hogy **mi a teljes recovery state, milyen konzisztencia mellett mentjük, és rendszeresen bizonyítjuk-e a visszaállítást**.

19.1. Először készíts state inventoryt

Production előtt írd össze, mi tartozik az alkalmazás állapotához. Tipikusan:

- application database;
- local-auth SQLite DB, ha local auth használatban van;
- AppFs `data_root`, például feltöltött média és más runtime adat;
- az immutable release azonosítója és artifact hash-e;
- a migration set és checksumok;
- a trusted config/policy fájlok verziója vagy hash-e;
- a backup időpontja, az alkalmazásverzió és a DB backend.

A secret fájlok külön secret-management boundaryhoz tartoznak. A recovery runbooknak tudnia kell, honnan állíthatók helyre a szükséges DB-, Redis-, TLS- vagy LDAP-credentialök, de **secret érték ne kerüljön plaintext release- vagy backup-manifestbe**.

19.2. Mi nem üzleti source of truth?

A V1 Redis-használat session-, rate-limit- és public-cache állapot. Ezeket ne kezeld üzleti backup részeként.

Disaster recovery után biztonságos és elfogadható baseline az üres Redis:

- a sessionök megszűnnek, ezért a felhasználók újra belépnek;
- a public cache újraépül;
- a rate-limit ablakok újraindulnak.

Ha ugyanaz a Redis cluster más alkalmazás üzleti adatait is tárolja, annak mentése már annak a rendszernek a saját backup-contractja.

19.3. RPO és RTO: a technikai mentés előtt üzleti döntés

Két számot még a backup implementálása előtt rögzíteni kell:

RPO – Recovery Point Objective legfeljebb mennyi adatvesztés fogadható el;

RTO – Recovery Time Objective mennyi idő alatt kell helyreállnia a szolgáltatásnak.

Ha például az RPO 24 óra, egy napi backup elvileg elegendő lehet. Ha az RPO néhány perc, már DB-szintű WAL/binlog/PITR vagy más infrastruktúra kellhet. Az RWLang runtime nem tud szervezeti RPO/RTO-t kitalálni; ezt az operatornak kell a backup technológiára lefordítania.

19.4. A V1 biztonságos baseline: kontrollált maintenance backup

A DB és az AppFs együtt alkothat üzleti állapotot. Egy DB sor például hivatkozhat egy feltöltött képre. Külön időpontban készült DB dump és AppFs tar ezért nem automatikusan konzisztens recovery point.

A legegyszerűbb auditálható V1 baseline:

```
traffic drain
-> SIGTERM / graceful shutdown
-> stopped state verified
-> application DB backup
-> local-auth DB backup, if used
-> AppFs data_root snapshot
-> recovery manifest + hashes
-> service start
-> readiness
```

Ez rövid írási szünettel jár, viszont világos konzisztenciahatárt ad. Online backup csak akkor nevezhető konzisztensnek, ha a DB/storage infrastruktúra ugyanahhoz a recovery ponthoz kötött snapshotot biztosít, vagy az alkalmazás adatmodellje bizonyíthatóan tolerálja az eltérő snapshot-időpontokat. A runtime önmagában nem ígér distributed snapshotot.

19.5. Application DB: a backend saját backup eszközt használj

Az RWLang nem próbálja elrejtetni a natív adatbázis-backup mechanizmusokat. Az operator a választott backend támogatott eszközével dolgozzon, lehetőleg külön backup credentialdel.

PostgreSQL

Egy tipikus logikai backup:

```
pg_dump --format=custom --no-owner --no-acl \
--file app.dump "$DATABASE_URL"
```

Izolált restore-próbához:

```
createdb rwlang_restore_test
pg_restore --no-owner --no-acl \
--dbname rwlang_restore_test app.dump
```

A production secretet ne tedd shell historyba. Nagy adatbázisnál fizikai backup és PITR lehet jobb választás; annak runbookja már a PostgreSQL platform része.

MariaDB

Tranzakcionális InnoDB schema esetén tipikus logikai mentés:

```
mariadb-dump --single-transaction \  
--routines --triggers app > app.sql
```

Restore izolált adatbázisba:

```
mariadb rwlang_restore_test < app.sql
```

A `-single-transaction` nem tesz nem tranzakcionális táblákat varázsütésre konzisztenssé. Vegyes schema esetén maradjon a kontrollált maintenance baseline.

SQLite

Élő SQLite adatbázist ne nevezd backupnak csak azért, mert `cp`-vel lemásoltad. Használd a SQLite backup mechanizmusát vagy stopped állapotban fájlrendszer-snapshotot.

```
sqlite3 /srv/rwlang/data/app.db \  
".backup '/backup/app.db'"
```

A restore-próba mindig külön test pathra történjen; ne production DB-n próbáld ki, hogy működik-e a mentés.

19.6. A local-auth DB külön security backup

Local auth használatakor az identity store külön SQLite boundary. Password hash, TOTP secret és recovery hash is lehet benne, ezért backupját **secretként** kezeld.

```
sqlite3 /srv/rwlang/auth/users.db \  
".backup '/backup/local-auth.db'"
```

A backup legyen titkosított storage-on, szűk hozzáféréssel és dokumentált retentionnel. Restore után ellenőrizd a fájl és a parent directory ownership/permission értékeit is. Az identity store elvesztése nem rekonstruálható az application DB-ből.

19.7. AppFs: a teljes data root tartós állapot lehet

Ne csak a HTTP-n publikált media alkönyvtárat mentsd. A deklarált AppFs `data_root` teljes tartalma lehet recovery state.

Stopped baseline mellett például:

```
tar -C /srv/rwlang -cpf data-root.tar data
```

Restore után őrizd meg vagy állítsd vissza a szükséges ownership/permission értékeket. A service user csak a konfigurációban deklarált writable rootokat írhasa; symlinkkel ne lazítsd az AppFs confinementet.

19.8. Minden recovery ponthoz tartozzon manifest

A backup objektumok önmagukban nem mondják meg, melyik release-hez és migration állapothoz tartoznak. Készíts secretmentes manifestet:

```
backup_id=2026-09-04T180000Z
release_id=2026-09-04.1
server_sha256=...
app_source_sha256=...
migrations_sha256=...
config_sha256=...
db_backend=postgresql
app_db_backup_sha256=...
local_auth_backup_sha256=...
data_root_backup_sha256=...
```

A manifest hashokat és azonosítókat tartalmazzon, ne credential-értékeket. A mellékelt `deploy/release-record.sh` read-only módon segít a release/source/migration/config állapot rögzítésében.

19.9. Restore drill: a backup bizonyítása

A sikeres backup job még nem bizonyít recovery képességet. Rendszeresen, izolált környezetben restore-próbát kell futtatni.

```
new empty restore DB
-> application DB restore
-> local-auth restore, if used
-> AppFs restore into separate root
-> select exact immutable release
-> restore-test config on isolated host/port
-> rqlang-server --check-config
-> rqlang-cli check main.rw
-> rqlang-cli migrate verify
-> start rqlang-server on isolated listener
-> /health/live + /health/ready
-> application-specific smoke test
-> log and audit verification
```

A `deploy/restore-verify.sh` a nem mutáló config/source/migration ellenőrzések egy részét automatizálja. Nem indít production listenert és nem futtat `migrate apply`-t.

A drill eredményét dátummal, backup ID-val és release ID-val dokumentáld. A „backup elkészült” nem release gate; a szervezet RPO/RTO elvárásához illő, friss restore-evidence kell.

19.10. Upgrade előtt legyen recovery döntés

Egy kontrollált production upgrade ajánlott sorrendje:

```
immutable release artifact + hashes
-> migration verify
-> compatibility decision
-> consistent backup + recent restore evidence
-> migration apply
```

```
-> migration verify
-> atomic release switch
-> controlled restart
-> live + ready + smoke
-> logs + metrics + audit review
```

A migration kompatibilitását release-szinten rögzíteni kell. A kulcskérdés: a régi alkalmazás fut-e az új schema mellett?

19.11. Expand/contract tartja nyitva a gyors rollback ablakot

Ha az új schema backward-compatible, egy hibás application release visszakapcsolható a korábbi immutable artifactra DB restore nélkül.

Ajánlott minta:

1. **expand**: új nullable/defaultolt oszlop, tábla vagy index úgy, hogy a régi alkalmazás tovább fusson;
2. deployold az új alkalmazást;
3. ha kell, végezz kontrollált adat-backfillt;
4. csak későbbi release-ben **contract**: távolítsd el a régi oszlopot/constraintet, amikor a rollback window már lezárult.

Destruktív rename/drop vagy azonnali inkompatibilis shape-váltás bezárhatja az app-only rollback lehetőségét. Ezt release előtt explicit jelöld.

19.12. Rollback és restore két külön művelet

Hibás alkalmazás, backward-compatible schema

```
traffic drain
-> previous immutable release
-> controlled restart
-> live + ready + smoke
```

Ilyenkor nincs DB restore, ezért a hiba után létrejött üzleti adat megmarad.

Hibás alkalmazás, de forward fix lehetséges

Preferáld az új immutable release-t vagy forward migrationt. Ne köss automatikus reverse SQL-t a release symlink visszaváltásához.

Schema vagy adat sérült

Ez már disaster recovery:

```
stop writes
-> choose and record recovery point
-> restore DB + auth DB + AppFs consistently
-> select matching release and config
-> verify + smoke
-> controlled return to traffic
```

A recovery point után keletkezett írások elveszhetnek. Ez üzleti incidensdöntés, nem technikai részlet: az adatvesztési ablakot explicit vállalni és dokumentálni kell.

19.13. Tiltott rövidítések

- élő SQLite DB egyszerű `cp` másolatát backupnak nevezni;
- csak az application DB-t menteni, ha AppFs is üzleti állapot;
- restore-próba nélkül sikeresnek tekinteni a backupot;
- automatikus down migrationt kötni application rollbackhez;
- production DB-n restore-próbát végezni;
- migration credentialt átadni a normál application service-nek;
- secret értéket release- vagy backup-manifestbe írni;
- Redis session/cache restore-t a disaster recovery feltételévé tenni.

19.14. Production recovery checklist

Egy rendszer akkor tekinthető recovery-readynek, ha legalább az alábbiak igazak:

1. a state inventory dokumentált;
2. az RPO és RTO rögzített;
3. a backup titkosítási, retention- és hozzáférési policy ismert;
4. van azonosítható, friss sikeres restore drill;
5. a release migration kompatibilitása dokumentált;
6. az app-only rollback eljárás próbált;
7. a destruktív restore tulajdonosa és jóváhagyási útja ismert;
8. Redis elvesztésének hatása elfogadott;
9. a recovery runbookból nem hiányzik a release/config/secret visszaállítási forrás;
10. incidens után live, ready, smoke, log és audit ellenőrzés is történik.

A recovery legfontosabb elve: **a mentés nem fájl, hanem bizonyított visszaállítási képesség**. Ha a restore drill nem fut, az RPO/RTO csak feltételezés.

VI. rész

Referencia és operátori gyorssegédlet

20. fejezet

Az RWLang CLI és a napi fejlesztői/üzemeltetői workflow

Az RWLang két telepített programot ad: a hosszú életű `rwlang-server` processzt és a rövid életű `rwlang-cli` operátori eszközt. A kettőt érdemes következetesen külön kezelni. A szerver HTTP-t szolgál ki és runtime policyt érvényesít; a CLI forrást ellenőriz, migrációt kezel és local-auth adminisztrációt végez.

Ez a fejezet nem új nyelvi feature-öket vezet be. Azt mutatja meg, hogyan áll össze a napi munka egy ismételhető parancsfolyamattá a szerkesztéstől a production ellenőrzésig.

20.1. Melyik feladathoz melyik program?

Feladat	Eszköz	Megjegyzés
RWLang forrás ellenőrzése	<code>rwlang-cli check</code>	Compiler-szintű, nem indít HTTP szervert.
Migration status/verify/apply	<code>rwlang-cli migrate</code>	DB URL secret fájlból.
Local-auth store admin	<code>rwlang-cli auth</code>	SQLite identity store, operátori művelet.
Server config preflight	<code>rwlang-server</code>	<code>-check-config</code> : config + app + lokális policy/resource fájlok ellenőrzése.
Effective config megtekintése	<code>rwlang-server</code>	<code>-print-effective-config</code> ; a secret értékek redaktálva maradnak.
Webalkalmazás futtatása	<code>rwlang-server</code>	Hosszú életű processz, proxy vagy közvetlen HTTPS mögött.
Workspace release build	<code>cargo build</code>	<code>-locked -release</code> módban készül a két bináris.
Platform teljes verifikációja	<code>./verify.sh</code>	Az RWLang repository release-evidence része, nem minden alkalmazás saját parancsa.

A production példákban a programok helye:

```
/usr/local/bin/rwlang-cli
```

```
/usr/local/bin/rwlang-server
```

A shell normál PATH beállítása mellett ezért a további példákban elég a rövid programnevet használni.

20.2. A CLI nem a szerver alternatívája

A `rwlang-cli` nem indít listenert, nem szolgál ki route-ot és nem tart fenn session state-et. Egy parancsot végrehajt, majd kilép. Ez biztonsági szempontból is hasznos: migration vagy user-admin művelethez nem kell külön admin HTTP endpointot nyitni.

A másik irányban a `rwlang-server` nem futtat automatikus migrationt startupkor. A schema változás explicit operátori döntés marad.

20.3. Az első parancs: `rwlang-cli check`

Napi fejlesztésben minden érdemi forrásmódosítás után ez legyen az első kapu:

```
rwlang-cli check main.rw
```

Siker esetén a CLI rövid összefoglalást ír a fordító által látott model-, query-, page-, action- és route-számról. Hiba esetén nem érdemes szerveret indítani: előbb a compile-time contractot kell kijavítani.

A `check` többek között olyan hibákat fog meg, mint:

- ismeretlen típus vagy hibás nyelvi szintaxis;
- query projection és model shape eltérése;
- hibás typed bind/decode contract;
- route/policy vagy resource profile hivatkozási hiba, amely compiler-szinten bizonyítható;
- tiltott raw HTML/SQL jellegű konstrukció.

A `check` viszont nem bizonyítja, hogy a DB vagy Redis hálózaton elérhető, és nem helyettesíti a HTTP integration tesztet.

20.4. Forrásfábol futtatás és telepített CLI

Az RWLang repository fejlesztésekor ugyanaz a program Cargo-n keresztül is futtatható:

```
cargo run --locked -p rwlang-cli -- check main.rw
```

Ez fejlesztői kényelmi forma. Production runbookban az immutable, telepített release-binárist használ:

```
/usr/local/bin/rwlang-cli check /srv/rwlang/app/main.rw
```

Ne keverd a kettőt egy deploymentben. A production művelet legyen egyértelműen egy release artifacthoz köthető.

20.5. Migration workflow

A migration parancs három műveletet ad:

status megmutatja az applied és pending migrationöket;
verify ellenőrzi a migration historyt és checksum-contractot;
apply lock mellett alkalmazza a pending migrationöket.

A DB URL nem parancssori plaintext secret, hanem egy egysoros secret fájl:

```
/run/secrets/rwlang/db-url
```

A tipikus ellenőrzési sorrend:

```
rwlang-cli migrate status \  
  --dir migrations \  
  --db-url-file /run/secrets/rwlang/db-url  
  
rwlang-cli migrate verify \  
  --dir migrations \  
  --db-url-file /run/secrets/rwlang/db-url
```

Jóváhagyott schema-változásnál:

```
rwlang-cli migrate apply \  
  --dir migrations \  
  --db-url-file /run/secrets/rwlang/db-url \  
  --lock-timeout-secs 30  
  
rwlang-cli migrate verify \  
  --dir migrations \  
  --db-url-file /run/secrets/rwlang/db-url
```

A `-lock-timeout-secs` értéke pozitív egész. Ha a migration lock nem szerezhető meg a megadott időn belül, a művelet hibával álljon meg; ne legyen párhuzamos schema-módosítás.

Az `-allow-insecure-db` nem kényelmi kapcsoló productionre

A migration CLI is ugyanazt a secure-by-default szemléletet követi, mint a server. Az `-allow-insecure-db` csak tudatos, kontrollált környezetben használható. Productionben a megfelelő TLS/DB security legyen az alapértelmezett, ne egy állandósult bypass flag.

20.6. Local-auth adminisztráció

Ha az alkalmazás local authot használ, az identity store kezelése szintén a CLI feladata. Először inicializálni kell a store-t:

```
rwlang-cli auth init \  
  --db-url-file /run/secrets/rwlang/local-auth-db-url
```

Felhasználó létrehozása:

```
rwlang-cli auth user-add \  
  --db-url-file /run/secrets/rwlang/local-auth-db-url \  
  --username <username> \  
  --password <password>
```

```
--username alice \  
--password-file /run/secrets/rwlang/bootstrap-password \  
--role Editor
```

Ha nincs `-role`, új usernél a CLI a `User` role-t használja. Több role több `-role` kapcsolóval adható meg.

Jelszócsere és account állapot:

```
rwlang-cli auth password-set \  
  --db-url-file /run/secrets/rwlang/local-auth-db-url \  
  --username alice \  
  --password-file /run/secrets/rwlang/new-password  
  
rwlang-cli auth disable \  
  --db-url-file /run/secrets/rwlang/local-auth-db-url \  
  --username alice  
  
rwlang-cli auth enable \  
  --db-url-file /run/secrets/rwlang/local-auth-db-url \  
  --username alice
```

Role-csere:

```
rwlang-cli auth roles-set \  
  --db-url-file /run/secrets/rwlang/local-auth-db-url \  
  --username alice \  
  --role Editor --role Publisher
```

A `roles-set` a megadott role-listát tekinti az új állapotnak; az operátor ne úgy kezelje, mint egyetlen role inkrementális hozzáadását.

20.7. TOTP adminisztráció különösen érzékeny művelet

TOTP enrolment:

```
rwlang-cli auth totp-enroll \  
  --db-url-file /run/secrets/rwlang/local-auth-db-url \  
  --username alice \  
  --recovery-count 8
```

A parancs kiírja a TOTP secretet, az `otpauth` URI-t és az egyszer megjelenő recovery code-okat. Emiatt ezt ne futtasd CI logban, shell transcriptben vagy közös terminálon. Az operátori folyamatnak biztosítania kell a secret és recovery code-ok azonnali, biztonságos átadását és tárolását.

TOTP kikapcsolása:

```
rwlang-cli auth totp-disable \  
  --db-url-file /run/secrets/rwlang/local-auth-db-url \  
  --username alice
```

20.8. A secret fájl is input contract

A CLI a DB URL és password fájlokat nem korlátlanul olvassa. A jelenlegi contract szerint a secret URL fájl:

- regular file;
- nem symlink;
- legfeljebb 16 KiB;
- pontosan egy nem üres sort tartalmaz.

A password file szintén regular, nem symlink fájl; legfeljebb 4096 byte, egyetlen sor, a jelszó pedig 12–1024 byte közötti.

Ezért ne process substitutionnel, symlinkkel vagy több soros secret template-tel próbáld megkerülni a contractot. A CLI boundary szándékosan szűk.

20.9. Server preflight: a CLI után a szerver saját ellenőrzése jön

A forrás és migration ellenőrzése után a production konfigurációt a szerverrel validáld:

```
rwlang-server \  
  --config /usr/local/etc/rwlang/server.toml \  
  --check-config
```

Ez beolvassa a trusted TOML-t, lefordítja az alkalmazást, ellenőrzi a route-rate/resource policy hivatkozásokat, TLS konfigurációt, valamint a deklarált storage/static rootok lokális feltételeit.

Ha a precedence-t vagy az effektív policyt kell auditálni:

```
rwlang-server \  
  --config /usr/local/etc/rwlang/server.toml \  
  --print-effective-config
```

A kimenet kifejezetten secret-redacted. Ettől még operátori információ, ezért automation logban csak indokolt retentionnel tárold.

20.10. Napi fejlesztői ciklus

Egy kis vagy közepes alkalmazásnál jó alapfolyamat:

```
edit .rw source  
-> rwlang-cli check main.rw  
-> rwlang-cli migrate verify  
-> if schema changed: migrate status/apply/verify on dev DB  
-> rwlang-server --config server.dev.toml --check-config  
-> start rwlang-server  
-> HTTP/form/JSON smoke tests  
-> automated application tests
```

A sorrend lényege, hogy az olcsóbb és determinisztikusabb hibákat előbb fogjuk meg. Nincs értelme HTTP smoke tesztet futtatni olyan forrásra, amely már compiler-szinten hibás.

20.11. Application release workflow

Egy production release minimális operátori folyamata:

```
immutable release selected
-> rwlang-cli check deployed main.rw
-> rwlang-server --check-config
-> rwlang-cli migrate status
-> rwlang-cli migrate verify
-> backup/recovery gate
-> approved rwlang-cli migrate apply, if pending
-> rwlang-cli migrate verify
-> controlled service restart/switch
-> /health/live
-> /health/ready
-> application smoke test
-> logs + metrics + audit review
```

A migration `apply` az egyetlen mutáló schema-lépés ebben a folyamatban; külön `approval` pontként kezeld.

20.12. Az RWLang platform release workflowja más

Ha magát az RWLang repositoryt buildeljük, a Rust workspace release-evidence is része a folyamatnak:

```
./verify.sh
cargo build --locked --release \
  -p rwlang-server -p rwlang-cli
```

A kész binárisok:

```
target/release/rwlang-server
target/release/rwlang-cli
```

Telepítés:

```
sudo install -m 0755 target/release/rwlang-server \
  /usr/local/bin/rwlang-server
sudo install -m 0755 target/release/rwlang-cli \
  /usr/local/bin/rwlang-cli
```

A saját RWLang webalkalmazásod napi workflowjához nem kell minden alkalommal a nyelv teljes Rust workspace-ét verifikálni. Ott a saját source-, migration-, config- és integration tesztjeid a relevánsak.

20.13. Automatizálás: read-only és mutáló lépések legyenek külön

CI/CD-ben különítsd el a biztonságosan ismételhető ellenőrzéseket a state-et módosító műveletektől.

Read-only/preflight:

```

rqlang-cli check main.rw
rqlang-cli migrate status \
  --dir migrations --db-url-file /run/secrets/rqlang/db-url
rqlang-cli migrate verify \
  --dir migrations --db-url-file /run/secrets/rqlang/db-url
rqlang-server --config /usr/local/etc/rqlang/server.toml \
  --check-config
rqlang-server --config /usr/local/etc/rqlang/server.toml \
  --print-effective-config
health/readiness GET

```

Mutáló/operator-approved:

```

rqlang-cli migrate apply \
  --dir migrations --db-url-file /run/secrets/rqlang/db-url
rqlang-cli auth init \
  --db-url-file /run/secrets/rqlang/local-auth-db-url
# A többi auth művelet ugyanilyen explicit --db-url-file
# mellett kapja a saját --username/--password-file/--role opcióit.

```

Az auth-admin parancsokat általában ne tedd automatikus application deployment pipeline-ba. Ezek identity-admin műveletek, külön audit- és hozzáférési boundaryval.

20.14. Gyakori hibák

check sikeres, a server mégsem indul A compile-time contract rendben van, de konfigurációs, TLS-, filesystem- vagy policy-startup hiba van. Futtasd a `--check-config` ellenőrzést és olvasd a server logot.

--check-config sikeres, readiness piros A lokális config helyes, de runtime dependency – például DB vagy Redis – nem érhető el vagy nem egészséges.

migrate verify hibázik Ne futtass reflexből apply-t. Előbb tisztázd a history/checksum eltérést.

Auth CLI nem nyitja meg a DB-t Ellenőrizd, hogy local-auth SQLite URL-t adtál-e, és a store inicializálva van-e.

Secret file hibát kap Ne lazítsd a fájlcontractot; regular, nem symlink, egysoros secretet adj.

20.15. Egy oldalnyi operátori runbook

Ha csak a minimális sorrendet akarod megjegyezni:

```

# source
rqlang-cli check main.rw

# schema
rqlang-cli migrate status ...
rqlang-cli migrate verify ...

# config
rqlang-server --config /usr/local/etc/rqlang/server.toml \
  --check-config

# approved change

```

```
rwlang-cli migrate apply ...
rwlang-cli migrate verify ...

# service
systemctl restart rwlang

# end-to-end health through the public TLS edge
curl -fsS https://www.example.com/health/live
curl -fsS https://www.example.com/health/ready

# proxy topology: optional local upstream probe
curl -fsS http://127.0.0.1:8080/health/ready

# evidence
review server/access/audit log + metrics
```

A konkrét systemd unit neve deploymentenként eltérhet; a könyvben szereplő `rwlang` csak példa. A fontos invariáns a sorrend: **source** → **schema** → **config** → **approved mutation** → **service** → **health** → **evidence**.

21. fejezet

A `server.toml` konfigurációs referencia

A korábbi fejezetek feladatorientáltan használták a szerverkonfigurációt. Ez a fejezet egy helyen adja meg a V1 production konfiguráció teljes, operátori nézetét. A referencia alapja a tényleges parser és a `config/server.toml.sample`; ha a kettő között eltérés lenne, a parser viselkedése az irányadó.

21.1. Betöltés, precedence és fail-closed szabályok

Az ajánlott production indítás:

```
rwlang-server \  
  --config /usr/local/etc/rwlang/server.toml
```

A precedence:

```
built-in defaults < server.toml < explicit CLI override
```

A TOML trusted konfiguráció, de a parser akkor is fail-closed:

- ismeretlen section vagy kulcs: startup error;
- duplikált kulcs vagy tábla: TOML parse error;
- a config fájl csak reguláris, nem symlink fájl lehet;
- maximum mérete 1 MiB;
- TOML-ból érkező filesystem path abszolút kell legyen;
- secret nem kerülhet plaintext mezőbe: DB/Redis/LDAP/local-auth credential csak `*_file` hivatkozással adható meg.

Ellenőrzés listener indítása nélkül:

```
rwlang-server \  
  --config /usr/local/etc/rwlang/server.toml \  
  --check-config
```

Secretmentes effective nézet:

```
rwlang-server \  
  --secretmentes
```

```
--config /usr/local/etc/rwlang/server.toml \
--print-effective-config
```

21.2. Hogyan olvasd a táblákat?

A **beépített default** azt jelenti, ami config nélkül érvényes. A **production minta** a projekt `config/server.toml.sample` ajánlott kiinduló értéke. A kettő nem mindig azonos: a sample több helyen tudatosan nagyobb, productionre szánt keretet ad.

21.3. [server] és [tls]

Kulcs	Beépített default	Jelentés / production megjegyzés
<code>server.app</code>	nincs	Az alkalmazás abszolút <code>main.rw</code> útvonala. Sample: <code>/srv/rwlang/current/main.rw</code> .
<code>server.listen</code>	<code>127.0.0.1:8080</code>	Listener. Reverse proxy mögött tipikusan loopback; közvetlen TLS esetén publikus cím is lehet.
<code>server.insecure_dev_cookies</code>	<code>false</code>	Csak fejlesztési HTTP-hoz. Productionben maradjon <code>false</code> .
<code>tls.cert_file</code>	nincs	TLS certificate chain abszolút útvonala.
<code>tls.key_file</code>	nincs	TLS private key abszolút útvonala.
<code>tls.handshake_timeout_ms</code>	<code>5000</code>	TLS handshake felső ideje.
<code>tls.http_redirect_listen</code>	nincs	Opcionális HTTP listener HTTPS redirecthez.
<code>tls.public_host</code>	nincs	Canonical/public host a redirect és Host/Origin-validációhoz; trusted HTTPS reverse proxy módban is szükséges.

Reverse proxy mögött

Apache vagy Nginx mögött az RWLang tipikusan nem publikus interfészen hallgat:

```
[server]
app = "/srv/rwlang/current/main.rw"
listen = "127.0.0.1:8080"
insecure_dev_cookies = false

[tls]
public_host = "www.example.com"
```

A publikus TLS-t ilyenkor a proxy terminálja. Az RWLang oldali certificate/key elmarad, de a `tls.public_host` továbbra is kötelező trusted Host/Origin pinninghez. A `web.trusted_proxy_cidrs` határozza meg, mely upstreamtől fogadunk el forwarded client/proto metadata-t. TLS nélküli production listener ebben a módban csak loopback címen engedett.

21.4. [database], [redis] és [auth]

Kulcs	Default	Jelentés
<code>database.url_file</code>	nincs	DB URL-t tartalmazó secret fájl.
<code>database.allow_insecure</code>	<code>false</code>	Plaintext <code>database.url</code> nincs. Titkosítatlan remote DB kapcsolat explicit engedélyezése; productionben általában ne használd.
<code>redis.url_file</code>	nincs	Redis URL secret fájlból.
<code>redis.allow_insecure</code>	<code>false</code>	Insecure Redis explicit engedélyezése.
<code>auth.ldap_url</code>	nincs	LDAP/LDAPS endpoint.
<code>auth.ldap_search_base</code>	nincs	LDAP search base.
<code>auth.ldap_username_attribute</code>	<code>uid</code>	Login név LDAP attribútuma.
<code>auth.ldap_service_bind_dn_file</code>	nincs	Service bind DN secret fájlból.
<code>auth.ldap_service_bind_password_file</code>	nincs	Bind jelszó secret fájlból.
<code>auth.totp_secrets_file</code>	nincs	TOTP secret store fájl.
<code>auth.roles_file</code>	nincs	Külső role mapping fájl.
<code>auth.local_auth_db_url_file</code>	nincs	Local-auth identity store URL secret fájlból.
<code>auth.require_totp</code>	<code>false</code>	Sikeres első faktor után TOTP kötelező-e.
<code>auth.login_max_attempts</code>	8	Login rate-limit ablakon belüli maximum próbálkozás.
<code>auth.login_window_secs</code>	300	Login rate-limit ablak másodpercben.

Az LDAP és a local auth ne legyen két párhuzamos elsődleges azonosítási forrás ugyanabban a deploymentben. Az identity authority legyen egyértelmű; a Redis session-, cache- és rate-limit runtime state, nem felhasználói source of truth.

21.5. [web]

Kulcs	Default	Jelentés
<code>web.trusted_proxy_cidrs</code>	üres lista	Csak ezekről a címekről hiteles a forwarded client/proto metadata.
<code>web.allow_missing_origin</code>	<code>false</code>	Origin nélküli unsafe requestek policyja. Productionben fail-closed alapérték.
<code>web.cors_origins</code>	üres lista	Exact-origin allowlist. Wildcard credentials CORS-hoz nem használható.

<code>web.cors_allow_credentials</code>	<code>false</code>	Session cookie/credential cross-origin engedélyezése; CSRF policyval együtt értelmezendő.
---	--------------------	---

21.6. [storage] és [static_assets]

Kulcs	Beépített default	Jelentés
<code>storage.data_root</code>	nincs	AppFs runtime data gyökér. Sample: <code>/srv/rwlang/data</code> .
<code>storage.fs_mode</code>	<code>rwc</code>	AppFs capability mód. Csak a szükséges jog legyen engedélyezve.
<code>storage.max_upload_bytes</code>	16 MiB	Egy upload felső byte-limitje.
<code>storage.max_image_pixels</code>	40 000 000	Dekódolt kép pixel ceilingje.
<code>static_assets.root</code>	nincs	Read-only static root. Sample: <code>/srv/rwlang/current/public</code> .
<code>static_assets.url_prefix</code>	<code>/assets/</code>	A static namespace URL prefixe.
<code>static_assets.max_asset_bytes</code>	8 MiB	Egy static asset felső mérete.
<code>static_assets.max_age_secs</code>	300	Nem fingerprintelt asset cache ideje.
<code>static_assets.immutable_max_age_secs</code>	31536000	Fingerprintelt immutable asset cache ideje.
<code>static_assets.precompressed</code>	<code>true</code>	Előre generált Brotli/gzip változatok használata.

A `data_root` és a static root ne legyen ugyanaz a könyvtár: az egyik mutable alkalmazásállapot, a másik release-owned read-only artifact.

21.7. [lifecycle], [observability] és [logging]

Kulcs	Default	Jelentés
<code>lifecycle.health_live_path</code>	<code>/health/live</code>	Process liveness endpoint.
<code>lifecycle.health_ready_path</code>	<code>/health/ready</code>	Dependency-aware readiness endpoint.
<code>lifecycle.health_dependency_timeout_ms</code>	1000	Readiness dependency probe timeout.
<code>lifecycle.shutdown_grace_ms</code>	30000	Graceful shutdown drain idő.
<code>observability.metrics_listen</code>	nincs	Prometheus listener; sample: <code>127.0.0.1:9090</code> .
<code>observability.allow_public_metrics</code>	<code>false</code>	Publikus metrics listener explicit engedélye.
<code>observability.access_log</code>	<code>true</code>	HTTP access események logolása.
<code>logging.server_file</code>	nincs	Error/system log fájl.
<code>logging.access_file</code>	nincs	Access log fájl.
<code>logging.audit_file</code>	nincs	Security/user-activity audit log.
<code>logging.stderr</code>	<code>true</code>	Stderr log output.

A log parent directoryt az operátor hozza létre. Létező log target nem lehet symlink vagy nem-reguláris fájl. Behind-proxy módban a SIGHUP újrainyítja a logokat és tranzakciósan

újraépíti a domain/application hosting állapotot; a process-szintű beállításokhoz továbbra is restart kell.

21.8. [rate_limit] és [cache]

Kulcs	Default	Jelentés
<code>rate_limit.policies_file</code>	nincs	Route rate-limit policy fájl. Sample: <code>/usr/local/etc/rwlang/rate-limits.toml</code> .
<code>rate_limit.allow_memory</code>	false	In-memory fallback explicit engedélyezése. Több instance productionnél maradjon false.
<code>cache.max_ttl_secs</code>	3600	Public cache maximális route TTL-je.
<code>cache.max_entries</code>	10000	In-memory cache entry ceiling.
<code>cache.max_bytes</code>	64 MiB	In-memory cache teljes byte ceiling.
<code>cache.allow_memory</code>	false	Redis nélküli memory cache explicit engedélyezése.

Ha több `rwlang-server` példány fut, a session/cache/rate-limit közös állapotát Redisnek kell hordoznia; a memory fallback csak tudatos, tipikusan egy-instance környezetben elfogadható.

21.9. [limits]

A request- és runtime-limitek defense-in-depth szerepet töltenek be. A sample több értéket explicit megad, hogy az operator auditálható módon vállalja a production budgetet.

Kulcs	Beépített default	Sample / jelentés
<code>limits.max_header_bytes</code>	16 KiB	Sample ugyanennyi.
<code>limits.max_body_bytes</code>	256 KiB	Sample ugyanennyi; upload külön storage limitet is kap.
<code>limits.max_connections</code>	4096	Egyidejű kapcsolatok ceilingje.
<code>limits.max_requests_per_connection</code>	100	Keep-alive request ceiling.
<code>limits.read_timeout_ms</code>	5000	Request read timeout.
<code>limits.request_timeout_ms</code>	15000	Teljes request budget.
<code>limits.write_timeout_ms</code>	5000	Response write timeout.
<code>limits.max_header_count</code>	64	Header count ceiling.
<code>limits.max_form_fields</code>	64	Form mezők maximuma.
<code>limits.max_form_field_bytes</code>	8 KiB	Egy form mező byte ceiling.
<code>limits.max_instructions</code>	100000	Sample: 5000000. Interpreter budget; emelés auditálandó.
<code>limits.max_runtime_alloc_bytes</code>	32 MiB	Sample: 256 MiB. Runtime allocation ceiling.
<code>limits.session_ttl_secs</code>	28800	8 óra.
<code>limits.max_sessions</code>	100000	Session ceiling.
<code>limits.max_process_memory_bytes</code>	nincs	Sample: 1 GiB; process address-space limit.

`limits.resource_profiles_` nincs
`file`

Sample:
`/usr/local/etc/rwlang/`
`resource-profiles.toml.`

Miért külön a request, runtime és process limit?

A `max_body_bytes` a hálózati inputot korlátozza, a `max_runtime_alloc_bytes` egy request interpreter-allokációját, a `max_process_memory_bytes` pedig a teljes process address space-t. Egyik nem helyettesíti a másikat.

21.10. [cgroup]

A cgroup korlátok Linux production defense-in-depth réteget adnak a process-szintű limitek fölé.

Kulcs	Default	Jelentés
<code>cgroup.dir</code>	nincs	A már előkészített cgroup könyvtár abszolút útvonala.
<code>cgroup.memory_max_bytes</code>	nincs	Cgroup memória ceiling; a systemd sample ezt nem állítja.
<code>cgroup.memory_swap_max_bytes</code>	nincs	Swap ceiling; delegated cgroup deploymentnél opcionális.
<code>cgroup.cpu_percent</code>	nincs	CPU budget százalékban; delegated cgroup deploymentnél opcionális.
<code>cgroup.pids_max</code>	nincs	PID/thread ceiling; delegated cgroup deploymentnél opcionális.

A cgroup könyvtár és delegáció operátori feladat; a szerver nem rendszeradminisztrációs eszköz. A repository systemd baseline-jában a service manager a cgroup authority (`MemoryMax`, `CPUQuota`, `TasksMax`), ezért ott a `[cgroup]` section szándékosan nincs bekapcsolva. RWLang-oldali cgroup írást csak külön delegált, írható cgroup v2 könyvtárnál használj; a két authorityt ne konfiguráld egyszerre.

21.11. Production baseline

Egy tipikus reverse-proxy mögötti minimum konfiguráció:

```
[server]
app = "/srv/rwlang/current/main.rw"
listen = "127.0.0.1:8080"
insecure_dev_cookies = false

[tls]
public_host = "www.example.com"

[database]
url_file = "/run/secrets/rwlang/database-url"

[redis]
url_file = "/run/secrets/rwlang/redis-url"
```

```
[web]
trusted_proxy_cidrs = ["127.0.0.1/32", "::1/128"]
allow_missing_origin = false

[storage]
data_root = "/srv/rwlang/data"

[logging]
server_file = "/var/log/rwlang/server.log"
access_file = "/var/log/rwlang/access.log"
audit_file = "/var/log/rwlang/audit.log"
stderr = true

[limits]
resource_profiles_file = "/usr/local/etc/rwlang/resource-profiles.toml"
```

Ez csak baseline: auth, static, metrics, rate-limit, TLS és cgroup a deployment topológiájához igazítandó.

21.12. Automatikus source reload

A globális `[reload]` section az `enabled`, `poll_interval_ms` és `debounce_ms` alapértékeket adja. Multi-domain módban ezek `[domains.reload]` alatt domainenként felülírhatók. Egyetlen közös supervisor figyeli a compiler által visszaadott source graphot `mtime + size` alapján, beleértve az atomikus symlink deploymenthez szükséges logikai entrypoint útvonalat is. Sikertelen candidate esetén a régi generáció marad aktív; sikeres commit csak az érintett domaint cseréli, és invalidálja annak public-cache generationjeit. Karbantartáskor a `-no-source-reload` kapcsoló processz-szinten tiltja a figyelést.

21.13. Változtatási workflow

Production config módosításnál a könyv egyetlen ajánlott sorrendet használ:

1. módosítsd a version-controlled config template-et;
2. secretet külön file-ban módosíts, ne TOML-ban;
3. futtasd a `rwlang-server -check-config` ellenőrzést;
4. nézd meg a `-print-effective-config` secretmentes összefoglalót;
5. controlled restart;
6. ellenőrizd a live/ready endpointokat, logot, metricet és auditot.

Behind-proxy módban a `SIGHUP` csak korlátozott hosting reloadot végez. Nem általános folyamat-újrakonfigurálás. Az alkalmazásforrás változásához normál esetben sem `SIGHUP`, sem restart nem kell. A reload supervisor az entrypointot és a tranzitív modulgráfot figyeli. Csak sikeresen forduló domain-generációt cserél be tranzakción.

22. fejezet

Fejlesztői referencia és továbblépés

22.1. Melyik témát hol keresd?

Téma	Részletes projekt-dokumentáció
Routing, input, validáció	docs/hu/04-routing-input.md
Adatbázis és SQL	docs/hu/05-adatbazis.md
CRUD, tranzakció	docs/hu/06-crud.md
Auth és Redis	docs/hu/07-auth-redis.md
Route rate limiting	docs/hu/08-rate-limiting.md
JSON API és CORS	docs/hu/05-json-api-cors.md
Statikus assetek	docs/hu/06-static-assets.md
Public page cache	docs/hu/13-cache.md
HTTPS és proxy security	docs/hu/08-https-security.md
Upload/AppFs	docs/hu/09-upload-appfs.md
Migration	docs/hu/10-database-migrations.md
Formok	docs/hu/12-forms.md
Komponensek/layoutok	docs/hu/14-components-layouts.md
Security checklist	docs/hu/15-security-checklist.md
Dependency security	docs/19-dependency-security.md
Server config	docs/hu/15-server-config.md
Markdown	docs/hu/16-markdown-rich-text.md
Media library	docs/hu/17-media-library.md
Object authorization	docs/hu/20-object-authorization.md
Projektstruktúra/modulok	docs/hu/21-modules-slugs-project-layout.md
Modulok és névterek	docs/21-modules-and-namespaces.md
Aritmetika, F32 matematika, időmérés	docs/44-math-and-timing.md
String builtinok	docs/46-string-builtins.md
Typed string listák	docs/47-string-lists.md
Typed string dictionaryk	docs/48-string-dictionaries.md
Reguláris kifejezések	docs/49-regular-expressions.md
Domain objectek	docs/hu/22-domain-objects.md
Enumok	docs/hu/23-enums.md
Optimistic locking	docs/24-optimistic-locking.md
Business audit	docs/hu/25-business-audit-trail.md
Domain-validáció	docs/hu/26-domain-validation.md

Canonical URL	docs/hu/27-canonical-url.md
PRG/flash/conflict UX	docs/hu/28-prg-flash-conflict.md
Production deployment	docs/hu/29-production-deployment.md
Backup/restore/upgrade	docs/hu/30-backup-restore-upgrade-rollback. md
V1 átadási lista	docs/hu/31-v1-developer-guide.md
IPv6-ready outbound egress	docs/32-ipv6-egress.md
CLI és napi workflow	docs/hu/33-cli-workflow.md
Server TOML referencia	docs/hu/34-server-toml-reference.md
Production checklist és minták	docs/hu/35-production-checklist.md
Debian csomag és dpkg telepítés	docs/hu/36-debian-csomag.md

22.2. Napi fejlesztési ellenőrzőlista

1. Typed route/input és zárt választásnál enum.
2. User input nem kerül raw SQL-be vagy raw HTML-be.
3. Mutáció transactionben.
4. Sikeres POST után PRG redirect.
5. User/session dependent oldal nem public cache.
6. Secret csak `*_file` formában.
7. Release előtt `./verify.sh`, config check és migration verify.
8. Production után live/ready, log, metric és audit review.

A. függelék

Függelékek és production checklist

Ez a fejezet nem új nyelvi elemeket vezet be. Az előző fejezetekből összeálló, röviden használható üzemeltetési segédlet: tipikus könyvtárstruktúra, reverse proxy minták, systemd unit, logrotate és egy release előtti ellenőrzőlista. A minták kiindulópontok; a konkrét domaineiket, tanúsítványokat, jogosultságokat és erőforráslimiteket a saját környezethez kell igazítani.

A.1. Ajánlott production könyvtárstruktúra

Az RWLang helyben telepített szoftver, ezért a könyv következetesen a `/usr/local` hierarchiát használja a programhoz és a konfigurációhoz. Az alkalmazás állapota és a logok ettől külön élnek.

```
/usr/local/bin/  
  rclang-server  
  rclang-cli  
  
/usr/local/etc/rclang/  
  server.toml  
  rate-limits.toml  
  resource-profiles.toml  
  
/run/secrets/rclang/  
  database-url  
  redis-url  
  local-auth-database-url  
  
/srv/rclang/  
  current/                # current application release  
  releases/  
    2026-09-04.1/  
  data/                   # AppFs / durable application data  
  
/var/log/rclang/  
  server.log  
  access.log  
  audit.log
```

A release tartalom legyen lehetőleg read-only. A szervernek csak azokon a helyeken legyen írási joga, ahol valóban tartós állapotot vagy logot kell írnia. A secret fájl ne legyen az alkalmazás

release könyvtárában és ne kerüljön verziókezelésbe.

A.2. Apache reverse proxy teljes minimálminta

A példa egyetlen Apache reverse proxyt feltételez. A publikus TLS az Apache feladata, az RWLang pedig csak loopbacken figyel.

Az ehhez tartozó RWLang minimum:

```
[server]
listen = "127.0.0.1:8080"
insecure_dev_cookies = false

[tls]
public_host = "www.example.com"

[web]
trusted_proxy_cidrs = ["127.0.0.1/32", "::1/128"]
```

Reverse proxy production módban a certificate/key a proxyn marad, de a trusted `public_host` kell a Host/Origin pinninghez. A proxy feladata a hiteles X-Forwarded-Proto: https metadata előállítás.

```
<VirtualHost *:80>
    ServerName www.example.com
    Redirect permanent / https://www.example.com/
</VirtualHost>

<VirtualHost *:443>
    ServerName www.example.com

    SSLEngine on
    SSLCertificateFile \
        /etc/letsencrypt/live/www.example.com/fullchain.pem
    SSLCertificateKeyFile \
        /etc/letsencrypt/live/www.example.com/privkey.pem

    ProxyPreserveHost On
    ProxyPass      / http://127.0.0.1:8080/
    ProxyPassReverse / http://127.0.0.1:8080/

    RequestHeader set X-Forwarded-Proto "https"
</VirtualHost>
```

Tipikus Apache modulok: `proxy`, `proxy_http`, `headers` és `ssl`. Az RWLang trusted-proxy listája csak a tényleges proxyt foglalja magában. Loopback proxy esetén ez tipikusan `127.0.0.1/32` és szükség esetén `::1/128`. Internetméretű trusted proxy tartomány hibás konfiguráció.

A.3. Nginx reverse proxy teljes minimálminta

```
server {
    listen 80;
    server_name www.example.com;
```

```

        return 301 https://$host$request_uri;
    }

server {
    listen 443 ssl;
    server_name www.example.com;

    ssl_certificate
        /etc/letsencrypt/live/www.example.com/fullchain.pem;
    ssl_certificate_key
        /etc/letsencrypt/live/www.example.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header X-Forwarded-For $remote_addr;
    }
}

```

A példában az Nginx közvetlenül látja a klienst, ezért a forwarding címet maga állítja elő. Ne örökíts tovább internet felől érkező tetszőleges **X-Forwarded-*** értéket authorityként. Több proxy esetén a trust láncot explicit kell megtervezni.

A.4. systemd service minta

A repository teljes mintája: `examples/systemd/rwlang.service`. A lényegi forma:

```

[Unit]
Description=RWLang application
Wants=network-online.target
After=network-online.target

[Service]
Type=simple
User=rwlang
Group=rwlang
WorkingDirectory=/srv/rwlang/current
ExecStartPre=/usr/local/bin/rwlang-server \
    --config /usr/local/etc/rwlang/server.toml \
    --check-config
ExecStart=/usr/local/bin/rwlang-server \
    --config /usr/local/etc/rwlang/server.toml
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=2s
TimeoutStopSec=45s

MemoryMax=1G
MemorySwapMax=0
CPUQuota=200%
TasksMax=256
LimitNOFILE=8192
# Direct TLS 80/443 eseten add hozzá:
# AmbientCapabilities=CAP_NET_BIND_SERVICE

```

```
# CapabilityBoundingSet=CAP_NET_BIND_SERVICE
NoNewPrivileges=yes
PrivateTmp=yes
PrivateDevices=yes
ProtectSystem=strict
ProtectHome=yes
ProtectKernelTunables=yes
ProtectKernelModules=yes
ProtectControlGroups=yes
RestrictSUIDSGID=yes
LockPersonality=yes
ReadWritePaths=/srv/rwlang/data /var/log/rwlang

[Install]
WantedBy=multi-user.target
```

A systemd hard limit és az RWLang request/runtime resource policyja egymást kiegészítő védelmi réteg. A systemd baseline maga legyen a process cgroup authority; ilyenkor ne kapcsolod be mellette az RWLang [cgroup] író módját. Közvetlen 80/443 bindnál csak a CAP_NET_BIND_SERVICE capabilityt add, reverse proxy mögötti magas portnál ezt is hagyd el. A TimeoutStopSec legyen nagyobb, mint az alkalmazott graceful shutdown időablak. Behind-proxy módban a SIGHUP tranzakciósan újratöltheti a domain/application hosting állapotot, míg process-szintű konfigurációváltozás után továbbra is kontrollált restart kell; pusztán alkalmazásforrás-változást normál esetben az automatikus source-reload supervisor aktivál.

A.5. logrotate minta

```
/var/log/rwlang/server.log /var/log/rwlang/access.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 rwlang rwlang
    sharedscripts
    postrotate
        systemctl kill -s HUP --kill-who=main \
            rwlang.service >/dev/null 2>&1 || true
    endscript
}

/var/log/rwlang/audit.log {
    daily
    rotate 90
    compress
    delaycompress
    missingok
    notifempty
    create 0640 rwlang rwlang
    sharedscripts
    postrotate
        systemctl kill -s HUP --kill-who=main \
            rwlang.service >/dev/null 2>&1 || true
    endscript
}
```

```
endscript
}
```

A konkrét retention jogi és kockázati döntés. Az auditlog általában hosszabb megőrzést és központi továbbítást érdemel; a minta számai nem univerzális előírások.

A.6. Release előtti production gate

A következő lista szándékosan rövid és végrehajtható. A részletes indoklást az előző fejezetek adják.

1. **Forrás és build:** a release commit azonosított; `Cargo.lock` rögzített; a platform buildje `cargo build -locked -release -p rclang-server -p rclang-cli`; az alkalmazás forrása `rclang-cli` `check` ellenőrzésen átment.
2. **Migration:** production előtt `migrate status` és `migrate verify`; a jóváhagyott schema változás explicit `migrate apply`; utána újabb `verify`.
3. **Konfiguráció:** a valódi production `server.toml` config-checken átment. Az effective config redaktált kimenete reviewzható.
4. **Secrets:** DB/Redis/auth secret fájlok nem a repositoryban vannak, szűk filesystem jogosultsággal rendelkeznek, és nem kerülnek logba vagy release-recordba.
5. **HTTP edge:** a domain, TLS és Host kezelés helyes. A trusted-proxy lista kizárólag a tényleges proxykat tartalmazza.
6. **Auth:** production cookie policy aktív; local auth/LDAP konfiguráció ellenőrzött; admin/MFA recovery folyamat dokumentált.
7. **CSRF és CORS:** state-changing böngészős kérés CSRF-védett; CORS allowlist explicit; credentialed CORS esetén nincs wildcard origin.
8. **Adatbázis:** az alkalmazás DB user least privilege; pool/budget összhangban van a backend kapacitásával; fontos queryk indexelése ismert.
9. **Storage/upload:** AppFs root confinement és jogosultságok helyesek; upload byte/pixel limitek aktívak; release tartalom nem írható alkalmazásból.
10. **Redis/cache/rate limit:** multi-instance környezetben a közös security/runtime state Redisben van; Redis kiesési viselkedése ismert és tesztelt.
11. **Egress:** csak named outbound target engedélyezett; host/CIDR/port/TLS/timeout/méret policy explicit; secret nem kerül URL-be vagy logba.
12. **Erőforrások:** request/runtime limitek és process hard limitek konzisztensen be vannak állítva; cgroup authorityként systemd vagy külön delegált RWLang cgroup legyen, ne mindkettő.
13. **Observability:** server/access/audit log írható; request ID működik; metrics és readiness elérhető a megfelelő belső hálózatról; riasztási minimum definiált.
14. **Backup/recovery:** van friss, ellenőrzött DB + AppFs + identity backup; recovery manifest rendelkezésre áll; restore drill nem csak papíron létezik.
15. **Deploy/rollback:** a release artifact és checksum ismert; a régi release visszaállítható; schema-változásnál tisztázott, hogy app rollback vagy teljes restore lehetséges.
16. **Indítás utáni evidence:** liveness, readiness és legalább egy üzleti smoke kérés sikeres; logban nincs startup fallback vagy váratlan policy-hiba.

A.7. Gyors hibadiagnosztikai térkép

Tünet	Első hely, ahol nézd	Következő kérdés
Nem indul a service	config-check, server log	Config/secret/path/policy vagy dependency readiness?
502/504 a proxyban	proxy log, RWLang listener/readiness	Fut-e a processz, loopback port helyes-e, readiness zöld-e?
400/415	access log + route body-contract	Malformed input, rossz schema vagy rossz Content-Type érkezett?
401	auth/session audit esemény	Nincs session, lejárt, invalidálva vagy MFA hiányzik?
403	route/object authorization, CSRF/CORS	Ki az actor, melyik policy tagadta meg?
404/405/406	route map + request method/Accept	Létezik a route, jó methodot és elfogadható reprezentációt kér a kliens?
409	optimistic-lock/business state	Közben megváltozott a record, vagy a Changed query nulla sort módosított?
413/431	request hard limitek	Body/upload vagy header lépte túl a konfigurált limitet?
421/426	Host/proxy/TLS policy	Helyes a public host, trusted proxy és az effektív HTTPS?
422	named form rerender	Melyik mező typed decode/validation hibája áll fenn?
429	rate-limit metrics/audit	Melyik limiter és melyik identity/IP bucket fogyott el?
500	request ID alapján server log	Valódi belső invariáns/internal hiba történt?
503	readiness + server log	DB/Redis/auth/storage vagy request resource limit tette átmenetileg kiszolgálhatatlanná?
DB hiba	DB readiness + query környezet	Kapcsolat, timeout, schema drift, lock vagy constraint?
Fájl/media hiba	AppFs path/jogosultság/limit	Storage root írható, confinement és byte/pixel limit rendben?
Külső API hiba	egress policy + adapter log	DNS, CIDR, port, TLS, timeout vagy response limit zárt?
Log eltűnik rotation után	rw_log_fallback_total, service log	SIGHUP eljutott-e, új fájl jogosultsága megfelelő-e?

A.8. Mit vigyél magaddal productionbe?

A könyv végére a legfontosabb gyakorlati szabályok röviden:

- előbb legyen typed adat és explicit policy, csak utána HTTP felület;
- a stabil production policy `server.toml`-ban legyen, ne hosszú service flaglistában;
- a secret érték fájlból érkezzon, ne parancssorból és ne repositoryból;
- schema változás explicit operátori lépés, nem szerver-startup mellékhatás;
- proxy header csak trusted proxy felől authority;
- Redis runtime/security state, nem üzleti source of truth;
- DB és AppFs együtt is alkothat recovery szempontból egy üzleti állapotot;
- a backup addig nem bizonyított, amíg izolált restore drill nem sikerült;
- release előtt checkelj, deploy után mérj és smoke-tesztelj;
- ha egy külső adat vagy erőforrás nincs explicit korlátozva, a boundary még nincs kész.

Zárszó

A könyv célja, hogy a webfejlesztő a nyelvi elemeket ne elszigetelt szintaxisként, hanem egy működő webalkalmazás részeként tanulja meg. A részletes magyar, feature-szintű referencia a projekt `docs/hu/` könyvtárában található; ez a könyv a napi fejlesztési útvonalat adja.